

Практическое занятие 17: Установка Django, создание проекта и настройка

Цель занятия:

Создание и настройка проекта Django для системы управления мероприятиями, включая установку необходимых инструментов, создание виртуального окружения, инициализацию Git репозитория и выполнение первого коммита.

Задачи занятия

1. Установка Python и Git
 2. Установка и настройка среды разработки Visual Studio Code (VSCode)
 3. Создание и активация виртуального окружения
 4. Установка Django
 5. Создание нового проекта Django
 6. Изучение структуры проекта
 7. Настройка основных параметров проекта
 8. Применение миграций
 9. Запуск сервера разработки
 10. Настройка системы контроля версий (Git)
 11. Первый коммит и публикация в удалённый репозиторий
-

Задача 1: Установка Python и Git

1.1. Проверка установки Python

На Windows:

1. Откройте **Командную строку (Cmd)** или **PowerShell**:
 - Нажмите клавишу **Win**, введите `cmd` или `powershell` и нажмите **Enter**.
2. Введите команду:

```
python --version
```

или

```
python3 --version
```

На macOS/Linux:

1. Откройте **Терминал**.
2. Введите команду:

```
python3 --version
```

Если Python не установлен, скачайте и установите его с официального сайта:

<https://www.python.org/downloads/>.

1.2. Проверка установки Git

На Windows:

1. Откройте **Командную строку (Cmd)** или **PowerShell**.
2. Введите команду:

```
git --version
```

На macOS/Linux:

1. Откройте **Терминал**.
2. Введите команду:

```
git --version
```

Если Git не установлен, скачайте и установите его с официального сайта: <https://git-scm.com/downloads>.

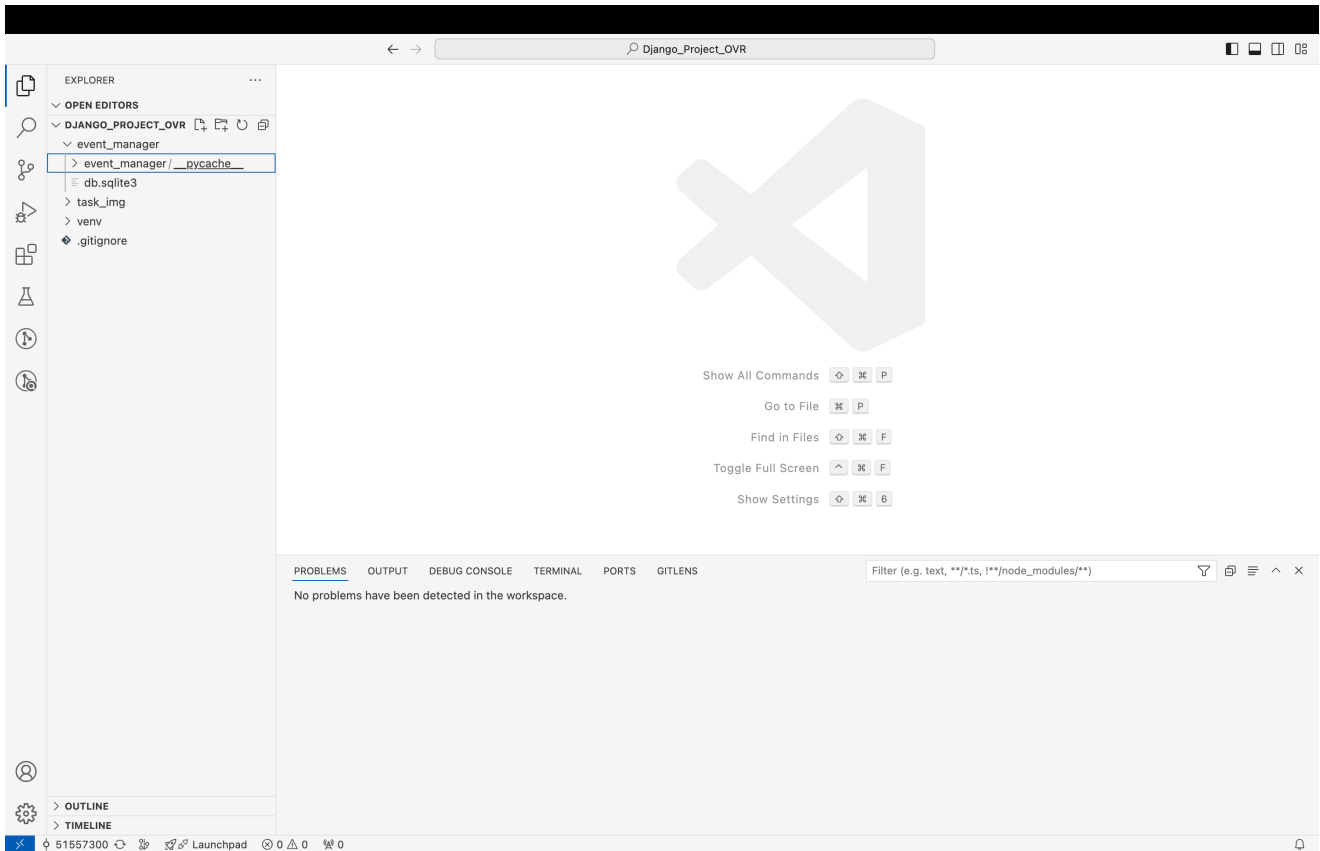
Задача 2: Установка и настройка среды разработки Visual Studio Code (VSCode)

2.1. Скачивание и установка VSCode

1. Перейдите на официальный сайт VSCode: <https://code.visualstudio.com/>
2. Скачайте установщик для вашей операционной системы.

3. Запустите установщик и следуйте инструкциям:

- **Windows:** Рекомендуется поставить галочку "Add to PATH" для удобства использования из терминала.
- **macOS/Linux:** Следуйте инструкциям на экране.



Внешний вид Visual Studio Code

2.2. Установка необходимых расширений

1. Откройте VSCode.

2. Перейдите в раздел **Extensions**:

- **Русский:** Нажмите на иконку **Расширения** в боковой панели или используйте сочетание клавиш `Ctrl+Shift+X` (Windows) / `Cmd+Shift+X` (macOS).
- **English:** Click on the **Extensions** icon in the sidebar or use the shortcut `Ctrl+Shift+X` (Windows) / `Cmd+Shift+X` (macOS).

3. Установите следующие расширения:

- **Python** от Microsoft
- **GitLens** (для улучшенной работы с Git)
- **Django** (для поддержки синтаксиса Django)
- **Pylance** (для улучшенной подсказки кода)

Задача 3: Создание и активация виртуального окружения

Виртуальное окружение изолирует зависимости вашего проекта от других проектов и системных пакетов.

3.1. Создание папки для проекта

1. Откройте **VSCode**.
2. Нажмите на **File > Open Folder...**:
 - **Русский:** **Файл > Открыть папку...**
 - **English:** **File > Open Folder...**
3. Выберите директорию, где хотите создать проект, или создайте новую папку, например, `event_manager`.
 - *Можно на рабочем столе*
4. Нажмите кнопку для открытия папки в VSCode.

3.2. Создание виртуального окружения

1. Откройте **Терминал** в VSCode:
 - **Русский:** Нажмите **Вид > Терминал** или используйте сочетание клавиш `Ctrl+``
 - **English:** Click **View > Terminal** or use the shortcut `Ctrl+``
2. В терминале убедитесь, что вы находитесь в корневой папке проекта (`event_manager`).
3. Введите команду для создания виртуального окружения:
 - **На Windows:**

```
python -m venv venv
```

- **На macOS/Linux:**

```
python3 -m venv venv
```

4. После выполнения команды в папке проекта появится новая директория `venv`.

3.3. Активация виртуального окружения

1. В терминале введите команду для активации виртуального окружения:
 - **На Windows:**

```
venv\Scripts\activate
```

- На macOS/Linux:

```
source venv/bin/activate
```

2. После активации вы увидите префикс `(venv)` перед строкой терминала, что означает, что виртуальное окружение активно.

3.4. Переходы между папками через терминал

Здесь вы можете ознакомиться с основными командами перемещения между папками в терминале. Выполнять их на данном шаге необязательно.

- **Просмотр содержимого текущей директории:**
 - **Русский:** Используйте команду `ls` (macOS/Linux) или `dir` (Windows).

```
ls          # macOS/Linux
dir         # Windows
```

- **Переход в другую директорию:**

```
cd название_папки
```

- **Переход на уровень выше:**

```
cd ..
```

- **Создание новой директории:**

```
mkdir имя_папки
```

- **Переход в корневую директорию:**

```
cd /
```

Задача 4: Установка Django

4.1. Обновление pip (опционально)

Рекомендуется обновить менеджер пакетов `pip` до последней версии:

```
pip install --upgrade pip
```

Примечание. В ходе тестирования на учебных компьютерах в РТУ МИРЭА обнаружилась ошибка верификации SSL-сертификатов (`SSLError`). В случае ошибки добавьте параметры командной строки:

```
pip install --trusted-host pypi.org --trusted-host pypi.python.org --  
trusted-host files.pythonhosted.org --upgrade pip
```

4.2. Установка Django

В активированном виртуальном окружении выполните команду:

```
pip install django
```

В случае ошибки с SSL-сертификатами (добавляйте данные параметры и в дальнейшем при установке пакетов или модулей):

```
pip install --trusted-host pypi.org --trusted-host pypi.python.org --  
trusted-host files.pythonhosted.org django
```

4.3. Проверка установки Django

Убедитесь, что Django установлен корректно, выполнив команду:

```
django-admin --version
```

Вы должны увидеть установленную версию Django, например `4.2.5`.

Задача 5: Создание нового проекта Django

5.1. Создание проекта

В терминале, находясь в директории `event_manager`, выполните команду:

```
django-admin startproject event_manager .
```

Примечание: Точка `.` в конце команды указывает на создание проекта в текущей директории.

5.2. Структура проекта

После выполнения команды ваша структура проекта будет выглядеть следующим образом:

```
ваш_репозиторий/  
├─ manage.py  
├─ event_manager/  
│   ├─ __init__.py  
│   ├─ settings.py  
│   ├─ urls.py  
│   ├─ asgi.py  
│   └─ wsgi.py  
└─ venv/
```

Задача 6: Изучение структуры проекта

- `manage.py` : утилита командной строки для взаимодействия с проектом.
- `event_manager/` : пакет с настройками и конфигурацией проекта.
 - `__init__.py` : обозначает директорию как пакет Python.
 - `settings.py` : конфигурация проекта.
 - `urls.py` : маршрутизация URL-адресов.
 - `asgi.py` и `wsgi.py` : настройки для развертывания приложения.

Задача 7: Настройка проекта

7.1. Открытие файла `settings.py`

1. В **VSCode** откройте файл `event_manager/settings.py`.
2. Найдите и измените следующие параметры:

```
LANGUAGE_CODE = 'ru-ru'  
  
TIME_ZONE = 'Europe/Moscow'
```

7.2. Настройка статических и медиа файлов

В конце файла `settings.py` добавьте следующие настройки:

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Задача 8: Применение миграций

Создайте необходимые таблицы в базе данных, выполнив команду:

```
python manage.py makemigrations

python manage.py migrate
```

Описание: Эти команды применяют все миграции, создавая необходимые таблицы в базе данных SQLite по умолчанию.

Задача 9: Запуск сервера разработки

9.1. Запуск сервера

В терминале выполните команду:

```
python manage.py runserver
```

Примечание: Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3`.

9.2. Проверка работы сервера

1. Откройте браузер.
2. Перейдите по адресу: <http://127.0.0.1:8000/>
3. Вы должны увидеть приветственную страницу Django: "The install worked successfully! Congratulations!"



Установка прошла успешно! Поздравляем!

Посмотреть [примечания к выпуску](#) для Django 5.1

Вы видите данную страницу, потому что указали `DEBUG=True` в файле настроек и не настроили ни одного обработчика URL-адресов.

django

```
System check identified 1 issue (0 silenced).
October 23, 2024 - 20:43:09
Django version 5.1.2, using settings 'event_manager.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

```
[23/Oct/2024 20:43:22] "GET / HTTP/1.1" 200 12451
```

□

Успешный запуск сервера Django

9.3. Остановка сервера разработки

Чтобы остановить работающий сервер Django, выполните следующие действия:

Через терминал:

1. Перейдите в терминал, где запущен сервер (например, встроенный терминал VSCode).
 2. Нажмите сочетание клавиш Ctrl + C (Windows) или Cmd + C (macOS/Linux).
- **Описание:** Это прерывает выполнение текущей команды и останавливает сервер разработки Django.

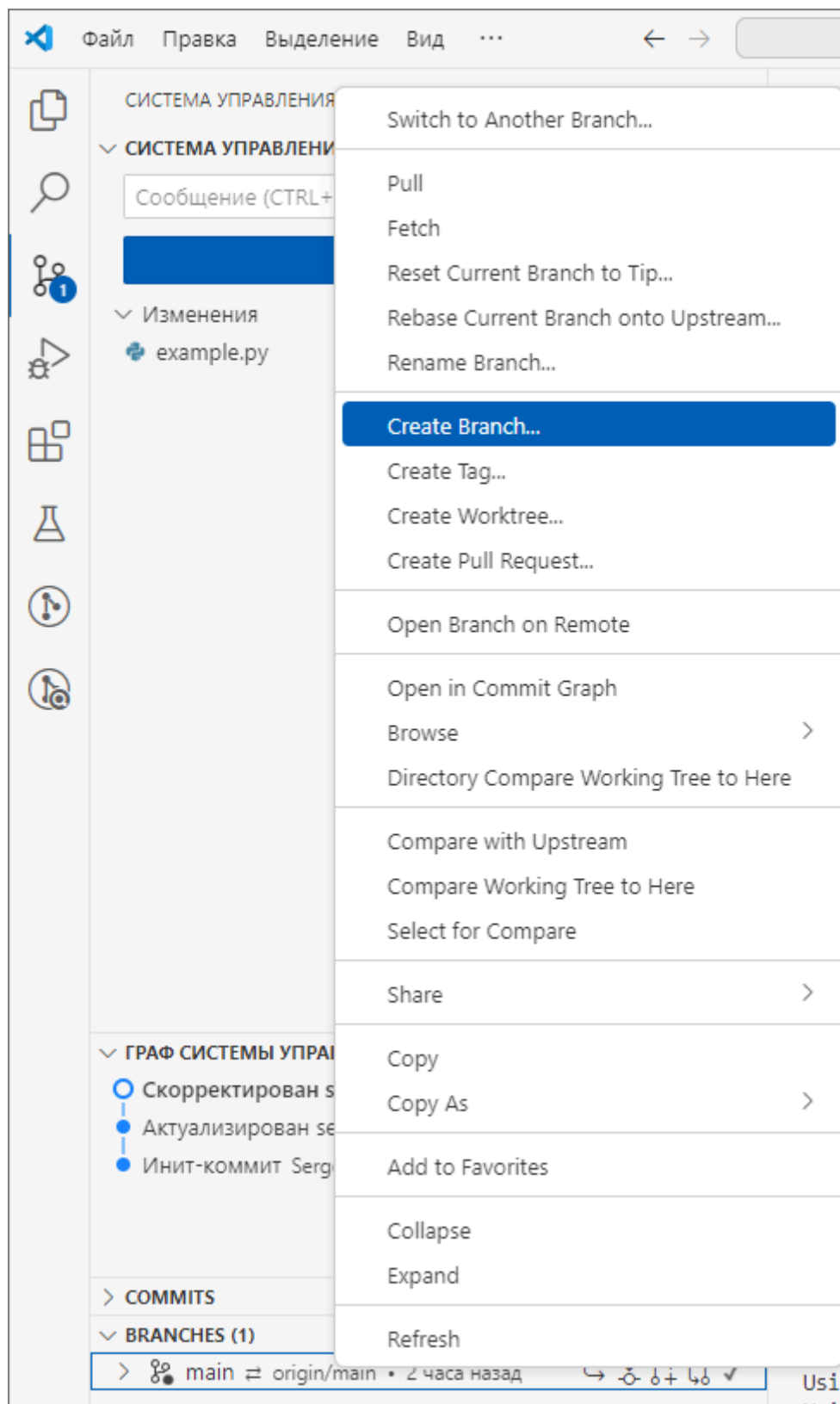
Через меню IDE (Visual Studio Code):

1. В терминале VSCode вы можете использовать кнопку **Trash** (иконка с мусорным ведром) для прерывания запущенной команды.
 2. Перейдите в меню **Terminal > Kill Terminal**:
 - **Русский:** Перейдите в меню **Терминал > Завершить терминал**.
 - **English:** Go to **Terminal > Kill Terminal**.
 3. Это также остановит выполнение сервера Django.
-

Задача 10: Настройка системы контроля версий (Git)

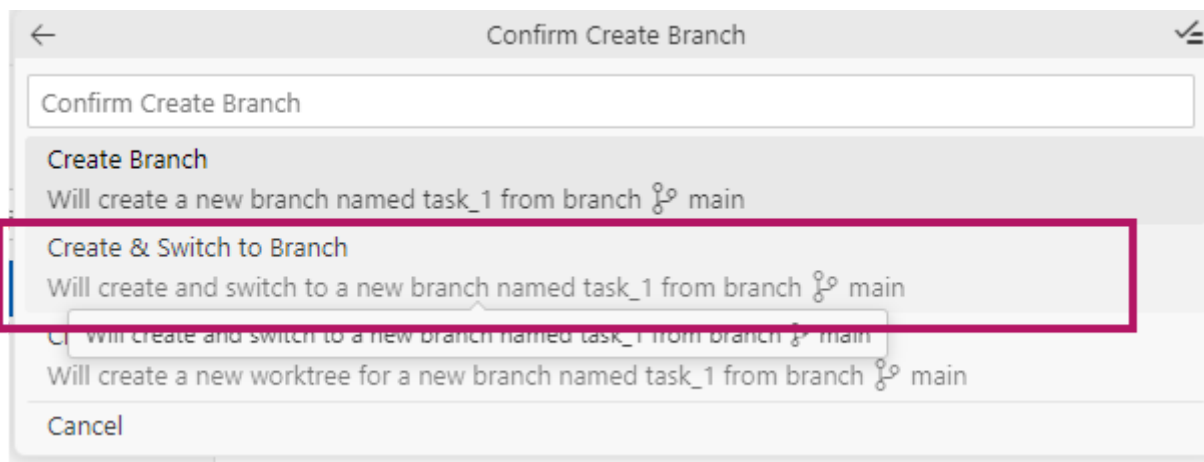
10.1. Сделайте ветку для выполнения первого задания

1. В **Системе управления версиями** откройте вкладку **Branches** и создайте ветку с названием **task_1** (правой кнопкой мыши по ветке **main** - **Create Branch...**)



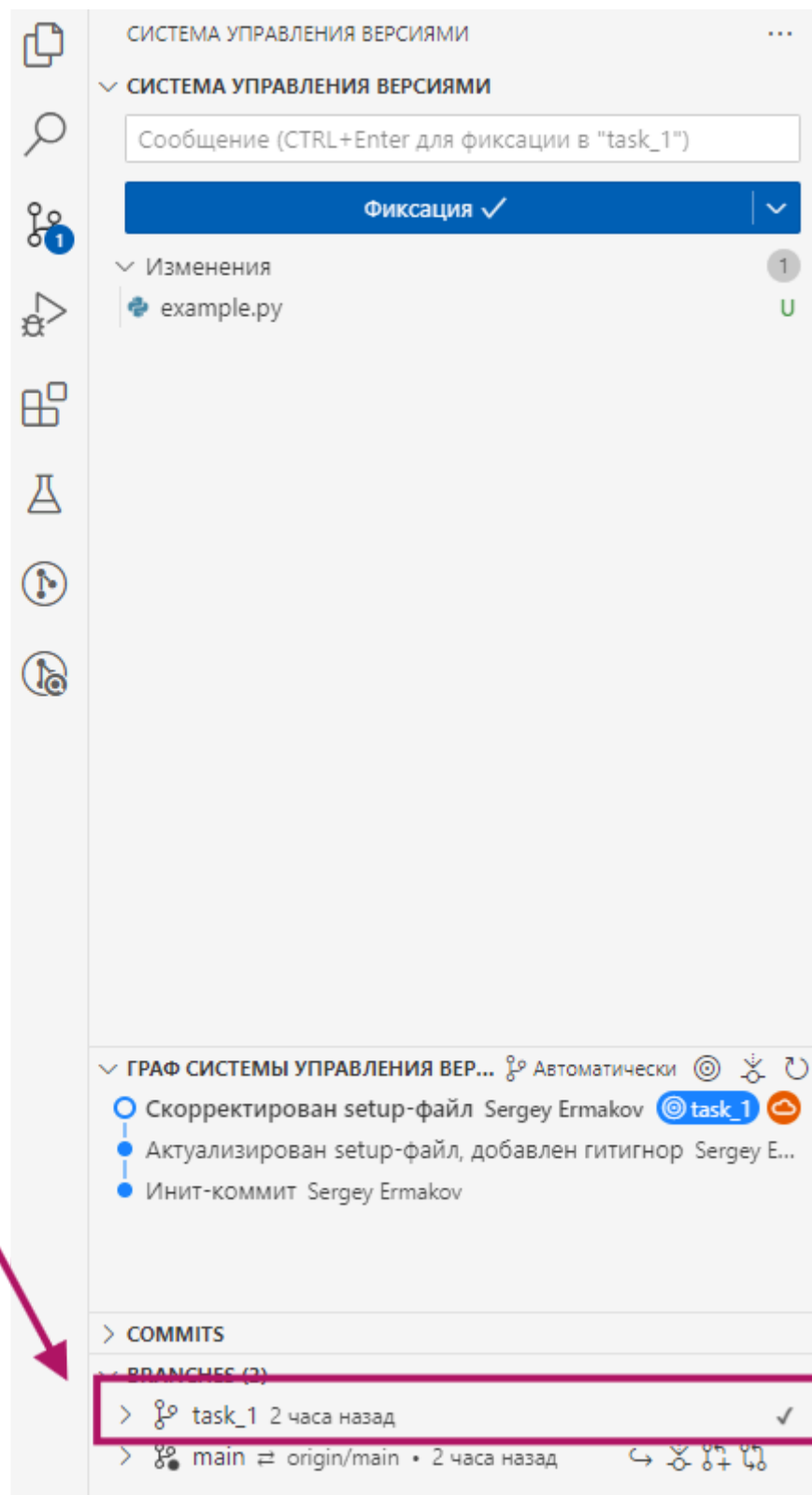
Создание ветки в VSC

2. После ввода названия выберите **Create & Switch to Branch** для переключения в ветку.



Создание и переключение на ветку в VSC

Убедитесь, что ветка переключилась - напротив task_1 должна стоять галочка ✓

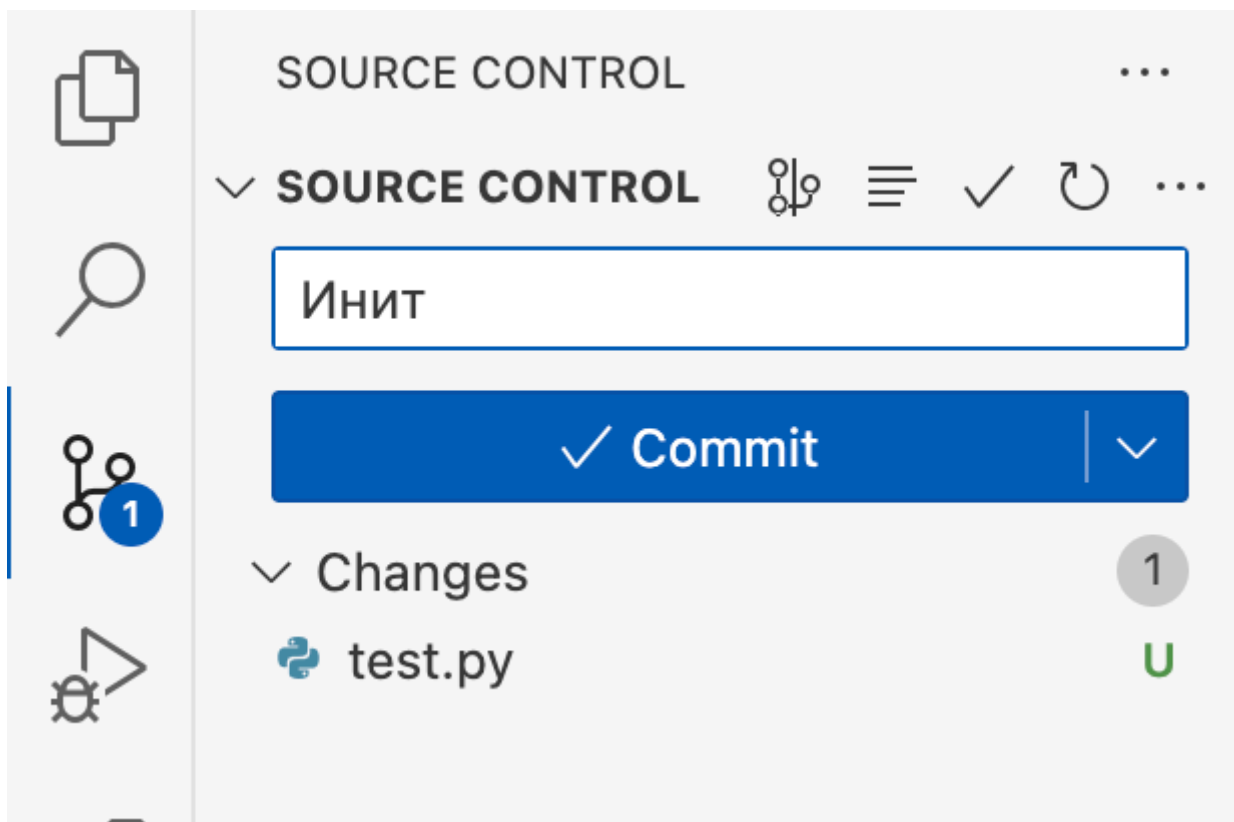


Активная ветка *task_1*

10.2. Сделайте первый коммит

Добавлять коммиты можно и через интерфейс VSCode.

Для правильного добавления нужно ввести сообщение коммита, нажать на **кнопку** Commit. В случае предупреждения рекомендуется согласиться на добавление всех файлов в индекс Git.



Коммит в VSCode

Другой способ: через консоль

Это не нужно делать, если вы сделали коммит с помощью интерфейса VSC.

1. Добавьте все файлы в индекс Git (**выполнять нужно из корневой папки**):

```
git add .
```

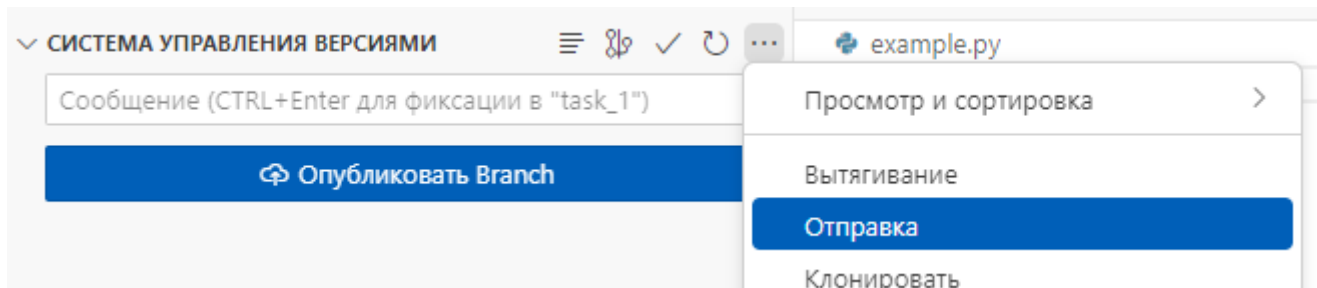
2. Выполните коммит с сообщением:

```
git commit -m "Инит: настройка проекта Django Event Manager"
```

Описание: Этот коммит сохраняет текущую версию проекта в истории Git.

Задача 11: Первый коммит и публикация в удалённый репозиторий

11.1. Отправка коммитов в удалённый репозиторий



Отправка в VSCode

В первую очередь, ветку нужно опубликовать. В интерфейсе появится кнопка **Опубликовать**. Для отправки изменений вы можете после публикации нажать на ту же кнопку (ее статус поменяется на синхронизацию изменений), или выбрать в дополнительном меню **Отправку**.

Другой способ: через консоль

Это не нужно делать, если вы сделали коммит с помощью интерфейса VSC.
Выполните команду:

```
git push -u origin main
```

Описание: Эта команда отправляет ваш локальный коммит в удалённый репозиторий и устанавливает связь между локальной веткой `main` и удалённой веткой `main`.

11.2. Проверка репозитория

1. Перейдите на страницу вашего репозитория.
2. Убедитесь, что все файлы вашего проекта загружены корректно.

Дополнительное задание

Для тех студентов, кто завершил основную часть раньше, предлагается выполнить следующие задания:

1. Исследование настроек проекта

- **Изучите файлы проекта:**
 - Откройте и ознакомьтесь с файлами `settings.py`, `urls.py` и `wsgi.py`.
- **Изменение параметра `DEBUG`:**
 - В `settings.py` измените значение `DEBUG` на `False` и попробуйте запустить сервер.

- Обратите внимание на изменения в поведении приложения, такие как отключение детальных ошибок.

2. Установка дополнительных пакетов

- **Установка `django-environ`:**

```
pip install django-environ
```

- **Настройка использования переменных окружения:**

1. Создайте файл `.env` в корневой директории проекта и добавьте переменные, например:

```
DEBUG=True
SECRET_KEY=your-secret-key
ALLOWED_HOSTS=127.0.0.1,localhost
```

2. В `settings.py` импортируйте и используйте `django-environ` для загрузки переменных:

```
import os
import environ

# Инициализируем объект Env
env = environ.Env(
    DEBUG=(bool, False) # Указываем тип переменной DEBUG и
    значение по умолчанию
)

# Определяем путь к корневой директории проекта
BASE_DIR =
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# Читаем переменные из .env файла
environ.Env.read_env(os.path.join(BASE_DIR, '.env'))

# Применяем переменные из .env
DEBUG = env('DEBUG')
SECRET_KEY = env('SECRET_KEY')
```

3. Работа с Git

- **Создание новой ветки:**


```
git checkout -b feature/setup-templates
```

- **Внесение изменений и коммит:**

- Добавьте новые шаблоны или измените существующие.

```
git add .  
git commit -m "Add base template and update navigation"
```

- **Слияние ветки с основной веткой:**

```
git checkout main  
git merge feature/setup-templates  
git push origin main
```

4. Дополнительная работа с удаленным репозиторием

- **Создание README.md:**

- Создайте файл `README.md` в корневой директории проекта и добавьте описание проекта, инструкции по установке и запуску.

- **Использование Issues и Pull Requests:**

- Создайте Issue для новой функциональности или исправления ошибки.
- Создайте Pull Request для внесённых изменений.

Советы и рекомендации

- **Регулярно коммитьте изменения:**

- Делайте коммиты после каждого значимого изменения в проекте.

- **Используйте ветвление в Git:**

- Для разработки новых функций используйте отдельные ветки, чтобы избежать конфликтов в основной ветке.

- **Следите за безопасностью:**

- Никогда не публикуйте секретные ключи и пароли в публичных репозиториях.

- **Документируйте проект:**

- Ведите README-файл с описанием проекта, его установкой и использованием.

Дополнительные советы

- **Установка дополнительных пакетов:**

Устанавливайте необходимые пакеты внутри активированного виртуального окружения:

```
pip install имя_пакета
```

- **Обновление requirements.txt :**

Чтобы сохранить список установленных пакетов:

```
pip freeze > requirements.txt
```

- **Установка пакетов из requirements.txt :**

На другом компьютере или после клонирования репозитория:

```
pip install -r requirements.txt
```

Пример кода и команд

- **Создание виртуального окружения и установка Django:**

```
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # macOS/Linux
pip install django
```

- **Создание проекта:**

```
django-admin startproject event_manager .
```

- **Запуск сервера разработки:**

```
python manage.py runserver
```

- **Настройка settings.py :**

```
# event_manager/settings.py

LANGUAGE_CODE = 'ru-ru'
TIME_ZONE = 'Europe/Moscow'
```

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Заключение

Поздравляем! Вы успешно:

- Установили Python и Django.
- Создали и активировали виртуальное окружение.
- Создали новый проект Django для системы управления мероприятиями.
- Настроили основные параметры проекта.
- Запустили сервер разработки и проверили работу приложения.
- Настроили систему контроля версий для вашего проекта.
- Опубликовали проект в удалённый репозиторий.

Вы готовы к дальнейшему изучению и разработке на Django. В следующем практическом занятии мы будем создавать первое приложение и знакомиться с базовой структурой сайта на Django.

Если у вас возникли вопросы или трудности на каком-либо этапе:

- Проверьте ошибки в терминале; они часто содержат полезную информацию.
- Убедитесь, что вы активировали виртуальное окружение перед установкой пакетов или запуском сервера.
- Обратитесь к документации Django: <https://docs.djangoproject.com/>
- Задайте вопрос преподавателю или обсудите с одногруппниками.

Успехов в разработке!

Вопросы для обсуждения

- Почему важно использовать виртуальное окружение при разработке Django-проекта?
- Виртуальное окружение изолирует зависимости вашего проекта от других проектов и системных пакетов. Это позволяет устанавливать специфичные версии библиотек, необходимые именно для данного проекта, без риска

конфликтов с другими проектами. Кроме того, виртуальное окружение облегчает переносимость проекта между разными системами и упрощает управление зависимостями, обеспечивая чистую и контролируемую среду разработки.

- **Как Git помогает в управлении проектом и почему стоит инициализировать репозиторий с самого начала?**
 - Git является системой контроля версий, которая позволяет отслеживать изменения в коде, работать над проектом совместно с другими разработчиками и восстанавливать предыдущие версии при необходимости. Инициализация репозитория Git с самого начала проекта обеспечивает историю изменений, упрощает управление различными ветками разработки и помогает избежать потери данных. Это также облегчает процесс развертывания и интеграции с удалёнными репозиториями.
 - **Что делает команда `django-admin startproject event_manager .` и почему в конце используется точка?**
 - Команда `django-admin startproject event_manager .` создаёт новый Django-проект с именем `event_manager` в текущей директории. Точка `.` в конце команды указывает Django создать файлы проекта непосредственно в текущей папке, а не в новой поддиректории с именем проекта. Это удобно, когда вы уже находитесь в нужной директории и хотите сразу начать работу без создания дополнительной вложенности каталогов.
-

Полезные ресурсы

- [Документация Django](#)
- [Документация Git](#)
- [Django Templates](#)
- [Django Static Files](#)

Практическое занятие 18: Создание приложения `events` и базовая структура сайта

Цель занятия:

Создать новое приложение `events` внутри проекта Django, настроить его интеграцию с проектом, разработать базовые маршруты и представления, а также создать и подключить шаблоны для отображения страниц. Дополнительно выполнить задания по созданию приложений `about` и `contact`.

Задачи занятия

1. Создание приложения `events`
 2. Регистрация приложения в проекте
 3. Настройка маршрутов приложения
 4. Создание представлений и шаблонов для `events`
 5. Выполнение практического упражнения. Изменение сообщения на домашней странице
 6. Выполнение дополнительных заданий. Создание приложений `about` и `contact`
-

Задача 1: Создание приложения `events`

1.1. Открытие проекта в Visual Studio Code (VSCode)

1. Запустите VSCode:
 - **Русский:** Запустите Visual Studio Code.
 - **English:** Open Visual Studio Code.
2. Откройте папку проекта:
 - **Русский:** Перейдите в меню **Файл > Открыть папку...**
 - **English:** Go to **File > Open Folder...**
 - Выберите директорию вашего проекта `Django_Project_OVR/` и нажмите **Открыть (Open)**.

1.2. Создание приложения `events` :

1. Откройте терминал в VSCode:

- **Русский:** Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+`` .
- **English:** Go to **View > Terminal** or use the shortcut `Ctrl+`` .

2. Выполните команду создания приложения:

```
python manage.py startapp events
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3` .

3. Обзор структуры приложения:

После выполнения команды структура проекта будет выглядеть следующим образом:

```
Django_Project_OVR/  
|  
|─ event_manager/  
|   |─ manage.py  
|   |─ event_manager/  
|   |   |─ __init__.py  
|   |   |─ settings.py  
|   |   |─ urls.py  
|   |   |─ asgi.py  
|   |   └─ wsgi.py  
|   |─ events/  
|   |   |─ __init__.py  
|   |   |─ admin.py  
|   |   |─ apps.py  
|   |   |─ models.py  
|   |   |─ tests.py  
|   |   |─ views.py  
|   |   └─ urls.py  
|   └─ venv/
```

Самостоятельно в ходе реализации практического задания вы добавите:

```
|   |─ templates/  
|   |   └─ events/  
|   |       └─ home.html  
|   |─ static/  
|   |   └─ events/
```

```
| | |
| | |   └─ css/
| | |   └─ styles.css
```

Задача 2: Регистрация приложения в проекте

1. Откройте файл настроек `settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

2. Добавьте приложение `events` в список `INSTALLED_APPS`:

Найдите раздел `INSTALLED_APPS` и добавьте `'events'`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'events', # Добавлено приложение events  
]
```

3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.

Задача 3: Настройка маршрутов приложения

3.1. Создание файла `urls.py` в приложении `events`:

1. Создайте файл `urls.py`:

- В папке `events/` создайте новый файл с именем `urls.py`.

2. Добавьте маршруты в `events/urls.py`:

Вставьте следующий код:

```
from django.urls import path  
from . import views  
  
urlpatterns = [
```

```
path('', views.home, name='home'),  
]
```

3.2. Подключение маршрутов приложения к проекту:

1. Откройте файл `event_manager/urls.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `urls.py`.

2. Импортируйте `include` и добавьте маршрут для приложения `events`:

Обновите содержимое файла следующим образом:

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls')), # Подключение маршрутов  
    приложения events  
]
```

3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.

Задача 4: Создание представлений и шаблонов для `events`

4.1. Создание функции `home` в `events/views.py`:

1. Откройте файл `events/views.py`:

- В папке `events/` откройте файл `views.py`.

2. Добавьте функцию `home`:

Замените или добавьте следующую функцию:

```
from django.shortcuts import render  
  
def home(request):  
    return render(request, 'events/home.html')
```

3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

4.2. Создание шаблона `home.html` :

1. Создайте директорию для шаблонов:

- В корневой директории проекта (на одном уровне с `manage.py`) создайте папку `templates` , если она ещё не существует.
- Внутри `templates/` создайте папку `events` .

2. Создайте файл `home.html` :

- В папке `templates/events/` создайте файл `home.html` .

3. Добавьте содержимое в `home.html` :

Вставьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
  <title>Главная страница</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Добро пожаловать в систему управления мероприятиями!</h1>
</body>
</html>
```

4.3. Создание файла стилей `styles.css` :

1. Создайте файл `styles.css` :

- В папке `static/events/css/` создайте файл `styles.css` .

2. Добавьте следующий CSS-код для стилизации страницы:

```
body {
  background-color: #f0f0f0;
  font-family: Arial, sans-serif;
  text-align: center;
  padding-top: 50px;
}

h1 {
  color: #333333;
}
```

3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

4.4. Настройка путей к шаблонам и статическим файлам в `settings.py` :

1. Откройте файл `event_manager/settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

2. Убедитесь, что параметр `'DIRS'` настроен следующим образом:

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

3. Настройка статических файлов:

Убедитесь, что следующие параметры присутствуют и настроены:

```
STATIC_URL = '/static/'  
STATICFILES_DIRS = [BASE_DIR / 'static']
```

4. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

Задача 5: Запуск и проверка работы приложения

1. Активируйте виртуальное окружение:

- На Windows:

```
venv\Scripts\activate
```

- На macOS/Linux:

```
source venv/bin/activate
```

2. Запустите сервер разработки:

```
python manage.py runserver
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3`.

3. Проверьте отображение страницы:

- Откройте веб-браузер.
- Перейдите по адресу: <http://127.0.0.1:8000/>
- Вы должны увидеть страницу с заголовком "Добро пожаловать в систему управления мероприятиями!" и применёнными стилями из `styles.css`.

4. Остановка сервера разработки:

- В терминале, где запущен сервер, нажмите сочетание клавиш `Ctrl + C` (Windows) или `Cmd + C` (macOS/Linux).

Задача 6: Практическое упражнение

Изменение сообщения на домашней странице:

1. Откройте файл `home.html`:

- В папке `templates/events/` откройте файл `home.html`.

2. Измените текст в `<h1>`:

- Найдите строку `<h1>Добро пожаловать в систему управления мероприятиями!`
`</h1>` и измените её на:

```
<h1>Добро пожаловать в нашу новую систему управления мероприятиями!  
</h1>
```

- Добавьте строку:

```
<p>Скоро здесь вы сможете создавать мероприятия и регистрироваться  
на них.</p>
```

3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

4. Запустите сервер и проверьте изменения:

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/> и убедитесь, что сообщение изменилось.

Дополнительные задания: Создание приложений `about` и `contact`

Цель:

Расширить функциональность проекта, добавив приложения `about` и `contact`, которые будут содержать страницы "О нас" и "Контакты" соответственно.

Задачи:

1. Создание приложений `about` и `contact`.
 2. Регистрация приложений в проекте.
 3. Настройка маршрутов для новых приложений.
 4. Создание представлений и шаблонов для страниц "О нас" и "Контакты".
 5. Реализация формы обратной связи на странице "Контакты".
-

Задание 1: Создание приложения `about`

1.1. Создание приложения `about`:

1. Откройте терминал в VSCode:

- **Русский:** Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+``.
- **English:** Go to **View > Terminal** or use the shortcut `Ctrl+``.

2. Выполните команду создания приложения:

```
python manage.py startapp about
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3`.

1.2. Регистрация приложения `about` в проекте:

1. Откройте файл `event_manager/settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

2. Добавьте приложение `about` в список `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    ...  
    'events',
```

```
'about', # Добавлено приложение about  
]
```

3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

1.3. Настройка маршрутов для приложения `about`:

1. Создайте файл `urls.py` в приложении `about`:

- В папке `about/` создайте новый файл с именем `urls.py`.

2. Добавьте маршруты в `about/urls.py`:

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.about, name='about'),  
]
```

3. Подключите маршруты приложения `about` к проекту:

- Откройте файл `event_manager/urls.py`:

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls')),  
    path('about/', include('about.urls')), # Подключение маршрутов  
                                           приложения about  
]
```

4. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

1.4. Создание представления и шаблона для страницы "О нас":

1. Создайте функцию `about` в `about/views.py`:

```
from django.shortcuts import render  
  
def about(request):  
    return render(request, 'about/about.html')
```

2. Создайте шаблон `about.html`:

- **Создайте директорию для шаблонов:**
 - В папке `templates/` создайте папку `about`.
- **Создайте файл `about.html` в `templates/about/`:**

```
<!DOCTYPE html>
<html>
<head>
  <title>О нас</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>О нашей компании</h1>
  <p>Информация о компании и команде.</p>
</body>
</html>
```

3. Сохраните изменения и проверьте страницу:

- Запустите сервер:

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/about/>
- Убедитесь, что страница "О нас" отображается корректно.

Задание 2: Создание приложения `contact`

2.1. Создание приложения `contact`:

1. Выполните команду создания приложения:

```
python manage.py startapp contact
```

- **Примечание:** Если вы используете macOS/Linux и команда `python` не работает, попробуйте `python3`.

2.2. Регистрация приложения `contact` в проекте:

1. Откройте файл `event_manager/settings.py`:

- Перейдите в папку `event_manager/` внутри проекта.
- Откройте файл `settings.py`.

2. Добавьте приложение `contact` в список `INSTALLED_APPS` :

```
INSTALLED_APPS = [  
    ...  
    'events',  
    'about',  
    'contact', # Добавлено приложение contact  
]
```

3. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

2.3. Настройка маршрутов для приложения `contact` :

1. Создайте файл `urls.py` в приложении `contact` :

- В папке `contact/` создайте новый файл с именем `urls.py`.

2. Добавьте маршруты в `contact/urls.py` :

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.contact, name='contact'),  
]
```

3. Подключите маршруты приложения `contact` к проекту:

- Откройте файл `event_manager/urls.py` :

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls')),  
    path('about/', include('about.urls')),  
    path('contact/', include('contact.urls')), # Подключение  
маршрутов приложения contact  
]
```

4. Сохраните изменения:

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS).

2.4. Создание представления и шаблона для страницы "Контакты":

1. Создайте функцию `contact` в `contact/views.py` :

```

from django.shortcuts import render
from django.http import HttpResponseRedirect

def contact(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        email = request.POST.get('email')
        message = request.POST.get('message')
        # Здесь можно добавить логику обработки формы, например,
        # отправку email
        return HttpResponseRedirect("Спасибо за ваше сообщение!")
    return render(request, 'contact/contact.html')

```

2. Создайте шаблон `contact.html`:

- **Создайте директорию для шаблонов:**
 - В папке `templates/` создайте папку `contact`.
- **Создайте файл `contact.html` в `templates/contact/`:**

```

<!DOCTYPE html>
<html>
<head>
    <title>Контакты</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Свяжитесь с нами</h1>
    <form method="post">
        {% csrf_token %}
        <label for="name">Имя:</label>
        <input type="text" id="name" name="name" required><br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required><br>
    <br>

        <label for="message">Сообщение:</label><br>
        <textarea id="message" name="message" rows="5" cols="40"
required></textarea><br><br>

        <button type="submit">Отправить</button>
    </form>
</body>
</html>

```

3. Сохраните изменения и проверьте страницу:

- Запустите сервер:

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/contact/>
- Убедитесь, что страница "Контакты" отображается корректно и форма работает.

Дополнительные подсказки по меню и терминалам

Терминал в Visual Studio Code (VSCode)

Русский:

- **Открытие терминала:**
 - Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+``.
- **Завершение работы терминала:**
 - Нажмите иконку **Trash** (мусорное ведро) в верхней части терминала или перейдите в меню **Терминал > Завершить терминал**.

English:

- **Opening the Terminal:**
 - Go to **View > Terminal** or use the shortcut `Ctrl+``.
- **Killing the Terminal:**
 - Click the **Trash** icon at the top of the terminal or go to **Terminal > Kill Terminal**.

Интерфейс VSCode

Русский:

- **Панель расширений:** Слева нажмите на иконку **Расширения** или используйте `Ctrl+Shift+X`.
- **Панель поиска файлов:** Используйте сочетание клавиш `Ctrl+P` для быстрого поиска файлов.
- **Панель управления Git:** Нажмите на иконку **Source Control** (иконка с веткой) или используйте `Ctrl+Shift+G`.

English:

- **Extensions Panel:** Click on the **Extensions** icon on the left or use `Ctrl+Shift+X`.

- **File Search Panel:** Use the shortcut `Ctrl+P` to quickly search for files.
- **Git Control Panel:** Click on the **Source Control** icon (branch icon) on the left or use `Ctrl+Shift+G`.

Общие советы по работе с терминалом в VSCode

- **Навигация по терминалу:**
 - Используйте стрелки вверх и вниз для просмотра предыдущих команд.
 - **Множественные терминалы:**
 - Вы можете открыть несколько терминалов одновременно, нажав на кнопку **+** в верхней части панели терминала.
 - **Переключение между терминалами:**
 - Используйте выпадающее меню рядом с кнопкой **+** для переключения между открытыми терминалами.
-

Заключение

Поздравляем! Вы успешно:

- Создали новое приложение `events` внутри проекта Django.
 - Зарегистрировали приложение в настройках проекта.
 - Настроили маршруты и представления для приложения.
 - Создали и подключили шаблоны для отображения страниц.
 - Добавили дополнительные приложения `about` и `contact` с соответствующими страницами и формой обратной связи.
 - Запустили сервер разработки и проверили работу приложения.
-

Вы готовы к дальнейшему изучению и разработке на Django. В следующих практических занятиях мы будем углубляться в создание моделей, работу с базой данных, настройку административного интерфейса и другие аспекты Django.

Если у вас остались вопросы или возникли трудности, пожалуйста, обратитесь за помощью к преподавателю или обсудите с одногруппниками.

Успехов в разработке!

Вопросы для обсуждения

- **Почему важно регистрировать приложение в INSTALLED_APPS?**
 - Регистрация приложения в INSTALLED_APPS позволяет Django обнаруживать и интегрировать его компоненты, такие как модели, админ-панель, шаблоны и статические файлы. Это необходимо для корректной работы приложения внутри проекта, обеспечения доступа к его функциональности и возможности применять миграции базы данных.
 - **Как работает подключение маршрутов приложения к проекту через функцию include?**
 - Функция include позволяет включать маршруты из приложения в основной файл urls.py проекта. Это обеспечивает модульность и упрощает управление URL-адресами, позволяя каждому приложению определять свои собственные маршруты, которые затем объединяются в общую систему URL проекта. Это особенно полезно при работе с несколькими приложениями в одном проекте.
 - **Как Django находит и использует шаблоны для отображения страниц?**
 - Django использует настройки TEMPLATES в settings.py для определения путей к шаблонам. При вызове функции render или использовании классов-представлений, Django ищет соответствующий шаблон в указанных директориях, включая директории внутри приложений, если включен параметр APP_DIRS. Затем шаблоны рендерятся с предоставленными контекстными данными и возвращаются как HTTP-ответы пользователю.
-

Полезные ресурсы

- [Документация Django](#)
- [Документация Git](#)
- [Django Templates](#)
- [Django Static Files](#)
- [Документация VSCode](#)

Практическое занятие 19. Настройка маршрутов (URLs) и обработка запросов

Цель занятия:

Научиться настраивать маршруты URL для различных страниц приложения `events`, создавать простые представления (views) и шаблоны для отображения контента. Дополнительно выполнить задания по созданию приложений `about` и `contact` с базовыми страницами.

Задачи занятия

1. Введение в маршрутизацию Django
 2. Добавление основных маршрутов в `events/urls.py`
 3. Создание простых представлений в `events/views.py`
 4. Создание базовых HTML-шаблонов
 5. Тестирование настроенных маршрутов и представлений
 6. Выполнение практического упражнения. Добавление нового маршрута и представления
 7. Выполнение дополнительных заданий. Создание приложений `about` и `contact`
-

Задача 1: Введение в маршрутизацию Django

1.1. Что такое маршрутизация в Django?

- **Маршрутизация (URL routing)** — это механизм сопоставления URL-адресов с соответствующими представлениями (views) в Django.
- Позволяет определить, какая часть кода будет обрабатывать тот или иной запрос от пользователя.

1.2. Основные компоненты маршрутизации:

- **URLConf (URL Configuration):** Файлы `urls.py`, где определяются маршруты.
 - **Представления (Views):** Функции или классы, которые обрабатывают запросы и возвращают ответы.
 - **Маршруты (Paths):** Правила сопоставления URL с представлениями.
-

Задача 2: Добавление основных маршрутов в `events/urls.py`

2.1. Создание файла `urls.py` в приложении `events` (если он еще не создан):

1. В VSCode перейдите в папку `events/`.
2. Если файла `urls.py` нет, создайте его: Правый клик по папке `events` > **New File** > назвать `urls.py`.

2.2. Добавление маршрутов:

Вставьте следующий код в `events/urls.py`:

```
from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
]
```

Пояснение маршрутов:

- `''` : Главная страница приложения `events`.
- `'events/'` : Страница со списком мероприятий.
- `'events/<int:event_id>/'` : Страница с деталями конкретного мероприятия по его ID.

Задача 3: Создание простых представлений в `events/views.py`

3.1. Открытие файла `events/views.py`:

- В папке `events/` откройте файл `views.py`.

3.2. Добавление функций-представлений:

Добавьте следующие функции в `views.py`:

```
from django.shortcuts import render
```

```
def home(request):
    return render(request, 'events/home.html')

def event_list(request):
    # В будущем здесь будет вывод списка мероприятий
    return render(request, 'events/event_list.html')

def event_detail(request, event_id):
    # В будущем здесь будет вывод деталей мероприятия
    return render(request, 'events/event_detail.html', {'event_id':
event_id})
```

Пояснение:

- **home:** Отображает главную страницу приложения `events`.
- **event_list:** Отображает страницу со списком мероприятий.
- **event_detail:** Отображает страницу с деталями конкретного мероприятия по его ID.

Задача 4: Создание базовых HTML-шаблонов

4.1. Создание директории для шаблонов (если она еще не создана):

1. В корневой директории проекта (на одном уровне с `manage.py`) создайте папку `templates`, если ее нет.
2. Внутри `templates/` создайте папку `events`.

4.2. Создание шаблонов:

1. Шаблон `home.html`:

Измените файл `home.html` в `templates/events/` и добавьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
    <title>Главная страница</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <nav>
        <a href="{% url 'events:event_list' %}">Список мероприятий</a>
```

```
|
    </nav>
</body>
</html>
```

Если вы выполнили дополнительные задания, добавьте

```
<a href="{% url 'about:about' %}">0 нас</a> |
<a href="{% url 'contact:contact' %}">Контакты</a>
```

2. Шаблон event_list.html :

Создайте файл event_list.html в templates/events/ и добавьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
    <title>Список мероприятий</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Список мероприятий</h1>
    <p>Здесь будет отображаться список всех мероприятий.</p>
    <a href="{% url 'events:home' %}">Вернуться на главную</a>
</body>
</html>
```

3. Шаблон event_detail.html :

Создайте файл event_detail.html в templates/events/ и добавьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
    <title>Детали мероприятия</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Детали мероприятия</h1>
    <p>Информация о мероприятии с ID: {{ event_id }}</p>
    <a href="{% url 'events:event_list' %}">Вернуться к списку
```

```
мероприятий</a>
</body>
</html>
```

Пояснение:

- **home.html:** Главная страница с навигацией по сайту.
- **event_list.html:** Страница для отображения списка мероприятий (пока статическая).
- **event_detail.html:** Страница для отображения деталей конкретного мероприятия по его ID (пока статическая).

Задача 5: Тестирование настроенных маршрутов и представлений

5.1. Настройка файла `event_manager/urls.py`:

Убедитесь, что в `event_manager/urls.py` подключены маршруты приложения `events`, а также маршруты для приложений `about` и `contact` (если они вами создавались в ходе выполнения дополнительных заданий):

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('events.urls', namespace='events')),
    # about только в случае выполнения дополнительных заданий
    path('about/', include('about.urls', namespace='about')),
    path('contact/', include('contact.urls',
        namespace='contact')),
    # contact только в случае выполнения дополнительных заданий
]
```

5.2. Запуск сервера разработки:

1. Активируйте виртуальное окружение (если не активировано):

- На Windows:

```
venv\Scripts\activate
```

- На macOS/Linux:


```
source venv/bin/activate
```

2. Запустите сервер:

```
python manage.py runserver
```

- **Примечание:** Если команда `python` не работает на macOS/Linux, используйте `python3`.

5.3. Проверка маршрутов в браузере:

1. Главная страница (`home`):

Перейдите по адресу <http://127.0.0.1:8000/>

- **Ожидаемый результат:** Страница с заголовком "Добро пожаловать на сайт мероприятий!" и навигацией.

2. Список мероприятий (`event_list`):

Перейдите по адресу <http://127.0.0.1:8000/events/>

- **Ожидаемый результат:** Страница с заголовком "Список мероприятий" и сообщением о будущей динамике.

3. Детали мероприятия (`event_detail`):

Перейдите по адресу <http://127.0.0.1:8000/events/1/>

- **Ожидаемый результат:** Страница с заголовком "Детали мероприятия" и информацией о мероприятии с ID 1.

5.4. Устранение возможных ошибок:

- **404 Ошибка:** Если страница не найдена, убедитесь, что маршруты и представления настроены правильно.
- **Ошибки в шаблонах:** Проверьте синтаксис HTML и правильность путей к статическим файлам.

Задача 6: Практическое упражнение: Добавление нового маршрута и представления

6.1. Добавление нового маршрута:

1. Откройте файл `events/urls.py`.
2. Добавьте новый маршрут для страницы "Контакты":

```
path('events/<int:event_id>/delete/', views.delete_event,
name='delete_event'),
```

Полный список маршрутов:

```
from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail,
name='event_detail'),
    path('events/<int:event_id>/delete/', views.delete_event,
name='delete_event'),
]
```

6.2. Создание представления `delete_event`:

1. Откройте файл `events/views.py`.
2. Добавьте функцию-представление:

```
from django.shortcuts import render, get_object_or_404, redirect

def delete_event(request, event_id):
    if request.method == 'POST':
        # Пока не реализовано удаление, просто перенаправляем на список
        return redirect('events:event_list')
    return render(request, 'events/delete_event.html', {'event_id':
event_id})
```

6.3. Создание шаблона `delete_event.html`:

1. Создайте файл `delete_event.html` в `templates/events/`.
2. Добавьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
    <title>Удалить мероприятие</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
```

```
'events/css/styles.css' %}">
</head>
<body>
  <h1>Удалить мероприятие</h1>
  <p>Вы уверены, что хотите удалить мероприятие с ID: {{ event_id }}?
</p>
  <form method="post">
    {% csrf_token %}
    <button type="submit">Да, удалить</button>
    <a href="{% url 'events:event_detail' event_id=event_id
%}">Отмена</a>
  </form>
</body>
</html>
```

6.4. Тестирование:

1. Перейдите по адресу <http://127.0.0.1:8000/events/1/delete/> (замените 1 на существующий event_id).
2. **Ожидаемый результат:** Страница подтверждения удаления мероприятия.
3. **Проверка:** Нажмите кнопку "Да, удалить" — должно произойти перенаправление на список мероприятий.

6.5. Обсуждение:

- Пока функция `delete_event` не реализует фактическое удаление, это подготовительный шаг для будущих занятий.
- Познакомились с обработкой POST-запросов и использованием CSRF-токенов в формах.

Задача 7: Дополнительные задания: Создание приложений `about` и `contact`

Цель:

Расширить функциональность проекта, добавив приложения `about` и `contact`, которые будут содержать страницы "О нас" и "Контакты" соответственно.

Задание 7.1: Создание приложения `about`

1. Создание приложения `about` :

В терминале, находясь в корневой директории проекта, выполните команду:

```
python manage.py startapp about
```

2. Регистрация приложения `about` в проекте:

1. Откройте файл `event_manager/settings.py`.
2. Добавьте `'about'` в список `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'events',  
    'about', # Добавлено приложение about  
]
```

3. Сохраните изменения (`Ctrl+S` или `Cmd+S`).

3. Настройка маршрутов для приложения `about`:

1. Создайте файл `about/urls.py` и добавьте следующий код:

```
from django.urls import path  
from . import views  
  
app_name = 'about'  
  
urlpatterns = [  
    path('', views.about, name='about'),  
]
```

2. Подключите маршруты приложения `about` в `event_manager/urls.py`:

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls', namespace='events')),  
    path('about/', include('about.urls', namespace='about')),  
    path('contact/', include('contact.urls', namespace='contact')),  
]
```

3. ****Сохраните изменения** (Ctrl+S или Cmd+S).

4. Создание представления `about` :

Откройте файл `about/views.py` и добавьте функцию:

```
from django.shortcuts import render

def about(request):
    return render(request, 'about/about.html')
```

5. Создание шаблона `about.html` :

1. **Создайте директорию** `templates/about/` **внутри проекта.**
2. **В** `templates/about/` **создайте файл** `about.html` **со следующим содержимым:**

```
<!DOCTYPE html>
<html>
<head>
    <title>О нас</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>О нашей компании</h1>
    <p>Здесь вы можете разместить информацию о вашей компании, миссии,
команде и т.д.</p>
    <a href="{% url 'events:home' %}">Вернуться на главную страницу</a>
</body>
</html>
```

6. Тестирование:

- Запустите сервер (если он не запущен):

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/about/>.
- Убедитесь, что страница "О нас" отображается корректно.

Задание 7.2: Создание приложения `contact`

1. Создание приложения `contact` :

В терминале, находясь в корневой директории проекта, выполните команду:

```
python manage.py startapp contact
```

2. Регистрация приложения `contact` в проекте:

1. Откройте файл `event_manager/settings.py` .
2. Добавьте `'contact'` в список `INSTALLED_APPS` :

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'events',  
    'about',  
    'contact', # Добавлено приложение contact  
]
```

3. Сохраните изменения (`Ctrl+S` или `Cmd+S`).

3. Настройка маршрутов для приложения `contact` :

1. Создайте файл `contact/urls.py` и добавьте следующий код:

```
from django.urls import path  
from . import views  
  
app_name = 'contact'  
  
urlpatterns = [  
    path('', views.contact, name='contact'),  
]
```

2. Подключите маршруты приложения `contact` в `event_manager/urls.py` :

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls', namespace='events')),  
]
```

```
path('about/', include('about.urls', namespace='about')),
path('contact/', include('contact.urls', namespace='contact')),
]
```

3. ****Сохраните изменения (Ctrl+S или Cmd+S).**

4. Создание представления `contact`:

Откройте файл `contact/views.py` и добавьте функцию:

```
from django.shortcuts import render

def contact(request):
    return render(request, 'contact/contact.html')
```

5. Создание шаблона `contact.html`:

1. **Создайте директорию `templates/contact/` внутри проекта.**
2. **В `templates/contact/` создайте файл `contact.html` со следующим содержимым:**

```
<!DOCTYPE html>
<html>
<head>
    <title>Контакты</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Свяжитесь с нами</h1>
    <p>Здесь вы можете разместить контактную информацию: адрес,
телефон, email и т.д.</p>
    <a href="{% url 'events:home' %}">Вернуться на главную страницу</a>
</body>
</html>
```

6. Тестирование:

- Запустите сервер (если он не запущен):

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/contact/>.
- Убедитесь, что страница "Контакты" отображается корректно.

Дополнительные подсказки по меню и терминалам

Терминал в Visual Studio Code (VSCode)

Русский:

- **Открытие терминала:**
 - Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+``.
- **Завершение работы терминала:**
 - Нажмите иконку **Trash** (мусорное ведро) в верхней части терминала или перейдите в меню **Терминал > Завершить терминал**.

English:

- **Opening the Terminal:**
 - Go to **View > Terminal** or use the shortcut `Ctrl+``.
- **Killing the Terminal:**
 - Click the **Trash** icon at the top of the terminal or go to **Terminal > Kill Terminal**.

Интерфейс VSCode

Русский:

- **Панель расширений:** Слева нажмите на иконку **Расширения** или используйте `Ctrl+Shift+X`.
- **Панель поиска файлов:** Используйте сочетание клавиш `Ctrl+P` для быстрого поиска файлов.
- **Панель управления Git:** Нажмите на иконку **Source Control** (иконка с веткой) или используйте `Ctrl+Shift+G`.

English:

- **Extensions Panel:** Click on the **Extensions** icon on the left or use `Ctrl+Shift+X`.
- **File Search Panel:** Use the shortcut `Ctrl+P` to quickly search for files.
- **Git Control Panel:** Click on the **Source Control** icon (branch icon) on the left or use `Ctrl+Shift+G`.

Общие советы по работе с терминалом в VSCode

- **Навигация по терминалу:**
 - Используйте стрелки вверх и вниз для просмотра предыдущих команд.
- **Множественные терминалы:**

- Вы можете открыть несколько терминалов одновременно, нажав на кнопку **+** в верхней части панели терминала.
 - **Переключение между терминалами:**
 - Используйте выпадающее меню рядом с кнопкой **+** для переключения между открытыми терминалами.
-

Заключение

Поздравляем! Вы успешно:

- Настроили маршруты URL для приложения `events`.
 - Создали простые представления и соответствующие HTML-шаблоны.
 - Добавили и настроили дополнительные приложения `about` и `contact` с базовыми страницами.
 - Поняли основы маршрутизации и обработки запросов в Django.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться к преподавателю или обсуждать с одногруппниками.

Успехов в разработке!

Вопросы для обсуждения

- **Что делать, если маршрут не работает?**
 - Проверьте синтаксис в `urls.py`.
 - Убедитесь, что соответствующее представление (`view`) существует и правильно подключено.
 - Проверьте, активировано ли виртуальное окружение и запущен ли сервер.
 - **Как добавить новые маршруты и представления?**
 - Следуйте аналогичной структуре: добавьте маршрут в `urls.py`, создайте соответствующее представление в `views.py`, и создайте шаблон для отображения.
 - **Как организовать шаблоны для разных приложений?**
 - Используйте директории внутри `templates/` для каждого приложения, например, `templates/events/`, `templates/about/`, `templates/contact/`.
-

Полезные ресурсы

- [Документация Django](#)
- [Документация Django REST Framework](#)
- [Документация Git](#)
- [Документация GitHub](#)
- [Django Templates](#)
- [Django Static Files](#)
- [Документация VSCode](#)
- [Simple JWT \(JSON Web Token\) Documentation](#)

Практическое занятие 20: Создание и отображение простых представлений

Цель занятия:

Научиться создавать различные типы представлений в Django (функциональные и классовые) и отображать контент на страницах сайта.

Задачи занятия

1. Введение в представления Django
 2. Создание функциональных представлений
 3. Создание шаблонов для новых страниц
 4. Создание классового представления
 5. Обновление навигации на сайте
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Создание собственного представления
 8. Выполнение дополнительных заданий. Представление списка пользователей, контекстные данные
-

Задача 1: Введение в представления Django

1.1. Что такое представления в Django?

- **Представления (Views)** — это функции или классы, которые принимают HTTP-запрос и возвращают HTTP-ответ.
- Они содержат логику, определяющую, какие данные будут отображаться пользователю и какой шаблон будет использоваться для рендеринга страницы.

1.2. Типы представлений

- **Функциональные представления (Function-Based Views, FBV):**
 - Представляют собой обычные функции Python.
 - Просты и удобны для небольших проектов или простых задач.
- **Классовые представления (Class-Based Views, CBV):**

- Основываются на классах и предоставляют больше возможностей через наследование и миксины.
 - Упрощают повторное использование кода и масштабирование приложения.
-

Задача 2: Создание функциональных представлений

2.1. Открытие файла `events/views.py`

- В VSCode откройте файл `events/views.py`.

2.2. Добавление функциональных представлений `services` и `team`

Добавьте следующие функции в `views.py`:

```
# events/views.py

from django.shortcuts import render

def services(request):
    return render(request, 'events/services.html')

def team(request):
    return render(request, 'events/team.html')
```

Пояснение:

- `services`: Функция для отображения страницы "Услуги".
- `team`: Функция для отображения страницы "Команда".

2.3. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.
-

Задача 3: Добавление маршрутов для новых представлений

3.1. Открытие файла `events/urls.py`

- В папке `events/` откройте файл `urls.py`.

3.2. Добавление маршрутов

Добавьте следующие маршруты в `urlpatterns`:

```
# events/urls.py

from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
    path('services/', views.services, name='services'),
    path('team/', views.team, name='team'),
]
```

Пояснение:

- `'services/'`: Маршрут для страницы "Услуги".
- `'team/'`: Маршрут для страницы "Команда".

3.3. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.
-

Задача 4: Создание шаблонов для новых страниц

4.1. Создание директории для шаблонов (если она еще не создана)

- Убедитесь, что в папке `templates/events/` есть место для хранения шаблонов.
- Если папка отсутствует, создайте ее: `templates/events/`.

4.2. Создание шаблона `services.html`

1. Создайте файл `services.html` в `templates/events/`.
2. Добавьте следующий код:

```
<!-- templates/events/services.html -->

<!DOCTYPE html>
```

```

<html>
<head>
  <title>Услуги</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Наши услуги</h1>
  <p>Описание предоставляемых услуг.</p>
</body>
</html>

```

4.3. Создание шаблона team.html

1. Создайте файл `team.html` в `templates/events/`.
2. Добавьте следующий код:

```

<!-- templates/events/team.html -->

<!DOCTYPE html>
<html>
<head>
  <title>Команда</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Наша команда</h1>
  <p>Информация о членах команды.</p>
</body>
</html>

```

4.4. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файлов.

Задача 5: Создание классового представления

5.1. Добавление классового представления `GalleryView`

Откройте файл `events/views.py` и добавьте следующее:

```
# events/views.py

from django.views.generic import TemplateView

class GalleryView(TemplateView):
    template_name = 'events/gallery.html'
```

Пояснение:

- `GalleryView`: Классовое представление для отображения страницы "Галерея".
- Наследуется от `TemplateView` и использует шаблон `gallery.html`.

5.2. Добавление маршрута для `GalleryView`

Откройте файл `events/urls.py` и добавьте маршрут:

```
# events/urls.py

from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
    path('services/', views.services, name='services'),
    path('team/', views.team, name='team'),
    path('gallery/', views.GalleryView.as_view(), name='gallery'),
]
```

Пояснение:

- `'gallery/'`: Маршрут для страницы "Галерея", использующий классовое представление `GalleryView`.

5.3. Создание шаблона `gallery.html`

1. Создайте файл `gallery.html` в `templates/events/`.
2. Добавьте следующий код:

```
<!-- templates/events/gallery.html -->

<!DOCTYPE html>
<html>
```

```
<head>
  <title>Галерея</title>
  {% load static %}
  <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
  <h1>Галерея мероприятий</h1>
  <p>Фотографии и видео с мероприятий.</p>
</body>
</html>
```

5.4. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файлов.
-

Задача 6: Обновление навигации на сайте

6.1. Добавление навигационного меню

Добавьте следующий код в секцию `<body>` вашего домашнего шаблона (home):

```
<nav>
  <a href="{% url 'events:home' %}">Главная</a> |
  <a href="{% url 'events:services' %}">Услуги</a> |
  <a href="{% url 'events:team' %}">Команда</a> |
  <a href="{% url 'events:gallery' %}">Галерея</a> |
</nav>
```

Дополните код для разделов `about` и `contact`, если они у вас реализованы

Пояснение:

- Навигационное меню с ссылками на основные страницы сайта.
- Используется тег `{% url %}` для создания ссылок на маршруты по их именам.

6.2. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файлов.
-

Задача 7: Запуск и проверка приложения

7.1. Запуск сервера разработки

1. Активируйте виртуальное окружение (если не активировано):

- На Windows:

```
venv\Scripts\activate
```

- На macOS/Linux:

```
source venv/bin/activate
```

2. Запустите сервер:

```
python manage.py runserver
```

- **Примечание:** Если команда `python` не работает на macOS/Linux, используйте `python3`.

7.2. Проверка работы новых представлений

1. Перейдите по адресу <http://127.0.0.1:8000/>:

- Убедитесь, что на главной странице отображается навигационное меню.

2. Проверьте страницу "Услуги":

- Перейдите по адресу <http://127.0.0.1:8000/services/>
- Ожидаемый результат: Страница с заголовком "Наши услуги" и описанием.

3. Проверьте страницу "Команда":

- Перейдите по адресу <http://127.0.0.1:8000/team/>
- Ожидаемый результат: Страница с заголовком "Наша команда" и информацией.

4. Проверьте страницу "Галерея":

- Перейдите по адресу <http://127.0.0.1:8000/gallery/>
- Ожидаемый результат: Страница с заголовком "Галерея мероприятий" и контентом.

7.3. Устранение возможных ошибок

- **404 Ошибка:** Проверьте правильность маршрутов и имен представлений.
 - **TemplateDoesNotExist:** Убедитесь, что шаблоны находятся в правильной директории (`templates/events/`) и что пути указаны верно.
-

Задача 8: Практическое упражнение: Создание собственного представления

8.1. Задание для студентов:

- Создайте новое функциональное представление `contact_us` в `events/views.py`.
- Создайте соответствующий маршрут `/contact_us/` в `events/urls.py`.
- Создайте шаблон `contact_us.html` в `templates/events/` с простой формой или текстом.
- Добавьте ссылку на новую страницу в навигационное меню.

8.2. Пример решения:

1. Добавьте функцию `contact_us` в `events/views.py`:

```
# events/views.py

def contact_us(request):
    return render(request, 'events/contact_us.html')
```

2. Добавьте маршрут в `events/urls.py`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('contact_us/', views.contact_us, name='contact_us'),
]
```

3. Создайте шаблон `contact_us.html`:

```
<!-- templates/events/contact_us.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Свяжитесь с нами</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Свяжитесь с нами</h1>
    <p>Здесь вы можете разместить форму обратной связи или контактную
информацию.</p>
```

```
</body>
</html>
```

4. Добавьте ссылку в навигационное меню:

```
<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:contact_us' %}">Свяжитесь с нами</a>
</nav>
```

8.3. Проверка работы:

- Перейдите по адресу http://127.0.0.1:8000/contact_us/ и убедитесь, что страница отображается корректно.

Дополнительное задание

1. Создание представления списка пользователей

Цель: Реализовать классическое представление `UserListView`, отображающее список зарегистрированных пользователей.

Шаги:

1. Добавьте представление в `events/views.py`:

```
# events/views.py

from django.views.generic import ListView
from django.contrib.auth.models import User

class UserListView(ListView):
    model = User
    template_name = 'events/user_list.html'
    context_object_name = 'users'
```

2. Добавьте маршрут в `events/urls.py`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('users/', views.UserListView.as_view(), name='user_list'),
]
```

3. Создайте шаблон `user_list.html`:

```
<!-- templates/events/user_list.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Список пользователей</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Список зарегистрированных пользователей</h1>
    <ul>
        {% for user in users %}
            <li>{{ user.username }} ({{ user.email }})</li>
        {% empty %}
            <li>Нет зарегистрированных пользователей.</li>
        {% endfor %}
    </ul>
</body>
</html>
```

4. Добавьте ссылку в навигационное меню:

```
<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:user_list' %}">Пользователи</a>
</nav>
```

5. Проверка работы:

- Перейдите по адресу <http://127.0.0.1:8000/users/> и убедитесь, что список пользователей отображается.

2. Добавление контекстных данных

Цель: Передать дополнительные данные в представления `services` и `team`, такие как список услуг или информация о команде.

Поскольку мы еще не используем модели и базы данных, будем использовать статические данные.

1. Обновите функцию `services` в `events/views.py`:

```
# events/views.py

def services(request):
    services_list = [
        'Организация мероприятий',
        'Аренда оборудования',
        'Кейтеринг',
        'Развлекательные программы',
    ]
    return render(request, 'events/services.html', {'services':
services_list})
```

2. Обновите шаблон services.html:

```
<!-- templates/events/services.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Услуги</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Наши услуги</h1>
    <ul>
        {% for service in services %}
            <li>{{ service }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

3. Обновите функцию team в events/views.py:

```
# events/views.py

def team(request):
    team_members = [
        {'name': 'Иван Иванов', 'position': 'Директор'},
        {'name': 'Петр Петров', 'position': 'Менеджер проектов'},
        {'name': 'Светлана Смирнова', 'position': 'Координатор
мероприятий'},
    ]
    return render(request, 'events/team.html', {'team': team_members})
```

4. Обновите шаблон `team.html`:

```
<!-- templates/events/team.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Команда</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Наша команда</h1>
    <ul>
        {% for member in team %}
            <li>{{ member.name }} - {{ member.position }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

5. Проверка работы:

- Перейдите на страницы "Услуги" и "Команда" и убедитесь, что данные отображаются корректно.

Заключение

Поздравляем! Вы успешно:

- Создали функциональные и классовые представления в Django.
- Настроили маршруты для новых представлений.
- Создали шаблоны и отобразили контент на страницах сайта.
- Обновили навигационное меню для доступа к новым страницам.
- Передали и отобразили контекстные данные в шаблонах.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться к преподавателю или обсуждать с одногруппниками.

Успехов в разработке!

Вопросы для обсуждения

- **Чем отличаются функциональные и классовые представления?**
 - Функциональные представления проще и легче для понимания, но классовые предоставляют больше возможностей для расширения и повторного использования кода.
 - **Когда стоит использовать классовые представления?**
 - Когда вам нужно реализовать стандартные операции (например, отображение списка объектов), или когда вы хотите воспользоваться наследованием и миксинами для добавления функциональности.
 - **Как передавать данные из представления в шаблон?**
 - Через контекст — словарь, который передается в функцию `render()` и доступен в шаблоне.
-

Полезные ресурсы

- [Документация Django: Представления](#)
- [Классовые представления Django](#)
- [Django Template Language](#)
- [Документация VSCode](#)

Практическое занятие 21: Создание HTML-шаблонов и динамическое содержимое

Цель занятия:

- Создать HTML-шаблоны для различных страниц.
 - Реализовать динамическое отображение данных, используя статические данные в представлениях.
-

Задачи занятия

1. Введение в шаблоны Django
 2. Создание базового шаблона `base.html`
 3. Создание наследуемых шаблонов
 4. Реализация динамического содержимого без моделей
 5. Создание дополнительных динамических элементов
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Добавление нового шаблона с динамическими данными
-

Задача 1: Введение в шаблоны Django

1.1. Важность HTML-шаблонов в Django

- **Шаблоны** позволяют отделить логику приложения от представления данных пользователю.
- Даже без моделей можно создавать динамические веб-страницы, используя данные, переданные из представлений.
- Обеспечивают переиспользование кода через механизм наследования шаблонов.

1.2. Обзор системы шаблонов Django

- **Django Template Language (DTL)** — встроенный язык шаблонов Django.
- Позволяет использовать теги, фильтры и переменные для отображения динамического контента.
- Поддерживает наследование шаблонов, включение других шаблонов и использование контекстных процессоров.

Задача 2: Создание базового шаблона `base.html`

2.1. Создание директории для шаблонов

- Убедитесь, что у вас есть директория `templates/events/` в вашем приложении `events`.
- Если нет, создайте её:

```
your_project/
├── events/
│   ├── templates/
│   │   └── events/
│   │       └── (шаблоны будут здесь)
```

2.2. Создание файла `base.html`

- В директории `events/templates/events/` создайте файл `base.html`.

2.3. Наполнение файла `base.html`

```
<!-- events/templates/events/base.html -->

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Система управления мероприятиями{% endblock %}
</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'events/css/styles.css' %}">
</head>
<body>
    <nav>
        <a href="{% url 'events:home' %}">Главная</a> |
        <a href="{% url 'events:services' %}">Услуги</a> |
        <a href="{% url 'events:team' %}">Команда</a> |
        <a href="{% url 'events:gallery' %}">Галерея</a> |
        <a href="{% url 'events:contact_us' %}">Свяжитесь с нами</a>
    </nav>
    <div class="content">
        {% block content %}
        <!-- Основной контент будет вставлен здесь -->
        {% endblock %}
    </div>
```

```
</body>
</html>
```

Пояснение:

- `{% block title %}{% endblock %}`: Позволяет переопределять заголовок страницы в дочерних шаблонах.
- `{% block content %}{% endblock %}`: Место, где будет вставлен основной контент из дочерних шаблонов.
- `{% load static %}`: Загружает тег `static` для доступа к статическим файлам.
- **Навигация**: Использует теги `{% url %}` для создания ссылок на маршруты по их именам.

2.4. Обновление файла стилей `styles.css`

- Перейдите в директорию для статических файлов: `events/static/events/css/`.
- Обновите файл `styles.css` в этой директории.

Пример содержимого `styles.css`:

```
/* events/static/events/css/styles.css */

body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
}

nav {
    background-color: #333;
    padding: 10px;
}

nav a {
    color: #ffffff;
    margin-right: 15px;
    text-decoration: none;
}

nav a:hover {
    text-decoration: underline;
}

.content {
    padding: 20px;
}
```

```
h1, h2 {
    color: #333;
}

ul {
    list-style-type: none;
    padding: 0;
}

li {
    margin-bottom: 10px;
}
```

2.5. Настройка статических файлов в `settings.py`

- Откройте `your_project/settings.py` и убедитесь, что настройки для статических файлов указаны правильно:

```
# your_project/settings.py

STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'events/static']
```

Задача 3: Создание наследуемых шаблонов

3.1. Обновление шаблона `home.html`

Шаги:

- Откройте файл `events/templates/events/home.html`.
- Измените его содержимое следующим образом:

```
<!-- events/templates/events/home.html -->

{% extends 'events/base.html' %}

{% block title %}Домашняя страница{% endblock %}

{% block content %}
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <p>Здесь вы найдете информацию о наших последних событиях.</p>
{% endblock %}
```

3.2. Обновление других шаблонов

Повторите аналогичные шаги для следующих шаблонов:

3.2.1. services.html

```
<!-- events/templates/events/services.html -->

{% extends 'events/base.html' %}

{% block title %}Услуги{% endblock %}

{% block content %}
    <h1>Наши услуги</h1>
    <ul>
        {% for service in services %}
            <li>{{ service }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

3.2.2. team.html

```
<!-- events/templates/events/team.html -->

{% extends 'events/base.html' %}

{% block title %}Команда{% endblock %}

{% block content %}
    <h1>Наша команда</h1>
    <ul>
        {% for member in team %}
            <li>{{ member.name }} - {{ member.position }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

3.2.3. gallery.html

```
<!-- events/templates/events/gallery.html -->

{% extends 'events/base.html' %}

{% block title %}Галерея{% endblock %}

{% block content %}
    <h1>Галерея мероприятий</h1>
```

```
<p>Фотографии и видео с наших мероприятий.</p>
{% endblock %}
```

3.2.4. contact_us.html

```
<!-- events/templates/events/contact_us.html -->

{% extends 'events/base.html' %}

{% block title %}Свяжитесь с нами{% endblock %}

{% block content %}
    <h1>Свяжитесь с нами</h1>
    <p>Здесь вы можете разместить форму обратной связи или контактную
информацию.</p>
{% endblock %}
```

3.3. Сохраните изменения

- Убедитесь, что вы сохранили все изменения в шаблонах.

Задача 4: Реализация динамического содержимого без моделей

Поскольку модели еще не введены, мы будем использовать статические данные, передаваемые из представлений в шаблоны.

4.1. Обновление представлений в events/views.py

Шаги:

1. Откройте файл events/views.py.
2. Добавьте или обновите следующие представления:

```
# events/views.py

from django.shortcuts import render

def home(request):
    return render(request, 'events/home.html')

def services(request):
    services_list = [
        'Организация мероприятий',
```

```

        'Аренда оборудования',
        'Кейтеринг',
        'Развлекательные программы',
    ]
    return render(request, 'events/services.html', {'services':
services_list})

team_members = [
    {'name': 'Иван Иванов', 'position': 'Директор'},
    {'name': 'Петр Петров', 'position': 'Менеджер проектов'},
    {'name': 'Светлана Смирнова', 'position': 'Координатор
мероприятий'},
]

def team(request):
    return render(request, 'events/team.html', {'team': team_members})

def gallery(request):
    return render(request, 'events/gallery.html')

def contact_us(request):
    return render(request, 'events/contact_us.html')

```

4.2. Использование переданных данных в шаблонах

- В шаблонах `services.html` и `team.html` мы уже используем переменные `services` и `team`, переданные из представлений.
- Убедитесь, что имена переменных в представлениях и шаблонах совпадают.

Задача 5: Создание дополнительных динамических элементов

5.1. Добавление списка мероприятий на главную страницу

Поскольку мы пока что не используем модели, мы можем передать статический список мероприятий.

Шаги:

1. Обновите представление `home` в `events/views.py`:

```

events = [
    {'id': 1, 'title': 'Концерт классической музыки', 'date':
'12.01.2025 19:00', 'description': 'Концерт с участием известных
исполнителей.', 'location': 'Концертный зал', 'organizer': 'Иван Иванов'},

```

```

        {'id': 2, 'title': 'Выставка современного искусства', 'date':
'02.02.2025 10:00', 'description': 'Выставка работ современных художников.',
'location': 'Галерея искусств', 'organizer': 'Петр Петров'},
        {'id': 3, 'title': 'Театральная постановка', 'date': '08.03.2025
18:30', 'description': 'Постановка классического произведения.', 'location':
'Драматический театр', 'organizer': 'Светлана Смирнова'},
    ]

def home(request):
    return render(request, 'events/home.html', {'events': events})

```

2. Обновите шаблон `home.html` для отображения списка мероприятий:

```

{% extends 'events/base.html' %}

{% block title %}Домашняя страница{% endblock %}

{% block content %}
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <p>Здесь вы найдете информацию о наших последних событиях.</p>

    <h2>Предстоящие мероприятия</h2>
    <ul>
        {% for event in events %}
            <li>
                <strong>{{ event.title }}</strong> - {{ event.date }}
            </li>
        {% empty %}
            <li>Нет предстоящих мероприятий.</li>
        {% endfor %}
    </ul>
{% endblock %}

```

5.2. Создание страницы деталей мероприятия

Поскольку у нас нет моделей, мы можем создать представление, которое будет принимать `event_id` и возвращать соответствующее мероприятие из списка.

Шаги:

1. Добавьте представление `event_detail` в `events/views.py`:

```

def event_detail(request, event_id):
    event = next((item for item in events if item['id'] == event_id), None)
    if event:
        return render(request, 'events/event_detail.html', {'event': event})

```

```
else:
    return render(request, 'events/event_not_found.html')
```

2. Измените шаблон event_detail.html :

```
<!-- events/templates/events/event_detail.html -->

{% extends 'events/base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <p><strong>Дата и время:</strong> {{ event.date }}</p>
    <p><strong>Место проведения:</strong> {{ event.location }}</p>
    <p><strong>Описание:</strong> {{ event.description }}</p>
    <p><strong>Организатор:</strong> {{ event.organizer }}</p>
{% endblock %}
```

3. Создайте шаблон event_not_found.html :

```
<!-- events/templates/events/event_not_found.html -->

{% extends 'events/base.html' %}

{% block title %}Мероприятие не найдено{% endblock %}

{% block content %}
    <h1>Мероприятие не найдено</h1>
    <p>К сожалению, мероприятие с указанным ID не найдено.</p>
{% endblock %}
```

4. Добавьте маршрут в events/urls.py :

```
# events/urls.py

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
    path('services/', views.services, name='services'),
    path('team/', views.team, name='team'),
    path('gallery/', views.GalleryView.as_view(), name='gallery'),
    path('contact_us/', views.contact_us, name='contact_us'),
]
```


5. Обновите ссылки на мероприятия в шаблоне `home.html` :

```
<ul>
  {% for event in events %}
    <li>
      <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> - {{ event.date }}
    </li>
  {% empty %}
    <li>Нет предстоящих мероприятий.</li>
  {% endfor %}
</ul>
```

Задача 6: Запуск и проверка приложения

6.1. Запуск сервера разработки

```
python manage.py runserver
```

6.2. Проверка работы приложения

- Перейдите на главную страницу:
 - <http://127.0.0.1:8000/>
- Убедитесь, что список мероприятий отображается корректно.
- Кликните на название мероприятия, чтобы перейти на страницу деталей мероприятия.
- Убедитесь, что информация о мероприятии отображается правильно.
- Попробуйте перейти по несуществующему `event_id`, например:
 - <http://127.0.0.1:8000/events/99/>
- Убедитесь, что отображается страница "Мероприятие не найдено".

Задача 7: Практическое упражнение

Задание для студентов:

- Создайте новый шаблон `about.html`, наследующийся от `base.html`.
- Отобразите в нём информацию о компании: историю, миссию, команду и т.д.

Шаги:

1. Создайте шаблон `about.html` в `events/templates/events/`:

```
<!-- events/templates/events/about.html -->

{% extends 'events/base.html' %}

{% block title %}О нас{% endblock %}

{% block content %}
    <h1>О нашей компании</h1>
    <p>Наша компания занимается организацией мероприятий с 2010 года.</p>
    <p>Наша миссия – создавать незабываемые события для наших клиентов.</p>
    <h2>Наша команда</h2>
    <ul>
        {% for member in team %}
            <li>{{ member.name }} – {{ member.position }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

2. Добавьте представление `about` в `events/views.py`:

```
def about(request):
    return render(request, 'events/about.html', {'team': team_members})
```

3. Добавьте маршрут в `events/urls.py`:

```
urlpatterns = [
    # ... другие маршруты ...
    path('about/', views.about, name='about'),
]
```

4. Добавьте ссылку в навигацию в `base.html`:

```
<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:about' %}">О нас</a>
</nav>
```

Заключение

Поздравляем! Вы успешно:

- Создали базовый шаблон `base.html` и настроили наследование шаблонов.
 - Обновили существующие шаблоны для использования `base.html`.
 - Реализовали динамическое отображение данных, используя статические данные в представлениях.
 - Создали дополнительные страницы и маршруты.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Как механизм наследования шаблонов помогает в разработке?**
 - Позволяет переиспользовать общие элементы интерфейса, сокращает дублирование кода, облегчает внесение изменений.
 - **Как можно передавать данные из представления в шаблон без использования моделей?**
 - Используя статические данные или данные из других источников, передавая их через словарь контекста в функцию `render()`.
 - **Как можно дальше улучшить функциональность сайта без моделей?**
 - Добавить интерактивность с помощью JavaScript.
 - Использовать формы для сбора данных от пользователя и обработки их в представлениях.
 - Подключить внешние API для получения данных.
-

Полезные ресурсы

- [Документация Django: Шаблоны](#)
- [Django Template Language \(DTL\) Cheatsheet](#)
- [Основы представлений в Django](#)
- [Настройка URL маршрутов](#)

Практическое занятие 22. Работа с контекстом и подключение стилей (CSS)

Цель занятия:

- Понять, как передавать данные в шаблоны через контекст.
 - Подключить и настроить CSS-стили для улучшения внешнего вида сайта.
-

Задачи занятия

1. Введение в контекст и статические файлы в Django
 2. Передача данных через контекст
 3. Использование контекста в шаблонах
 4. Подключение CSS-стилей
 5. Использование стилей в шаблонах
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Улучшение внешнего вида сайта
 8. Выполнение дополнительных заданий. Добавление медиа-файлов, создание адаптивного дизайна
-

Задача 1: Введение в контекст и статические файлы в Django

1.1. Что такое контекст в Django?

- **Контекст** — это словарь данных, которые вы передаете из представления (view) в шаблон.
- Позволяет динамически отображать информацию на странице, основываясь на данных, полученных или обработанных в представлении.

1.2. Работа со статическими файлами

- **Статические файлы** — это файлы, которые не изменяются при работе приложения: CSS, JavaScript, изображения и т.д.
- Django предоставляет специальный механизм для работы со статическими файлами, позволяя легко подключать их в шаблонах.

Задача 2: Передача данных через контекст

2.1. Обновление представления `event_detail`

Шаги:

1. Откройте файл `events/views.py`.
2. Добавьте или обновите функцию `event_detail`:

```
# events/views.py

from django.shortcuts import render, get_object_or_404

events = [
    {'id': 1, 'title': 'Концерт классической музыки', 'date': '2023-11-10 19:00', 'description': 'Концерт с участием известных исполнителей.', 'location': 'Концертный зал', 'organizer': 'Иван Иванов', 'category': 'Музыка', 'comments': [
        {'user': 'Пользователь1', 'comment': 'Потрясающее мероприятие!', 'created_at': '2023-11-11 10:00'},
        {'user': 'Пользователь2', 'comment': 'Очень понравилось!', 'created_at': '2023-11-12 12:00'},
    ]},
    # Добавьте другие мероприятия, если необходимо
]

def event_detail(request, event_id):
    # Статические данные, так как модели не используются
    event = next((item for item in events if item['id'] == event_id), None)
    if event:
        comments = event.get('comments', [])
        return render(request, 'events/event_detail.html', {
            'event': event,
            'comments': comments
        })
    else:
        return render(request, 'events/event_not_found.html')
```

Пояснение:

- `event` : Текущее мероприятие, выбранное по `event_id`.
 - `comments` : Список комментариев для данного мероприятия.
 - Мы используем статические данные, так как модели не введены.
-

Задача 3: Использование контекста в шаблонах

3.1. Обновление шаблона `event_detail.html`

Шаги:

1. Откройте файл `events/templates/events/event_detail.html`.
2. Измените его содержимое следующим образом:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'events/base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <p><strong>Дата и время:</strong> {{ event.date }}</p>
    <p><strong>Место проведения:</strong> {{ event.location }}</p>
    <p><strong>Категория:</strong> {{ event.category }}</p>
    <p>{{ event.description }}</p>

    <h2>Отзывы</h2>
    {% if comments %}
        {% for comment in comments %}
            <div class="comment">
                <p><strong>{{ comment.user }}</strong> - <em>{{
comment.created_at }}</em></p>
                <p>{{ comment.comment }}</p>
            </div>
        {% endfor %}
    {% else %}
        <p>Нет отзывов о данном мероприятии.</p>
    {% endif %}
{% endblock %}
```

Пояснение:

- Используем переменные `event` и `comments`, переданные из представления.
- Отображаем информацию о мероприятии и список комментариев.

Задача 4: Подключение CSS-стилей

4.1. Создание директории для статических файлов

Шаги:

1. **Создайте директорию** `events/static/events/css/`.
 - Если директории `static` или `css` не существуют, создайте их.
2. **Структура должна быть следующей:**

```
your_project/
├─ events/
│   └─ static/
│       └─ events/
│           └─ css/
│               └─ styles.css
```

4.2. Дополнение файла `styles.css`

Дополните файл `styles.css` в `events/static/events/css/`, добавьте следующий код:

```
/* events/static/events/css/styles.css */

body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
}

nav {
    background-color: #333;
    color: #fff;
    padding: 15px;
    text-align: center;
}

nav a {
    color: #fff;
    margin: 0 10px;
    text-decoration: none;
}

nav a:hover {
    text-decoration: underline;
}

.content {
    padding: 20px;
```

```

}

h1, h2 {
    color: #333;
}

ul {
    list-style-type: none;
    padding: 0;
}

li {
    background-color: #fff;
    margin-bottom: 10px;
    padding: 10px;
    border-radius: 5px;
    box-shadow: 0 0 5px rgba(0,0,0,0.1);
}

.comment {
    background-color: #e9e9e9;
    margin-bottom: 10px;
    padding: 10px;
    border-radius: 5px;
}

.pagination a {
    margin: 0 5px;
    text-decoration: none;
    color: #4CAF50;
}

.pagination span {
    margin: 0 5px;
}

```

4.3. Настройка статических файлов в `settings.py`

- Убедитесь, что в `your_project/settings.py` указаны правильные настройки:

```

# your_project/settings.py

STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
STATICFILES_DIRS = [BASE_DIR / 'events/static']

```

4.4. Подключение CSS-стилей в `base.html`

Шаги:

1. Откройте файл `events/templates/events/base.html`.
2. В секции `<head>` убедитесь в подключении стилей:

```
<!-- events/templates/events/base.html -->

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Система управления мероприятиями{% endblock %}
</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'events/css/styles.css' %}">
</head>
<body>
    <!-- Остальной код -->
</body>
</html>
```

Пояснение:

- `{% load static %}`: Загружает тег `static` для использования в шаблоне.
- `<link rel="stylesheet" href="{% static 'events/css/styles.css' %}">`: Подключает файл стилей.

Задача 5: Использование стилей в шаблонах

5.1. Обновление шаблона `event_list.html`

Шаги:

1. Откройте файл `events/templates/events/event_list.html`.
2. Измените его содержимое следующим образом:

```
<!-- events/templates/events/event_list.html -->

{% extends 'events/base.html' %}

{% block title %}Список мероприятий{% endblock %}

{% block content %}
    <h1>Мероприятия</h1>
    <ul>
```

```

        {% for event in events %}
            <li>
                <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> -
                {{ event.date }}
            </li>
        {% empty %}
            <li>Нет доступных мероприятий.</li>
        {% endfor %}
    </ul>
{% endblock %}

```

Пояснение:

- Используем класс `li` из стилей для оформления списка мероприятий.

5.2. Обновление других шаблонов

- **Убедитесь**, что в других шаблонах вы используете соответствующие теги и классы для оформления.

Пример для `home.html`:

```

<!-- events/templates/events/home.html -->

{% extends 'events/base.html' %}

{% block title %}Домашняя страница{% endblock %}

{% block content %}
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <p>Здесь вы найдете информацию о наших последних событиях.</p>

    <h2>Предстоящие мероприятия</h2>
    <ul>
        {% for event in events %}
            <li>
                <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> - {{ event.date }}
            </li>
        {% empty %}
            <li>Нет предстоящих мероприятий.</li>
        {% endfor %}
    </ul>
{% endblock %}

```

Задача 6: Запуск и проверка приложения

6.1. Запуск сервера разработки

```
python manage.py runserver
```

6.2. Проверка работы приложения

- Перейдите на главную страницу:
 - <http://127.0.0.1:8000/>
- Убедитесь, что стили применяются корректно.
- Перейдите на страницу деталей мероприятия и проверьте отображение комментариев и стилей.

6.3. Устранение возможных проблем

- Если стили не применяются:
 - Убедитесь, что путь к файлу стилей в `base.html` указан правильно.
 - Проверьте настройки `STATIC_URL` и `STATICFILES_DIRS` в `settings.py`.
 - Убедитесь, что у вас запущен сервер разработки и он обновлен до последних изменений.

Задача 7: Практическое упражнение: Улучшение внешнего вида сайта

Задание для студентов:

- Добавьте свои стили или измените существующие для улучшения внешнего вида сайта.
 - Измените цвета, шрифты, отступы и т.д.
 - Добавьте оформление для кнопок, ссылок и других элементов.
 - Сделайте сайт более привлекательным и удобным для пользователя.

Пример изменений:

- Добавьте эффекты при наведении на ссылки в навигации:

```
nav a:hover {  
    background-color: #555;  
    padding: 5px;
```

```
border-radius: 5px;
}
```

- Измените фон сайта:

```
body {
    background-color: #e0e0e0;
}
```

Дополнительное задание

1. Добавление медиа-файлов

Цель: Настроить работу с изображениями, добавив возможность отображения изображений мероприятий.

Шаги:

1. Создайте директорию для медиа-файлов:

```
your_project/
├── events/
│   ├── media/
│   │   └── events/
```

2. Добавьте изображения мероприятий в `events/media/events/`.
3. Настройте `settings.py` для работы с медиа-файлами:

```
# your_project/settings.py

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'events/media'
```

4. Настройте URL для медиа-файлов в `your_project/urls.py`:

```
# your_project/urls.py

from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # ... ваши маршруты ...
```

```
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

5. Обновите представления и шаблоны для отображения изображений:

- В представлении `event_detail` добавьте путь к изображению:

```
def event_detail(request, event_id):
    # ... ваш существующий код ...
    event = {
        # ... другие поля ...
        'image': f'events/event_{event_id}.jpg',
    }
    # ... остальной код ...
```

- В шаблоне `event_detail.html` отобразите изображение:

```
{% if event.image %}
    
{% endif %}
```

Пояснение:

- `MEDIA_URL` и `MEDIA_ROOT` : Настройки для работы с медиа-файлами.
- `static()` : Функция для обслуживания медиа-файлов в режиме разработки.

2. Создание адаптивного дизайна

Цель: Использовать CSS-фреймворк, например, Bootstrap, для улучшения адаптивности и внешнего вида сайта.

Шаги:

1. Подключите Bootstrap в `base.html` :

```
<!-- events/templates/events/base.html -->

<!DOCTYPE html>
<html>
<head>
    <!-- ... ваш существующий код ... -->
    <!-- Подключение Bootstrap CSS из CDN -->
```

```

<link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.c
ss">
</head>
<body>
  <!-- ... ваш существующий код ... -->
  <!-- Подключение Bootstrap JS и зависимостей -->
  <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.5.2/dist/js/bootstrap.bundle.m
in.js"></script>
</body>
</html>

```

2. Обновите шаблоны, используя классы Bootstrap:

- Пример для `event_list.html`:

```

<!-- events/templates/events/event_list.html -->

{% extends 'events/base.html' %}

{% block content %}
  <div class="container">
    <h1 class="mt-5">Мероприятия</h1>
    <div class="list-group">
      {% for event in events %}
        <a href="{% url 'events:event_detail' event.id %}"
class="list-group-item list-group-item-action">
          <h5 class="mb-1">{{ event.title }}</h5>
          <small>{{ event.date }}</small>
        </a>
      {% empty %}
        <p>Нет доступных мероприятий.</p>
      {% endfor %}
    </div>
  </div>
{% endblock %}

```

3. Удалите или измените ваши собственные стили в `styles.css`, чтобы не конфликтовать с Bootstrap.

Заключение

Поздравляем! Вы успешно:

- Поняли, как передавать данные в шаблоны через контекст.
 - Научились использовать контекст в шаблонах для отображения динамического содержимого.
 - Подключили и настроили CSS-стили для улучшения внешнего вида сайта.
 - Улучшили внешний вид сайта, используя собственные стили или CSS-фреймворки.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Как контекст помогает в передаче данных из представления в шаблон?**
 - Позволяет передавать любые данные в виде словаря, которые затем можно использовать в шаблоне для динамического отображения информации.
 - **Почему важно использовать статические файлы в Django и как их правильно подключать?**
 - Статические файлы необходимы для оформления и функциональности сайта. Django предоставляет удобный механизм для их организации и подключения через тег `{% static %}`.
 - **Какие преимущества дает использование CSS-фреймворков, таких как Bootstrap?**
 - Ускоряет разработку, предоставляет готовые стили и компоненты, обеспечивает адаптивность и кроссбраузерную совместимость.
-

Полезные ресурсы

- [Документация Django: Работа со статическими файлами](#)
- [Документация Django: Шаблоны](#)
- [Bootstrap Documentation](#)
- [CSS Flexbox Guide](#)

Практическое занятие 23: Создание моделей и миграций в Django

Цель занятия:

- Научиться создавать модели для хранения информации о мероприятиях, категориях и пользователях.
 - Понять процесс создания и применения миграций в Django.
-

Задачи занятия

1. Введение в модели и миграции Django
 2. Создание модели `Category`
 3. Создание модели `Event`
 4. Создание модели `Registration`
 5. Создание модели `Review`
 6. Создание модели `Comment`
 7. Создание модели `Profile`
 8. Создание и применение миграций
 9. Запуск и проверка приложения
 10. Выполнение практического упражнения. Создание собственной модели
 11. Выполнение дополнительных заданий. Добавление моделей и методов
-

Задача 1: Введение в модели и миграции Django

1.1. Что такое модели в Django?

- **Модель** — это класс в Django, который представляет таблицу в базе данных.
- Модели определяют структуру данных, включая поля и типы данных.
- Позволяют взаимодействовать с базой данных с помощью объектов Python, используя ORM (Object-Relational Mapping).

1.2. Что такое миграции в Django?

- **Миграции** — это механизм Django для управления изменениями в модели данных.

- Они позволяют синхронизировать модели с базой данных.
 - Команды `makemigrations` и `migrate` используются для создания и применения миграций.
-

Задача 2: Создание модели `Category`

2.1. Открытие файла `models.py`

- Откройте файл `events/models.py` в вашем редакторе кода.

2.2. Импорт необходимых модулей

```
# events/models.py

from django.db import models
```

2.3. Создание класса `Category`

```
# events/models.py

class Category(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField(blank=True)

    def __str__(self):
        return self.name
```

Пояснение:

- `name` : Название категории, ограниченное 100 символами.
- `description` : Описание категории, может быть пустым (`blank=True`).
- **Метод `__str__`** : Определяет строковое представление объекта, полезно в админ-панели.

2.4. Сохраните изменения

- Нажмите `Ctrl+S` (Windows) или `Cmd+S` (macOS) для сохранения файла.
-

Задача 3: Создание модели `Event`

3.1. Добавление класса `Event` в `models.py`

```
# events/models.py

class Event(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    date = models.DateTimeField()
    location = models.CharField(max_length=200)
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='events')

    def __str__(self):
        return self.title
```

Пояснение:

- `title`: Название мероприятия.
- `description`: Описание мероприятия.
- `date`: Дата и время проведения.
- `location`: Место проведения.
- `category`: Связь с моделью `Category`, при удалении категории удаляются связанные мероприятия (`on_delete=models.CASCADE`).
- `related_name='events'`: Позволяет обращаться к мероприятиям из категории через `category.events.all()`.

3.2. Сохраните изменения

- Сохраните файл `models.py`.

Задача 4: Создание модели `Registration`

4.1. Импорт модели `User`

```
# events/models.py

from django.contrib.auth.models import User
```

4.2. Добавление класса `Registration`

```
# events/models.py
```

```
class Registration(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='registrations')
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='registrations')
    registration_date = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.user.username} зарегистрировался на {self.event.title}'
```

Пояснение:

- `user`: Связь с моделью `User`, представляющей пользователей.
- `event`: Связь с моделью `Event`.
- `registration_date`: Дата и время регистрации, автоматически устанавливается при создании (`auto_now_add=True`).
- `related_name='registrations'`: Позволяет обращаться к регистрациям пользователя или мероприятия.

4.3. Сохраните изменения

- Сохраните файл `models.py`.

Задача 5: Создание модели `Review`

5.1. Добавление класса `Review`

```
# events/models.py

class Review(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='reviews')
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='reviews')
    rating = models.IntegerField()
    comment = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Отзыв {self.user.username} о {self.event.title}'
```

Пояснение:

- `rating` : Оценка мероприятия (например, от 1 до 5).
- `comment` : Текст отзыва.
- `created_at` : Дата и время создания отзыва.

5.2. Сохраните изменения

- Сохраните файл `models.py`.
-

Задача 6: Создание модели `Comment`

6.1. Добавление класса `Comment`

```
# events/models.py

class Comment(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='comments')
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='comments')
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Комментарий {self.user.username} к {self.event.title}'
```

Пояснение:

- `content` : Содержимое комментария.
- `created_at` : Дата и время создания комментария.

6.2. Сохраните изменения

- Сохраните файл `models.py`.
-

Задача 7: Создание модели `Profile`

7.1. Добавление класса `Profile`

```
# events/models.py

from django.db.models.signals import post_save
```

```

from django.dispatch import receiver

class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(blank=True)
    avatar = models.ImageField(upload_to='avatars/', blank=True, null=True)
    subscribed = models.BooleanField(default=False)

    def __str__(self):
        return f'Профиль {self.user.username}'

```

Пояснение:

- `OneToOneField`: Связь "один к одному" с моделью `User`.
- `bio`: Биография пользователя.
- `avatar`: Аватар пользователя, сохраняется в директорию `avatars/`.
- `subscribed`: Флаг подписки на рассылку.

7.2. Создание сигналов для автоматического создания и сохранения профиля

```

# events/models.py

@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

@receiver(post_save, sender=User)
def save_user_profile(sender, instance, **kwargs):
    instance.profile.save()

print("Модели успешно созданы")

```

Пояснение:

- **Сигналы** `post_save`: Автоматически создают профиль при создании нового пользователя и сохраняют изменения в профиле при сохранении пользователя.

7.3. Сохраните изменения

- Сохраните файл `models.py`.

Задача 8: Создание и применение миграций

8.1. Создание миграций

- Откройте терминал в корневой директории вашего проекта.

```
python manage.py makemigrations
```

Пояснение:

- Команда `makemigrations` создаст файлы миграций на основе изменений в моделях.

8.2. Применение миграций

```
python manage.py migrate
```

Пояснение:

- Команда `migrate` применит миграции к базе данных, создавая необходимые таблицы.

Задача 9: Запуск и проверка приложения

9.1. Запуск сервера разработки

```
python manage.py runserver
```

9.2. Проверка

- Перейдите по адресу <http://127.0.0.1:8000/> и убедитесь, что сайт работает без ошибок.
В терминале выведется сообщение `Модели успешно созданы`.

9.3. Проверка базы данных

- Вы можете использовать админ-панель Django для просмотра и редактирования данных.
- Создайте суперпользователя:

```
python manage.py createsuperuser
```

- Следуйте инструкциям для создания учетной записи администратора.

- Перейдите по адресу <http://127.0.0.1:8000/admin/> и войдите под учетной записью администратора.

Задача 10: Практическое упражнение

Задание для студентов:

- Создайте новую модель `Sponsor` в `events/models.py`, которая будет хранить информацию о спонсорах мероприятий.
- Поля модели могут включать:
 - `name` (`CharField`)
 - `logo` (`ImageField`)
 - `website` (`URLField`)
 - `events` (`ManyToManyField` с моделью `Event`)
- Примените миграции и убедитесь, что модель создана в базе данных.

Решение:

1. Добавление класса `Sponsor`:

```
# events/models.py

class Sponsor(models.Model):
    name = models.CharField(max_length=200)
    logo = models.ImageField(upload_to='sponsors/', blank=True, null=True)
    website = models.URLField(blank=True)
    events = models.ManyToManyField(Event, related_name='sponsors',
    blank=True)

    def __str__(self):
        return self.name
```

2. Применение миграций:

```
python manage.py makemigrations
python manage.py migrate
```

Дополнительное задание

1. Добавление модели `Venue`

Цель: Создать модель `Venue` для хранения информации о местах проведения мероприятий и связать её с моделью `Event`.

Шаги:

1. Создайте модель `Venue` в `events/models.py`:

```
class Venue(models.Model):
    name = models.CharField(max_length=200)
    address = models.CharField(max_length=300)
    capacity = models.IntegerField()

    def __str__(self):
        return self.name
```

2. Обновите модель `Event` для связи с `Venue`:

```
class Event(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    date = models.DateTimeField()
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='events')
    venue = models.ForeignKey(Venue, on_delete=models.CASCADE,
related_name='events')

    def __str__(self):
        return self.title
```

3. Примените миграции:

```
python manage.py makemigrations
python manage.py migrate
```

2. Добавление метода `get_absolute_url` в модель `Event`

Цель: Добавить метод для получения URL мероприятия, что упростит навигацию и ссылку на объект.

Шаги:

1. Импортируйте функцию `reverse`:

```
from django.urls import reverse
```

2. Добавьте метод `get_absolute_url` в класс `Event`:


```
class Event(models.Model):
    # ... ваши поля ...

    def get_absolute_url(self):
        return reverse('events:event_detail', args=[str(self.id)])
```

Пояснение:

- Теперь вы можете использовать `event.get_absolute_url()` для получения URL мероприятия.

3. Настройка отображения связей в админ-панели

Цель: Улучшить админ-панель, чтобы можно было просматривать связанные объекты.

Шаги:

1. Откройте файл `events/admin.py`.

2. Зарегистрируйте модели и настройте отображение:

```
# events/admin.py

from django.contrib import admin
from .models import Category, Event, Registration, Review, Comment, Profile,
Venue, Sponsor

@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ('name', 'description')
    search_fields = ('name',)

@admin.register(Event)
class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'category', 'venue')
    list_filter = ('category', 'date')
    search_fields = ('title', 'description')
    date_hierarchy = 'date'

@admin.register(Venue)
class VenueAdmin(admin.ModelAdmin):
    list_display = ('name', 'address', 'capacity')
    search_fields = ('name', 'address')

# Зарегистрируйте остальные модели
admin.site.register(Registration)
admin.site.register(Review)
admin.site.register(Comment)
```

```
admin.site.register(Profile)
admin.site.register(Sponsor)
```

Пояснение:

- Использование декораторов `@admin.register` для регистрации моделей.
 - Настройка отображения списка объектов и полей для поиска.
-

Заключение

Поздравляем! Вы успешно:

- Создали модели для хранения информации о категориях, мероприятиях, пользователях и других сущностях.
 - Научились создавать и применять миграции в Django.
 - Настроили админ-панель для управления данными.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Что такое миграции и почему они важны?**
 - Миграции позволяют отслеживать изменения в моделях и применять их к базе данных, обеспечивая синхронизацию структуры данных.
 - **Как устанавливаются связи между моделями в Django?**
 - С помощью полей `ForeignKey`, `OneToOneField`, `ManyToManyField` и аргументов `related_name`, `on_delete`.
 - **Как сигналы помогают в работе с моделями?**
 - Сигналы позволяют выполнять действия при определенных событиях, например, автоматически создавать профиль при создании пользователя.
-

Полезные ресурсы

- [Документация Django: Модели](#)
- [Документация Django: Миграции](#)
- [Документация Django: Админ-панель](#)
- [Django ORM Cookbook](#)

Практическое занятие 24: CRUD-операции с базой данных в Django

Цель занятия:

- Научиться реализовывать основные операции с данными: создание, чтение, обновление и удаление (CRUD).
 - Познакомиться с механизмами обработки запросов в Django и работой с формами.
-

Задачи занятия

1. Введение в CRUD-операции
 2. Реализация операции создания (Create)
 3. Реализация операции чтения (Read)
 4. Реализация операции обновления (Update)
 5. Реализация операции удаления (Delete)
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Создание своего мероприятия
 8. Выполнение дополнительных заданий. Массовое удаление, подтверждение, импорт/экспорт данных
-

Задача 1: Введение в CRUD-операции

1.1. Что такое CRUD-операции?

- **CRUD** — аббревиатура от Create (Создать), Read (Прочитать), Update (Обновить), Delete (Удалить).
- Это базовые операции, которые используются для работы с данными в большинстве веб-приложений.
- Позволяют пользователям взаимодействовать с базой данных через интерфейс приложения.

1.2. Почему CRUD-операции важны?

- **Управление данными:** Позволяют создавать новые записи, просматривать существующие, вносить изменения и удалять ненужные данные.

- **Пользовательский опыт:** Предоставляют интерфейс для взаимодействия с приложением, делая его полезным и функциональным.
- **Архитектура приложения:** CRUD-операции являются фундаментальными для построения RESTful API и MVC-паттернов.

Задача 2: Реализация операции создания (Create)

2.1. Создание формы `EventForm`

Шаги:

1. Создайте файл `forms.py` в приложении `events`, если его еще нет.
2. Добавьте код для формы:

```
# events/forms.py

from django import forms
from .models import Event

class EventForm(forms.ModelForm):
    class Meta:
        model = Event
        fields = ['title', 'description', 'date', 'location', 'category']
        widgets = {
            'date': forms.DateTimeInput(attrs={'type': 'datetime-local'}),
        }
```

Пояснение:

- **Импортируем `forms` и `Event`:** Используем встроенные формы Django и модель `Event`, которую мы создали ранее.
- **`EventForm`:** Наследуется от `forms.ModelForm`, что упрощает создание формы на основе модели.
- **`Meta` класс:**
 - `model`: Указывает модель, с которой связана форма.
 - `fields`: Определяет, какие поля будут включены в форму.
 - `widgets`: Позволяет настроить виджет для поля `date`, чтобы использовать HTML5-элемент `datetime-local`.

2.2. Создание представления `add_event`

Шаги:

1. Откройте файл `views.py` в приложении `events`.
2. Импортируйте необходимые модули:

```
# events/views.py

from django.shortcuts import render, redirect
from .forms import EventForm
from django.contrib.auth.decorators import login_required
from django.contrib import messages
```

3. Добавьте функцию `add_event`:

```
# events/views.py

@login_required
def add_event(request):
    if request.method == 'POST':
        form = EventForm(request.POST)
        if form.is_valid():
            event = form.save()
            messages.success(request, 'Мероприятие успешно добавлено.')
            return redirect('events:event_detail', event_id=event.id)
    else:
        form = EventForm()
    return render(request, 'events/add_event.html', {'form': form})
```

Пояснение:

- `@login_required`: Декоратор, который требует авторизации пользователя для доступа к этому представлению.
- **Обработка метода POST**:
 - Если форма отправлена, проверяем ее на валидность.
 - Если форма валидна, сохраняем новое мероприятие и перенаправляем пользователя на страницу деталей мероприятия.
- **Обработка метода GET**:
 - Если страница запрошена впервые, отображаем пустую форму для заполнения.
- Используем `messages` для отображения уведомлений пользователю.

2.3. Создание шаблона `add_event.html`

Шаги:

1. Создайте файл `add_event.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/add_event.html -->

{% extends 'events/base.html' %}

{% block title %}Добавить мероприятие{% endblock %}

{% block content %}
    <h1>Добавить мероприятие</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Сохранить</button>
    </form>
    <a href="{% url 'events:event_list' %}">Назад к списку мероприятий</a>
{% endblock %}
```

Пояснение:

- `{% extends 'events/base.html' %}`: Шаблон наследуется от базового.
- `{% csrf_token %}`: Токен защиты от CSRF-атак, обязателен для форм с методом POST.
- `{{ form.as_p }}`: Отображает форму с использованием тегов `<p>`.
- **Ссылка на список мероприятий**: Позволяет пользователю вернуться к списку.

2.4. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `add_event`:

```
# events/urls.py

from django.urls import path
from . import views

urlpatterns = [
    # ... другие маршруты ...
    path('events/add/', views.add_event, name='add_event'),
]
```

Задача 3: Реализация операции чтения (Read)

3.1. Создание представления `event_detail`

Шаги:

1. Откройте файл `views.py`.
2. Импортируйте необходимые модули:

```
# events/views.py

from django.shortcuts import render, get_object_or_404
from .models import Event
```

3. Отредактируйте функцию `event_detail`:

```
# events/views.py

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    comments = event.comments.all().order_by('-created_at')
    return render(request, 'events/event_detail.html', {
        'event': event,
        'comments': comments
    })
```

Пояснение:

- `get_object_or_404`: Получает объект по ID или возвращает 404 ошибку, если объект не найден.
- **Получаем комментарии к мероприятию:** Используем связь `event.comments.all()`.

3.2. Редактирование шаблона `event_detail.html`

Шаги:

1. Отредактируйте файл `event_detail.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'events/base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
```



```

<h1>{{ event.title }}</h1>
<p><strong>Дата и время:</strong> {{ event.date|date:"d.m.Y H:i" }}</p>
<p><strong>Место:</strong> {{ event.location }}</p>
<p><strong>Категория:</strong> {{ event.category.name }}</p>
<p>{{ event.description }}</p>

<h2>Отзывы</h2>
{% if event.reviews.exists %}
    {% for review in event.reviews.all %}
        <div>
            <p><strong>{{ review.user.username }}</strong> – Оценка: {{
review.rating }}/5</p>
            <p>{{ review.comment }}</p>
            <p><em>{{ review.created_at|date:"d.m.Y H:i" }}</em></p>
        </div>
    {% endfor %}
{% else %}
    <p>Нет отзывов о данном мероприятии.</p>
{% endif %}

<!-- Ссылки для редактирования и удаления (если пользователь
авторизован) -->
    <a href="{% url 'events:edit_event' event.id %}">Редактировать</a> |
    <a href="{% url 'events:delete_event' event.id %}">Удалить</a>
{% endblock %}

```

Пояснение:

- **Отображаем детали мероприятия:** Используем переменные `event` и его поля.
- **Отображаем отзывы:** Проверяем, есть ли отзывы, и выводим их.
- **Добавляем ссылки на редактирование и удаление:** Отображаются только для авторизованных пользователей.

3.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `event_detail`:

```

# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
]

```

Задача 4: Реализация операции обновления (Update)

4.1. Создание представления `edit_event`

Шаги:

1. Откройте файл `views.py`.
2. Импортируйте форму `EventForm`, если еще не сделали этого:

```
# events/views.py

from .forms import EventForm
```

3. Добавьте функцию `edit_event`:

```
# events/views.py

@login_required
def edit_event(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    if request.method == 'POST':
        form = EventForm(request.POST, instance=event)
        if form.is_valid():
            form.save()
            messages.success(request, 'Мероприятие успешно обновлено.')
            return redirect('events:event_detail', event_id=event.id)
    else:
        form = EventForm(instance=event)
    return render(request, 'events/edit_event.html', {'form': form, 'event': event})
```

Пояснение:

- `instance=event`: Позволяет загрузить существующие данные мероприятия в форму.
- При сохранении: Обновляет существующий объект, а не создает новый.

4.2. Создание шаблона `edit_event.html`

Шаги:

1. Создайте файл `edit_event.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/edit_event.html -->

{% extends 'events/base.html' %}

{% block title %}Редактировать мероприятие{% endblock %}

{% block content %}
    <h1>Редактировать мероприятие: {{ event.title }}</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Сохранить изменения</button>
    </form>
    <a href="{% url 'events:event_detail' event.id %}">Назад к
мероприятию</a>
{% endblock %}
```

4.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `edit_event`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/<int:event_id>/edit/', views.edit_event,
name='edit_event'),
]
```

Задача 5: Реализация операции удаления (Delete)

5.1. Создание представления `delete_event`

Шаги:

1. Откройте файл `views.py`.
2. Импортируйте необходимые модули:

```
# events/views.py
```

```
from django.views.generic import DeleteView
from django.urls import reverse_lazy
```

3. Добавьте класс `EventDeleteView`:

```
# events/views.py

class EventDeleteView(DeleteView):
    model = Event
    template_name = 'events/delete_event.html'
    success_url = reverse_lazy('events:event_list')

    def dispatch(self, request, *args, **kwargs):
        if not request.user.is_authenticated:
            # return redirect('login')
            return super().dispatch(request, *args, **kwargs)
```

Пояснение:

- **Используем класс `DeleteView`**: Предоставляет готовую реализацию для удаления объектов.
- `success_url`: Указывает, куда перенаправить пользователя после успешного удаления.
- **Переопределяем `dispatch`**: Для проверки авторизации пользователя.

5.2. Создание шаблона `delete_event.html`

Шаги:

1. Отредактируйте файл `delete_event.html` в директории `events/templates/events/`.
2. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/delete_event.html -->

{% extends 'events/base.html' %}

{% block title %}Удалить мероприятие{% endblock %}

{% block content %}
    <h1>Удалить мероприятие</h1>
    <p>Вы уверены, что хотите удалить мероприятие "{{ object.title }}"?</p>
    <form method="post">
        {% csrf_token %}
        <button type="submit">Да, удалить</button>
        <a href="{% url 'events:event_detail' object.id %}">Отмена</a>
    </form>
{% endblock %}
```

```
</form>
{% endblock %}
```

5.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `events/urls.py`.
2. Добавьте маршрут для `EventDeleteView`:

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/<int:pk>/delete/', views.EventDeleteView.as_view(),
        name='delete_event'),
]
```

Пояснение:

- Используем `pk` вместо `event_id`: Классовые представления ожидают параметр `pk` по умолчанию.

Задача 6: Добавление категорий

6.1. Создание дополнительного приложения

Поскольку категории нами еще не созданы, добавим их автоматически с помощью дополнительного небольшого приложения.

Измените файл `events/apps.py` следующим образом:

```
# events/apps.py

import os
from django.apps import AppConfig

class EventsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'events'

    def ready(self):
        print("Метод ready() вызван")
        if os.environ.get('RUN_MAIN', None) != 'true':
            # Избегаем выполнения кода при каждом автоперезагрузке сервера
```

```

        print("RUN_MAIN не равен 'true', метод ready() не будет
выполняться полностью")
        return
    else:
        print("RUN_MAIN равен 'true', продолжаем выполнение метода
ready()")

from django.db.utils import OperationalError, ProgrammingError
from django.core.exceptions import ObjectDoesNotExist
from django.db import connections
from .models import Category

try:
    # Проверяем, применены ли миграции
    db_conn = connections['default']
    db_conn.ensure_connection()
    if not Category.objects.exists():
        print("Категории не существуют, создаём их")
        Category.objects.create(name='Музыка',
description='Мероприятия, связанные с музыкой и концертами.')
        Category.objects.create(name='Искусство',
description='Выставки, галереи и другие художественные события.')
        Category.objects.create(name='Спорт',
description='Спортивные мероприятия и соревнования.')
    else:
        print("Категории уже существуют")
except (OperationalError, ProgrammingError) as e:
    # База данных еще не создана или миграции не применены
    print(f"Ошибка при попытке создать категории: {e}")
    pass

```

В файле `events/__init__.py` **добавьте:**

```

# events/__init__.py

default_app_config = 'events.apps.EventsConfig'

```

Важно: В Django 3.2 и выше это не требуется, но в некоторых случаях может помочь.

6.2. Примените все миграции

```

python manage.py makemigrations
python manage.py migrate

```

Перезапустите сервер разработки

Остановите сервер, если он запущен, и запустите его снова:

```
python manage.py runserver
```

6.4. Проверьте вывод в консоли

- Обратите внимание на сообщения, которые выводятся в консоли.

Пример ожидаемого вывода:

```
Метод ready() вызван RUN_MAIN равен 'true', продолжаем выполнение метода  
ready() Категории не существуют, создаём их`
```

Если вы не видите этих сообщений, значит метод `ready` не выполняется или не достигает этих строк кода.

Задача 7: Запуск и проверка приложения

7.1. Запуск сервера разработки

```
python manage.py runserver
```

7.2. Проверка работы приложения

- **Перейдите на страницу добавления мероприятия:**
 - <http://127.0.0.1:8000/events/add/>
- **Создайте новое мероприятие**, заполнив форму.
- **Проверьте страницу деталей мероприятия:**
 - Убедитесь, что отображаются все данные мероприятия и отзывы (если есть).
- **Отредактируйте мероприятие:**
 - Перейдите по ссылке "Редактировать" и измените данные.
- **Удалите мероприятие:**
 - Нажмите на ссылку "Удалить" и подтвердите удаление.

6.3. Устранение возможных проблем

- **Если возникают ошибки**, проверьте следующее:
 - Правильность импортов в файлах `views.py` и `urls.py`.
 - Наличие всех необходимых шаблонов в правильных директориях.
 - Корректность названий маршрутов и их использование в шаблонах.
-

Задача 7: Практическое упражнение

Задание для студентов:

- **Создайте свое собственное мероприятие**, используя форму добавления.
- **Просмотрите** его детали на соответствующей странице.
- **Отредактируйте** мероприятие, изменив некоторые данные.
- **Удалите** мероприятие, убедившись, что оно исчезло из списка.

Цель упражнения:

- Закрепить навыки работы с CRUD-операциями.
- Понять, как пользователь взаимодействует с приложением для управления данными.

Дополнительное задание

1. Реализация операции массового удаления

Цель: Добавить функциональность для массового удаления нескольких мероприятий через админ-панель.

Шаги:

1. Откройте файл `events/admin.py`.
2. Добавьте функцию для массового удаления:

```
# events/admin.py

from django.contrib import admin
from .models import Event

@admin.action(description='Удалить выбранные мероприятия')
def delete_selected_events(modeladmin, request, queryset):
    queryset.delete()

@admin.register(Event)
class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'category')
    actions = [delete_selected_events]
```

Пояснение:

- Создаем пользовательское действие `delete_selected_events`, которое удаляет выбранные мероприятия.
- Добавляем действие в список доступных действий в админ-панели.

2. Добавление подтверждения при удалении

Цель: Реализовать всплывающее окно подтверждения перед удалением мероприятия с помощью JavaScript.

Шаги:

1. Обновите ссылку на удаление в шаблоне `event_detail.html`:

```
<!-- events/templates/events/event_detail.html -->

<!-- ... ваш существующий код ... -->
<a href="{% url 'events:edit_event' event.id %}">Редактировать</a> |
<a href="{% url 'events:delete_event' event.id %}" onclick="return
confirm('Вы уверены, что хотите удалить это мероприятие?');">Удалить</a>
```

Пояснение:

- Добавляем обработчик `onclick`: При нажатии на ссылку появится диалоговое окно с подтверждением.

3. Создание представления для импорта/экспорта данных

Цель: Реализовать возможность импорта и экспорта списка мероприятий в формате CSV.

Шаги:

1. Создание представления для экспорта:

```
# events/views.py

import csv
from django.http import HttpResponse

def export_events_csv(request):
    events = Event.objects.all()
    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = 'attachment; filename="events.csv"'

    writer = csv.writer(response)
    writer.writerow(['Title', 'Description', 'Date', 'Location',
                    'Category'])
```

```
for event in events:
    writer.writerow([event.title, event.description, event.date,
event.location, event.category.name])

return response
```

2. Добавление маршрута в `urls.py` :

```
# events/urls.py

urlpatterns = [
    # ... другие маршруты ...
    path('events/export/csv/', views.export_events_csv,
name='export_events_csv'),
]
```

3. Добавление ссылки на экспорт в шаблон `event_list.html` :

```
<!-- events/templates/events/event_list.html -->

<a href="{% url 'events:export_events_csv' %}">Экспортировать в CSV</a>
```

Пояснение:

- **Представление** `export_events_csv` : Генерирует CSV-файл со списком мероприятий.
- **Ссылка на экспорт**: Позволяет пользователю скачать файл.

Заключение

Поздравляем! Вы успешно:

- Реализовали основные CRUD-операции с базой данных в Django.
- Научились создавать формы и обрабатывать запросы в представлениях.
- Поняли, как работать с шаблонами для отображения данных пользователю.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Как обеспечить безопасность при выполнении CRUD-операций?**
 - Проверять права доступа пользователя.
 - Использовать декораторы `@login_required` и проверять авторство.
 - Обрабатывать возможные исключения и ошибки.
- **Почему важно использовать CSRF-токены в формах?**
 - Для защиты от CSRF-атак, когда злоумышленник пытается выполнить действие от имени пользователя без его ведома.
- **Как можно улучшить пользовательский интерфейс при работе с CRUD-операциями?**
 - Использовать JavaScript для динамического обновления страницы.
 - Применять фреймворки CSS, такие как Bootstrap, для улучшения дизайна.
 - Реализовать AJAX-запросы для обновления данных без перезагрузки страницы.

Полезные ресурсы

- [Документация Django: Работа с формами](#)
- [Документация Django: Представления на основе классов](#)
- [Django CRUD Tutorial](#)
- [Django Messages Framework](#)

Практическое занятие 25: Настройка админ-панели для управления данными

Цель занятия:

- Познакомиться с админ-панелью Django и ее возможностями.
 - Научиться регистрировать модели в админ-панели.
 - Настроить отображение моделей в админ-панели для удобства использования.
 - Реализовать отображение списка мероприятий на сайте.
-

Задачи занятия

1. Введение в админ-панель Django
 2. Регистрация моделей в админ-панели
 3. Базовая настройка отображения моделей
 4. Реализация отображения списка мероприятий (event_list)
 5. Выполнение практического упражнения. Проверка админ-панели и списка мероприятий
 6. **Выполнение дополнительных заданий. Добавление отзывов, улучшение отображения списка мероприятий
-

Задача 1: Введение в админ-панель Django

1.1. Что такое админ-панель Django?

- **Админ-панель** — встроенный инструмент Django для управления данными приложения.
- Позволяет администраторам просматривать, добавлять, изменять и удалять объекты моделей через удобный веб-интерфейс.
- Предоставляет средства аутентификации и авторизации пользователей.

1.2. Почему важно настроить админ-панель?

- **Удобство управления данными:** Администраторы могут легко управлять данными без необходимости писать код или использовать базу данных напрямую.
- **Экономия времени:** Быстрое создание и редактирование данных во время разработки и тестирования.

- **Расширяемость:** Возможность кастомизировать админ-панель под специфические нужды приложения.
-

Задача 2: Начало регистрации моделей в админ-панели

2.1. Создание суперпользователя

Перед началом работы необходимо создать суперпользователя для доступа к админ-панели.

Шаги:

1. Откройте терминал и выполните команду:

```
python manage.py createsuperuser
```

2. Следуйте инструкциям для ввода имени пользователя, адреса электронной почты (опционально) и пароля.

2.2. Открытие файла `admin.py`

- В приложении `events` откройте файл `admin.py`.

2.3. Импорт моделей

Добавьте следующий код:

```
# events/admin.py

from django.contrib import admin
from .models import Category, Event
```

2.4. Применение миграций (если необходимо)

Если вы недавно добавили новые модели или изменения в модели, выполните миграции:

```
python manage.py makemigrations
python manage.py migrate
```

Задача 3: Базовая настройка отображения моделей

3.1. Настройка модели Event в админ-панели

Создайте класс `EventAdmin` :

```
# events/admin.py

class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'location', 'category')
```

Зарегистрируйте модель с использованием класса `EventAdmin` :

```
# events/admin.py

admin.site.register(Event, EventAdmin)
```

Пояснение:

- `list_display` определяет, какие поля отображаются в списке объектов модели в админ-панели.

3.2. Настройка модели Category

Создайте класс `CategoryAdmin` :

```
# events/admin.py

class CategoryAdmin(admin.ModelAdmin):
    list_display = ('name',)
```

Зарегистрируйте модель `Category` с классом `CategoryAdmin` :

```
# events/admin.py

admin.site.register(Category, CategoryAdmin)
```

3.3. Сохранение и запуск сервера

- Сохраните изменения в `admin.py` .
- Запустите сервер разработки, если он не запущен:

```
python manage.py runserver
```

3.4. Проверка админ-панели

- Перейдите по адресу <http://127.0.0.1:8000/admin/>.
- Введите учетные данные суперпользователя.
- Убедитесь, что модели `Event` и `Category` отображаются и имеют настроенные поля в списке объектов.

Задача 4: Реализация отображения списка мероприятий (`event_list`)

4.1. Создание представления `event_list`

Шаги:

1. Откройте файл `views.py` в приложении `events`.
2. Удалите список `events`
3. Добавьте/измените функцию `event_list`:

```
# events/views.py

from django.shortcuts import render
from .models import Event

# сразу после импорта пропишите новую переменную events
events = Event.objects.all().order_by('date')

def event_list(request):
    print(f"Количество мероприятий: {events.count()}")
    return render(request, 'events/event_list.html', {'events':
events})
```

Пояснение:

- Импортируем модель `Event`.
- Получаем все мероприятия из базы данных, сортируя их по дате.
- Используем функцию `render` для отображения шаблона `event_list.html`, передавая список мероприятий.

4.2. Создание шаблона `event_list.html`

Шаги:

1. Создайте директорию `templates/events/` в вашем приложении `events`, если ее еще нет.
2. Создайте файл `event_list.html` в директории `templates/events/`.
3. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/event_list.html -->

{% extends 'base.html' %}

{% block title %}Список мероприятий{% endblock %}

{% block content %}
    <h1>Список мероприятий</h1>
    <ul>
        {% for event in events %}
            <li>
                <a href="{% url 'events:event_detail'
event.id %}">{{ event.title }}</a> -
                {{ event.date|date:"d.m.Y H:i" }} -
                {{ event.location }}
            </li>
            {% empty %}
            <li>Нет доступных мероприятий.</li>
        {% endfor %}
    </ul>
<!-- Пагинация -->
```

Пояснение:

- Используем цикл `{% for event in events %}` для отображения списка мероприятий.
- Проверяем, есть ли мероприятия, с помощью `{% empty %}`.
- Используем фильтр `date` для форматирования даты.
- Ссылка на детальную страницу мероприятия (`event_detail`).

4.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `urls.py` в приложении `events`.
2. Добавьте маршрут для `event_list`:

```
# events/urls.py

from django.urls import path
```



```
from . import views

urlpatterns = [
    # ... другие маршруты ...
    path('events/', views.event_list, name='event_list'),
]
```

4.4. Добавление ссылки на список мероприятий

Шаги:

1. Откройте базовый шаблон вашего проекта `base.html`.
2. Добавьте ссылку на страницу списка мероприятий:

```
<!-- templates/base.html -->

<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:event_list' %}">Мероприятия</a>
</nav>
```

Пояснение:

- Это позволит пользователям перейти на страницу со списком мероприятий из любой страницы сайта.

4.5. Проверка работы

- Перейдите по адресу <http://127.0.0.1:8000/events/>.
- Убедитесь, что список мероприятий отображается корректно.
- Если у вас нет мероприятий, создайте их через админ-панель или ранее созданные формы.

Задача 7: Практическое упражнение: Проверка админ-панели и списка мероприятий

Задание для студентов:

- Создайте несколько мероприятий через админ-панель.
- Перейдите на страницу списка мероприятий и убедитесь, что они отображаются.

- Проверьте ссылки на детали мероприятия, убедитесь, что они ведут на соответствующие страницы (если у вас есть представление `event_detail`).
- Измените данные мероприятий через админ-панель и убедитесь, что изменения отображаются на сайте.

Цель упражнения:

- Закрепить навыки работы с админ-панелью и отображения данных на сайте.
- Понять связь между данными в базе и их отображением на сайте.

Дополнительное задание

1. Добавление поиска и фильтрации в админ-панели

Цель: Улучшить админ-панель для удобства поиска нужных данных.

Шаги:

1. Добавьте в класс `EventAdmin` поля `search_fields` и `list_filter`:

```
# events/admin.py

class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'location', 'category')
    search_fields = ('title', 'description')
    list_filter = ('category', 'date')
```

2. Обновите админ-панель и убедитесь, что поиск и фильтры работают.

2. Улучшение отображения списка мероприятий на сайте

Цель: Сделать список мероприятий на сайте более информативным.

Шаги:

1. Добавьте категорию мероприятия в шаблон `event_list.html`:

```
<!-- events/templates/events/event_list.html -->

<!-- ... ваш существующий код ... -->
{% for event in events %}
    <li>
        <a href="{% url 'event_detail' event.id %}">{{ event.title }}
    </a> -
```

```
        {{ event.date|date:"d.m.Y H:i" }} -  
        {{ event.location }} -  
        Категория: {{ event.category.name }}  
    </li>  
{% empty %}  
    <!-- ... -->  
{% endfor %}
```

2. **Отформатируйте список с помощью таблицы или CSS** для улучшения внешнего вида (опционально).

Заключение

Поздравляем! Вы успешно:

- Настроили админ-панель Django для базового управления данными.
- Научились регистрировать модели и настраивать их отображение в админ-панели.
- Реализовали отображение списка мероприятий на сайте.
- Познакомились с интерфейсом админ-панели и ее возможностями.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в изучении Django!

Вопросы для обсуждения

- **Как админ-панель упрощает управление данными в приложении?**
 - Предоставляет удобный интерфейс для создания, редактирования и удаления данных без необходимости писать код или работать напрямую с базой данных.
 - **Какие преимущества дает реализация списка мероприятий на сайте?**
 - Позволяет пользователям видеть доступные мероприятия, улучшает функциональность сайта, демонстрирует умение работать с представлениями и шаблонами.
-

Полезные ресурсы

- [Документация Django: Админ-панель](#)
- [Введение в админ-панель Django](#)
- [Django Admin Cookbook](#)
- [Django Шаблоны](#)

Практическое занятие 26: Добавление и редактирование данных через админ-панель

Цель занятия:

- Научиться добавлять и редактировать данные через админ-панель Django.
 - Реализовать возможность добавления главных фото и документов (PDF или DOCX) к мероприятиям.
 - Настроить отображение загруженных файлов в админ-панели и на сайте.
-

Задачи занятия

1. Введение в добавление файлов через админ-панель
 2. Настройка модели для загрузки изображений и документов
 3. Настройка админ-панели для работы с файлами
 4. Настройка обработки медиа-файлов в Django
 5. Отображение загруженных файлов на сайте
 6. Выполнение практического упражнения. Добавление мероприятий с файлами через админ-панель
 7. Выполнение дополнительных заданий. Ограничение типов, добавление миниатюр
-

Задача 1: Введение в добавление файлов через админ-панель

1.1. Возможности админ-панели по работе с файлами

- **Админ-панель Django** позволяет загружать и управлять файлами, связанными с моделями.
- **ImageField** и **FileField**: Поля моделей Django, предназначенные для хранения ссылок на загруженные изображения и файлы.
- **Медиа-файлы**: Требуют особой настройки в проекте Django для правильной работы.

1.2. Что мы будем реализовывать

- **Добавление главного изображения** к каждому мероприятию.

- **Добавление документа** (PDF или DOCX) с дополнительной информацией о мероприятии.
 - **Настройка админ-панели** для удобного управления этими файлами.
 - **Отображение файлов** на сайте для пользователей.
-

Задача 2: Настройка модели для загрузки изображений и документов

2.1. Обновление модели Event

Шаги:

1. Откройте файл `events/models.py`.
2. Импортируйте необходимые модули:

```
from django.db import models
from django.utils import timezone
```

3. Добавьте поля для изображения и документа в модель `Event` :

```
# events/models.py

class Event(models.Model):
    # ... существующие поля ...
    title = models.CharField(max_length=200)
    description = models.TextField()
    date = models.DateTimeField()
    location = models.CharField(max_length=200)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)

    # Новые поля
    main_image = models.ImageField(upload_to='event_images/',
                                   blank=True, null=True)
    document = models.FileField(upload_to='event_documents/',
                                blank=True, null=True)

    def __str__(self):
        return self.title
```

Пояснение:

- `main_image` : Поле для загрузки изображения. `upload_to` определяет папку внутри `MEDIA_ROOT` , куда будут сохраняться изображения.

- `document` : Поле для загрузки файлов. Поддерживает любые типы файлов.
`upload_to` определяет папку для документов.

2.2. Применение миграций

После изменения модели необходимо создать и применить миграции.

Шаги:

1. Создайте миграции:

```
python manage.py makemigrations
```

2. Примените миграции:

```
python manage.py migrate
```

Пояснение:

- Эти команды обновят структуру базы данных, добавив новые поля к таблице `Event`.

Задача 3: Настройка админ-панели для работы с файлами

3.1. Обновление `EventAdmin` для отображения новых полей

Шаги:

1. Откройте файл `events/admin.py`.
2. Импортируйте необходимые модули:

```
from django.contrib import admin
from .models import Event, Category
```

3. Обновите класс `EventAdmin` :

```
# events/admin.py

class EventAdmin(admin.ModelAdmin):
```

```

list_display = ('title', 'date', 'location', 'category',
'display_image')
list_filter = ('category', 'date')
search_fields = ('title', 'description')
ordering = ('date',)
readonly_fields = ('display_image_preview',)

fieldsets = (
    (None, {
        'fields': ('title', 'description', 'date', 'location',
'category')
    }),
    ('Media', {
        'fields': ('main_image', 'display_image_preview',
'document')
    }),
)

def display_image(self, obj):
    if obj.main_image:
        return 'Да'
    return 'Нет'
display_image.short_description = 'Изображение'

def display_image_preview(self, obj):
    if obj.main_image:
        return format_html('', obj.main_image.url)
    return 'Нет изображения'
display_image_preview.short_description = 'Предпросмотр
изображения'

```

Пояснение:

- `display_image`: Метод, который отображает, есть ли у мероприятия изображение.
- `display_image_preview`: Метод для отображения предпросмотра изображения в админ-панели.
- `readonly_fields`: Поля, которые нельзя редактировать, но можно просматривать.
- `fieldsets`: Организует поля на странице редактирования в группы.

3.2. Импорт `format_html`

Не забудьте импортировать `format_html` для безопасного отображения HTML в админ-панели.

```

from django.utils.html import format_html

```


3.3. Сохранение и проверка

- Сохраните файл `admin.py`.
 - Перейдите в админ-панель и попробуйте добавить или отредактировать мероприятие.
 - Убедитесь, что можете загружать изображения и документы.
 - Проверьте, что предпросмотр изображения отображается корректно.
-

Задача 4: Настройка обработки медиа-файлов в Django

4.1. Настройка `MEDIA_URL` и `MEDIA_ROOT`

Шаги:

1. Откройте файл `settings.py` вашего проекта.
2. Добавьте в конец файла следующие настройки:

```
# settings.py

import os

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

Пояснение:

- `MEDIA_URL` : URL, по которому будут доступны медиа-файлы.
- `MEDIA_ROOT` : Путь на сервере, где будут храниться загруженные файлы.

4.2. Настройка URL для медиа-файлов

Шаги:

1. Откройте файл `urls.py` вашего проекта (не приложения).
2. Добавьте импорт и настройки для обработки медиа-файлов в режиме разработки:

```
# your_project/urls.py

from django.contrib import admin
from django.urls import path, include
```

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('events.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT)
```

Пояснение:

- `static()` : Позволяет Django обслуживать медиа-файлы во время разработки при включенном режиме `DEBUG` .

4.3. Проверка настроек

- Перезапустите сервер разработки:

```
python manage.py runserver
```

- Убедитесь, что можете загружать файлы через админ-панель и что они сохраняются в директорию `media/` .

Задача 5: Отображение загруженных файлов на сайте

5.1. Обновление шаблона `event_detail.html`

Шаги:

1. Откройте файл `event_detail.html` в директории `templates/events/` .
2. Добавьте отображение изображения и документа:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
```

```

<h1>{{ event.title }}</h1>
<p><strong>Дата и время:</strong> {{ event.date|date:"d.m.Y H:i" }}
</p>
<p><strong>Место:</strong> {{ event.location }}</p>
<p><strong>Категория:</strong> {{ event.category.name }}</p>
<p>{{ event.description }}</p>

{% if event.main_image %}
    
{% endif %}

{% if event.document %}
    <p><a href="{{ event.document.url }}">Скачать дополнительную
информацию</a></p>
{% endif %}

<!-- Остальной код -->
{% endblock %}

```

Пояснение:

- `{{ event.main_image.url }}`: Ссылка на загруженное изображение.
- `{{ event.document.url }}`: Ссылка на загруженный документ.

5.2. Обновление шаблона `event_list.html` (опционально)

Вы можете добавить отображение миниатюры изображения в списке мероприятий.

Шаги:

1. Откройте файл `event_list.html` в директории `templates/events/`.
2. Обновите код цикла по мероприятиям:

```

<!-- events/templates/events/event_list.html -->

{% extends 'base.html' %}

{% block title %}Список мероприятий{% endblock %}

{% block content %}
    <h1>Список мероприятий</h1>
    <ul>
        {% for event in events %}
            <li>
                {% if event.main_image %}
                    
    {% endif %}
    <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> -
    {{ event.date|date:"d.m.Y H:i" }} -
    {{ event.location }}
</li>
{% empty %}
    <li>Нет доступных мероприятий.</li>
{% endfor %}
</ul>
{% endblock %}

```

5.3. Проверка работы

- Перейдите на страницу детали мероприятия и убедитесь, что изображение и ссылка на документ отображаются.
- Перейдите на страницу списка мероприятий и проверьте отображение миниатюр (если добавляли).

Задача 6: Обновление галереи

6.1. Добавление ссылки на список мероприятий

В файле `events/views.py` добавьте следующее:

```

# events/views.py

from django.shortcuts import render
from .models import Event

def gallery(request):
    return render(request, 'events/gallery.html', {'events': events})

```

Пояснение:

- Импортируем модель `Event`.
- Получаем все мероприятия с помощью `Event.objects.all()` выше.
- Передаём список мероприятий в шаблон `gallery.html`.

6.2. Создание или обновление шаблона `gallery.html`

В директории `templates/events/` создайте файл `gallery.html` со следующим содержанием:

```
<!-- events/templates/events/gallery.html -->

{% extends 'base.html' %}

{% block title %}Галерея{% endblock %}

{% block content %}
    <h1>Галерея</h1>
    {% for event in events %}
        {% if event.main_image %}
            
        {% endif %}
    {% endfor %}
{% endblock %}
```

Пояснение:

- Проходимся по всем мероприятиям с помощью цикла `{% for event in events %}`.
- Проверяем, есть ли у мероприятия изображение: `{% if event.main_image %}`.
- Отображаем изображение с помощью ``.

6.3. Добавление маршрута для галереи.

В файле `events/urls.py` добавьте/измените маршрут:

```
# events/urls.py

from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    # ... другие маршруты ...
    path('gallery/', views.gallery, name='gallery'),
]
```

Пояснение:

- Добавляем путь `/gallery/`, который ссылается на представление `gallery`.

Задача 7: Практическое упражнение: Добавление мероприятий с файлами через админ-панель (10 минут)

Задание для студентов:

- **Добавьте новое мероприятие** через админ-панель, загрузив изображение и документ.
- **Проверьте** отображение загруженных файлов на сайте.
- **Отредактируйте мероприятие**, заменив изображение и документ на другие файлы.
- **Удалите файлы** из мероприятия и убедитесь, что сайт корректно отображает отсутствие файлов.

Цель упражнения:

- Закрепить навыки работы с загрузкой и отображением файлов в Django.
- Убедиться в корректности настроек медиа-файлов в проекте.

Дополнительное задание

1. Ограничение типов загружаемых файлов

Цель: Ограничить загрузку файлов только определенных типов (например, только PDF и DOCX для документов).

Шаги:

1. Установите библиотеку `django-cleanup` (опционально):

```
pip install django-cleanup
```

Пояснение:

- `django-cleanup` автоматически удаляет старые файлы при обновлении или удалении модели.

2. Добавьте `FileExtensionValidator` к полю `document`:

```
# events/models.py

from django.core.validators import FileExtensionValidator

class Event(models.Model):
```

```
# ... существующие поля ...
document = models.FileField(
    upload_to='event_documents/',
    blank=True,
    null=True,
    validators=[FileExtensionValidator(allowed_extensions=['pdf',
'docx'])])
)
```

Пояснение:

- `FileExtensionValidator`: Валидатор, который проверяет расширение загружаемого файла.

3. Примените миграции (не обязательно, если вы только добавили валидатор):

```
python manage.py makemigrations
python manage.py migrate
```

4. Проверьте загрузку документов через админ-панель, попробуйте загрузить файл неподдерживаемого типа.

2. Добавление миниатюр изображений с помощью `django-imagekit`

Цель: Автоматически создавать миниатюры изображений для отображения на сайте.

Шаги:

1. Установите библиотеку `django-imagekit`:

```
pip install django-imagekit
```

2. Добавьте `imagekit` в `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'imagekit',
]
```

3. Обновите модель `Event` для создания миниатюр:

```
# events/models.py

from imagekit.models import ImageSpecField
from imagekit.processors import ResizeToFill

class Event(models.Model):
    # ... существующие поля ...
    main_image = models.ImageField(upload_to='event_images/',
blank=True, null=True)
    main_image_thumbnail = ImageSpecField(
        source='main_image',
        processors=[ResizeToFill(100, 50)],
        format='JPEG',
        options={'quality': 60}
    )
```

4. Обновите шаблоны для использования миниатюр:

```
<!-- events/templates/events/event_list.html -->

{% if event.main_image %}
    
{% endif %}
```

Пояснение:

- `ImageSpecField`: Позволяет создавать производные изображения на основе исходного.

Заключение

Поздравляем! Вы успешно:

- Научились добавлять и редактировать данные через админ-панель Django.
- Реализовали возможность загрузки изображений и документов в модели.
- Настроили админ-панель для удобной работы с файлами.
- Настроили обработку и отображение медиа-файлов в проекте Django.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Вопросы для обсуждения

- **Как обеспечить безопасность при загрузке файлов пользователями?**
 - Использовать валидаторы для проверки типов и размеров файлов.
 - Ограничивать доступ к директориям с файлами.
 - Проверять файлы на наличие вредоносного кода.
 - **Почему важно правильно настроить `MEDIA_ROOT` и `MEDIA_URL` ?**
 - Чтобы Django знал, где хранить и как обслуживать медиа-файлы.
 - Неправильная настройка может привести к ошибкам при загрузке или отображении файлов.
 - **Какие дополнительные возможности предоставляет админ-панель для работы с файлами?**
 - Предпросмотр изображений.
 - Возможность удаления и замены файлов.
 - Использование сторонних пакетов для расширения функциональности.
-

Полезные ресурсы

- [Документация Django: Работа с файлами](#)
- [Документация Django: Настройка админ-панели](#)
- [django-imagekit](#)
- [django-cleanup](#)

Практическое занятие 27: Создание и обработка форм — Реализация отзывов без авторизации

Цель занятия:

- Научиться создавать и обрабатывать формы в Django.
 - Реализовать возможность добавления отзывов к мероприятиям.
 - Понять принципы работы с формами и валидацией данных.
-

Задачи занятия

1. Введение в формы Django
 2. Создание модели `Review`
 3. Создание формы `ReviewForm`
 4. Обновление представления `event_detail` для обработки формы
 5. Обновление шаблона `event_detail.html` для отображения формы и отзывов
 6. Выполнение практического упражнения. Добавление отзывов к мероприятиям
 7. Выполнение дополнительных заданий. Реализация капчи, добавление сообщения об успешной отправке отзыва
-

Задача 1: Введение в формы Django

1.1. Что такое формы в Django?

- **Формы** — это способ взаимодействия пользователей с вашим веб-приложением.
- Позволяют собирать данные от пользователей и обрабатывать их на сервере.
- Django предоставляет встроенную поддержку для работы с формами, включая валидацию и защиту от CSRF-атак.

1.2. Почему важно уметь работать с формами?

- **Взаимодействие с пользователями:** Формы — основной способ ввода данных пользователями.

- **Валидация данных:** Обеспечивает корректность и безопасность введенных данных.
 - **Улучшение UX:** Правильно реализованные формы делают приложение более удобным для пользователей.
-

Задача 2: Создание модели `Review`

2.1. Определение модели `Review`

Шаги:

1. Откройте файл `events/models.py`.
2. Добавьте класс `Review`:

```
# events/models.py

from django.db import models

class Review(models.Model):
    event = models.ForeignKey('Event', related_name='reviews',
on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    email = models.EmailField()
    rating = models.PositiveSmallIntegerField(choices=[(i, i) for i in
range(1, 6)])
    comment = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Review by {self.name} for {self.event.title}'
```

Пояснение:

- `event`: Связь с моделью `Event`, к которой относится отзыв.
- `name`: Имя автора отзыва.
- `email`: Электронная почта автора (для обратной связи, если потребуется).
- `rating`: Оценка мероприятия (от 1 до 5).
- `comment`: Текст отзыва.
- `created_at`: Дата и время создания отзыва.

2.2. Применение миграций

Шаги:

1. Создайте миграции:

```
python manage.py makemigrations
```

2. Примените миграции:

```
python manage.py migrate
```

Задача 3: Создание формы `ReviewForm`

3.1. Создание файла `forms.py`

Если у вас уже есть файл `forms.py` в приложении `events`, пропустите этот шаг.

1. Создайте файл `forms.py` в директории `events/`.

3.2. Определение формы `ReviewForm`

Шаги:

1. Добавьте следующий код в `forms.py`:

```
# events/forms.py

from django import forms
from .models import Review

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }
```

Пояснение:

- Наследуемся от `forms.ModelForm` для автоматического создания полей на основе модели.
- `Meta` класс указывает модель и поля, которые будут использоваться в форме.

- `widgets` позволяет задать виджеты для полей. Для `rating` используем выпадающий список.

Задача 4: Обновление представления `event_detail` для обработки формы

4.1. Обновление функции `event_detail`

Шаги:

1. Откройте файл `views.py` в приложении `events`.
2. Импортируйте `ReviewForm`:

```
# events/views.py

from .forms import ReviewForm
```

3. Обновите функцию `event_detail`:

```
# events/views.py

from django.shortcuts import render, get_object_or_404, redirect
from django.contrib import messages

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':
        review_form = ReviewForm(request.POST)
        if review_form.is_valid():
            review = review_form.save(commit=False)
            review.event = event
            review.save()
            messages.success(request, 'Ваш отзыв успешно добавлен.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            messages.error(request, 'Пожалуйста, исправьте
ошибки в форме.')
    else:
        review_form = ReviewForm()

    return render(request, 'events/event_detail.html', {
        'event': event,
```

```
        'reviews': reviews,
        'review_form': review_form,
    })
```

Пояснение:

- Обработываем метод `POST` для добавления нового отзыва.
- Создаем экземпляр формы `ReviewForm` с данными из запроса.
- Проверяем валидность формы и сохраняем отзыв, связывая его с текущим мероприятием.
- Перенаправляем пользователя на ту же страницу после успешного добавления отзыва.
- Если метод `GET`, отображаем пустую форму для заполнения.

Задача 5: Обновление шаблона `event_detail.html` для отображения формы и отзывов

5.1. Отображение отзывов

Шаги:

1. Откройте файл `event_detail.html` в директории `templates/events/`.
2. Обновите раздел отображения отзывов:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <p><strong>Дата и время:</strong> {{ event.date|date:"d.m.Y H:i" }}</p>
    <p><strong>Место:</strong> {{ event.location }}</p>
    <p><strong>Категория:</strong> {{ event.category.name }}</p>
    <p>{{ event.description }}</p>

    {% if event.main_image %}
        
    {% endif %}
{% endblock %}
```

```

{% if event.document %}
    <p><a href="{{ event.document.url }}">Скачать дополнительную
информацию</a></p>
{% endif %}

<h2>Отзывы</h2>
{% if reviews %}
    {% for review in reviews %}
        <div>
            <p><strong>{{ review.name }}</strong> - Оценка: {{
review.rating }}/5</p>
            <p>{{ review.comment }}</p>
            <p><em>{{ review.created_at|date:"d.m.Y H:i" }}</em>
</p>
        </div>
    {% endfor %}
{% else %}
    <p>Нет отзывов о данном мероприятии.</p>
{% endif %}

<!-- Форма добавления отзыва -->
<h2>Оставить отзыв</h2>
<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    {{ review_form.as_p }}
    <button type="submit">Отправить</button>
</form>

<!-- Ссылки для редактирования и удаления (если необходимо) -->
<!-- ... ваш существующий код ... -->
{% endblock %}

```

Пояснение:

- Отображаем список отзывов, используя переменную `reviews`.
- Добавляем форму для добавления нового отзыва, используя `{{ review_form.as_p }}`.

5.2. Отображение сообщений

Шаги:

1. В базовый шаблон `base.html` добавьте отображение сообщений:

```
<!-- templates/base.html -->
```

```
{% if messages %}
    <ul class="messages">
        {% for message in messages %}
            <li{% if message.tags %} class="{{ message.tags }}"{% endif
            %}>{{ message }}</li>
        {% endfor %}
    </ul>
{% endif %}
```

Пояснение:

- Используем фреймворк сообщений **Django** для отображения уведомлений пользователю (например, об успешном добавлении отзыва).

Задача 6: Практическое упражнение: Добавление отзывов к мероприятиям

Задание для студентов:

- Перейдите на страницу любого мероприятия на сайте.
- Добавьте отзыв, заполнив форму.
- Проверьте, что ваш отзыв отображается в списке отзывов для данного мероприятия.
- Попробуйте добавить несколько отзывов с разными данными.

Цель упражнения:

- Закрепить навыки работы с формами в Django.
- Убедиться в корректной работе добавления и отображения отзывов.

Дополнительное задание

1. Добавление капчи для защиты от спама

Цель: Защитить форму от автоматической отправки спам-ботами.

Шаги:

1. Установите пакет `django-simple-captcha`:


```
pip install django-simple-captcha
```

2. Добавьте `captcha` в `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'captcha',
]
```

3. Добавьте маршрут для капчи в `urls.py` вашего проекта:

```
# your_project/urls.py

from django.urls import path, include

urlpatterns = [
    # ... другие маршруты ...
    path('captcha/', include('captcha.urls')),
]
```

4. Обновите `ReviewForm`, добавив поле капчи:

```
# events/forms.py

from captcha.fields import CaptchaField

class ReviewForm(forms.ModelForm):
    captcha = CaptchaField()

    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment', 'captcha']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }
```

5. Примените миграции для капчи:

```
python manage.py migrate
```

6. Проверьте работу формы, убедившись, что капча отображается и проверяется

при отправке.

2. Добавление сообщения об успешном добавлении отзыва

Цель: Улучшить пользовательский интерфейс, показывая пользователю подтверждение об успешном добавлении отзыва.

Шаги:

1. Убедитесь, что в представлении `event_detail` используется `messages.success` при успешном добавлении отзыва:

```
# events/views.py

messages.success(request, 'Ваш отзыв успешно добавлен.')
```

2. Убедитесь, что в шаблоне `base.html` отображаются сообщения, как показано ранее.
3. Проверьте, что сообщение отображается после добавления отзыва.

Заключение

Поздравляем! Вы успешно:

- Научились создавать и обрабатывать формы в Django.
- Реализовали возможность добавления отзывов к мероприятиям.
- Поняли, как работать с моделями, формами и шаблонами для взаимодействия с пользователем.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- Почему важно проверять валидность формы перед сохранением данных?

- Чтобы обеспечить корректность и безопасность данных, предотвратить ошибки и некорректные записи в базе данных.
 - **Как можно улучшить форму отзыва для лучшего пользовательского опыта?**
 - Добавить интерактивные элементы, использовать AJAX для отправки формы без перезагрузки страницы, предоставить мгновенную обратную связь при валидации.
 - **Какие методы защиты форм от спама существуют?**
 - Использование капчи, ограничение частоты отправки, проверка на роботизированные запросы, использование скрытых полей (honeypot).
-

Полезные ресурсы

- [Документация Django: Работа с формами](#)
- [Документация Django: Фреймворк сообщений](#)
- [Django Crispy Forms](#) — библиотека для улучшения отображения форм.
- [django-simple-captcha](#) — библиотека для добавления капчи в формы.

Практическое занятие 28: Работа с валидацией данных и защитой форм

Цель занятия:

- Научиться работать с валидацией данных в формах Django.
 - Реализовать дополнительные проверки и ограничения на вводимые данные.
 - Защитить формы от спама и автоматических отправок.
 - Понять, как обрабатывать ошибки валидации и отображать их пользователю.
-

Задачи занятия

1. Введение в валидацию данных и защиту форм
 2. Добавление пользовательской валидации в форму `ReviewForm`
 3. Реализация защиты формы от спама
 4. Обработка и отображение ошибок валидации
 5. Выполнение практического упражнения. Улучшение формы отзыва
 6. Выполнение дополнительных заданий. Реализация reCAPTCHA, ограничение количества отзывов
-

Задача 1: Введение в валидацию данных и защиту форм

1.1. Почему важна валидация данных?

- **Безопасность:** Предотвращение внедрения вредоносного кода и SQL-инъекций.
- **Корректность данных:** Обеспечение того, что данные соответствуют ожидаемому формату.
- **Улучшение UX:** Предоставление пользователю мгновенной обратной связи о неверно введенных данных.

1.2. Методы валидации в Django

- **Встроенные валидаторы:** Django предоставляет множество встроенных валидаторов, таких как `EmailValidator`, `MaxLengthValidator` и другие.

- **Пользовательские валидаторы:** Можно создавать собственные функции валидации.
- **Методы `clean` и `clean_field`:** Используются для валидации на уровне формы или отдельного поля.

1.3. Защита форм

- **CSRF-токен:** Защищает от Cross-Site Request Forgery атак.
- **Капча:** Предотвращает автоматическую отправку форм ботами.
- **Ограничение частоты отправки:** Предотвращает флуд и перегрузку сервера.

Задача 2: Добавление пользовательской валидации в форму `ReviewForm`

2.1. Проверка уникальности отзыва от пользователя

Задача: Запретить пользователю отправлять более одного отзыва для одного мероприятия с одним и тем же email.

Шаги:

1. Откройте файл `events/forms.py`.
2. Добавьте метод `clean_email` в класс `ReviewForm`:

```
# events/forms.py

from django import forms
from .models import Review

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }

    def __init__(self, *args, **kwargs):
        self.event = kwargs.pop('event', None)
        super().__init__(*args, **kwargs)

    def clean_email(self):
        email = self.cleaned_data.get('email')
```

```

        if Review.objects.filter(event=self.event,
email=email).exists():
            raise forms.ValidationError('Вы уже оставляли отзыв для
этого мероприятия.')
        return email

```

Пояснение:

- **Метод `__init__`**: Принимает дополнительный аргумент `event`, чтобы знать, для какого мероприятия проверять отзывы.
- **Метод `clean_email`**: Проверяет, есть ли уже отзыв от этого email для данного мероприятия.

3. Обновите представление `event_detail`, чтобы передавать `event` в форму:

```

# events/views.py

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':
        review_form = ReviewForm(request.POST, event=event)
        if review_form.is_valid():
            review = review_form.save(commit=False)
            review.event = event
            review.save()
            messages.success(request, 'Ваш отзыв успешно добавлен.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
    else:
        review_form = ReviewForm(event=event)

    return render(request, 'events/event_detail.html', {
        'event': event,
        'reviews': reviews,
        'review_form': review_form,
    })

```

Пояснение:

- Передаем параметр `event` в форму при создании ее экземпляра.

2.2. Ограничение длины комментария

Задача: Ограничить максимальную длину комментария до 500 символов.

Шаги:

1. Добавьте валидатор к полю `comment` в модели `Review`:

```
# events/models.py

from django.core.validators import MaxLengthValidator

class Review(models.Model):
    # ... существующие поля ...
    comment = models.TextField(validators=[MaxLengthValidator(500)])
    # ... остальной код ...
```

2. Проверьте, что при вводе более 500 символов появляется ошибка валидации.

Задача 3: Реализация защиты формы от спама

3.1. Добавление скрытого поля (honeypot)

Задача: Использовать скрытое поле, которое боты заполняют, а реальные пользователи нет.

Шаги:

1. Добавьте скрытое поле в форму `ReviewForm`:

```
# events/forms.py

class ReviewForm(forms.ModelForm):
    honeypot = forms.CharField(required=False,
                               widget=forms.HiddenInput, label="Leave empty")

    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }

    # ... остальной код ...
```

2. Добавьте проверку этого поля в методе `clean`:

```
# events/forms.py

def clean(self):
    cleaned_data = super().clean()
    honeypot = cleaned_data.get('honeypot')
    if honeypot:
        raise forms.ValidationError('Пожалуйста, оставьте это поле пустым.')
    return cleaned_data
```

Пояснение:

- Поле `honeypot` не отображается для пользователей, но боты могут его заполнить.
- Если поле заполнено, мы генерируем ошибку валидации.

3. Убедитесь, что поле не отображается в шаблоне:

```
<!-- events/templates/events/event_detail.html -->

<!-- ... ваш существующий код ... -->

<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    {{ review_form.as_p }}
    <button type="submit">Отправить</button>
</form>
```

Пояснение:

- Поле с виджетом `HiddenInput` не будет отображаться в форме.

Задача 4: Обработка и отображение ошибок валидации

4.1. Отображение ошибок формы в шаблоне

Шаги:

1. Обновите форму в шаблоне `event_detail.html`, чтобы отображать ошибки:

```
<!-- events/templates/events/event_detail.html -->

<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    <div>
```



```

        {{ review_form.name.label_tag }}
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <div>
        {{ review_form.email.label_tag }}
        {{ review_form.email }}
        {{ review_form.email.errors }}
    </div>
    <div>
        {{ review_form.rating.label_tag }}
        {{ review_form.rating }}
        {{ review_form.rating.errors }}
    </div>
    <div>
        {{ review_form.comment.label_tag }}
        {{ review_form.comment }}
        {{ review_form.comment.errors }}
    </div>
    <button type="submit">Отправить</button>
</form>

```

Пояснение:

- Отдельно отображаем ошибки для каждого поля, используя `{{ field.errors }}`.
- `non_field_errors` отображает ошибки, связанные с формой в целом.

4.2. Стилизация ошибок

Шаги:

1. Добавьте стили для отображения ошибок в ваш CSS-файл или внутри тега `<style>`:

```

<style>
.errorlist {
    color: red;
    list-style-type: none;
}
.errorlist li {
    margin-left: 0;
}
</style>

```

Пояснение:

- Стилизуем ошибки, чтобы они были заметны пользователю.

4.3. Обработка исключений в представлении

Шаги:

1. Обновите представление `event_detail`, чтобы обработать возможные исключения:

```
# events/views.py

from django.core.exceptions import ValidationError

def event_detail(request, event_id):
    # ... ваш существующий код ...

    if request.method == 'POST':
        review_form = ReviewForm(request.POST, event=event)
        try:
            if review_form.is_valid():
                review = review_form.save(commit=False)
                review.event = event
                review.save()
                messages.success(request, 'Ваш отзыв успешно
добавлен.')
                return redirect('events:event_detail',
event_id=event.id)
            else:
                messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
        except ValidationError as e:
            review_form.add_error(None, e)
    else:
        review_form = ReviewForm(event=event)

    # ... остальной код ...
```

Пояснение:

- Обработываем исключения `ValidationError`, которые могут возникнуть при сохранении модели.
- Добавляем ошибки в форму, чтобы они отображались пользователю.

Задача 5: Практическое упражнение: Улучшение формы отзыва

Задание для студентов:

- Добавьте проверку на нецензурные слова в комментариях.

Подсказка: Создайте пользовательский валидатор для поля `comment`, который будет проверять наличие запрещенных слов.

- Убедитесь, что при вводе нецензурных слов появляется соответствующая ошибка.

Цель упражнения:

- Научиться создавать пользовательские валидаторы для полей формы.
- Закрепить навыки работы с валидацией данных и обработкой ошибок.

Шаг 1: Создание списка запрещенных слов

Примечание: Поскольку мы не можем использовать реальные нецензурные слова, вы можете создать свой список запрещенных слов в виде примера.

Пример:

```
# events/forms.py

PROFANITY_WORDS = ['badword1', 'badword2', 'badword3']
```

Пояснение:

- `PROFANITY_WORDS` — список слов, которые вы считаете нецензурными или нежелательными в комментариях.

Шаг 2: Создание пользовательского валидатора

Мы создадим функцию-валидатор, которая будет проверять наличие запрещенных слов в комментарии.

Код:

```
# events/forms.py

import re
from django.core.exceptions import ValidationError

def validate_no_profanity(value):
    for bad_word in PROFANITY_WORDS:
        if re.search(r'\b' + re.escape(bad_word) + r'\b', value,
re.IGNORECASE):
            raise ValidationError('Ваш комментарий содержит недопустимые
слова.')
```

Пояснение:

- Используем регулярные выражения для поиска запрещенных слов в тексте комментария.
- `re.escape(bad_word)` обеспечивает корректную обработку специальных символов.
- `\b` в регулярном выражении обозначает границы слова, чтобы избежать частичных совпадений.
- Если запрещенное слово найдено, вызывается `ValidationError`.

Шаг 3: Добавление валидатора к полю `comment` в форме

Код:

```
# events/forms.py

from django import forms
from .models import Review

PROFANITY_WORDS = ['badword1', 'badword2', 'badword3']

def validate_no_profanity(value):
    for bad_word in PROFANITY_WORDS:
        if re.search(r'\b' + re.escape(bad_word) + r'\b', value,
re.IGNORECASE):
            raise ValidationError('Ваш комментарий содержит недопустимые
слова.')

class ReviewForm(forms.ModelForm):
    # ... ваш существующий код ...

    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1, 6)]),
        }

    comment = forms.CharField(
        widget=forms.Textarea,
        validators=[validate_no_profanity],
        max_length=500,
        label='Комментарий'
    )

    # ... остальной код ...
```

Пояснение:

- Добавляем валидатор `validate_no_profanity` к полю `comment`.
- Определяем поле `comment` явно в форме, чтобы можно было добавить валидатор.
- Указываем `max_length=500`, если нужно ограничить длину комментария.

Шаг 4: Обновление обработки ошибок в шаблоне

Убедитесь, что ошибки валидации для поля `comment` отображаются пользователю.

Код:

```
<!-- events/templates/events/event_detail.html -->

<!-- ... ваш существующий код ... -->

<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    <div>
        {{ review_form.name.label_tag }}
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <div>
        {{ review_form.email.label_tag }}
        {{ review_form.email }}
        {{ review_form.email.errors }}
    </div>
    <div>
        {{ review_form.rating.label_tag }}
        {{ review_form.rating }}
        {{ review_form.rating.errors }}
    </div>
    <div>
        {{ review_form.comment.label_tag }}
        {{ review_form.comment }}
        {{ review_form.comment.errors }}
    </div>
    <button type="submit">Отправить</button>
</form>
```

Пояснение:

- Обязательно выводим `{{ review_form.comment.errors }}`, чтобы пользователь видел сообщения об ошибках валидации комментария.

Шаг 5: Тестирование

1. Добавьте запрещенные слова в список `PROFANITY_WORDS`.

Например:

```
PROFANITY_WORDS = ['foo', 'bar', 'baz']
```

2. Попробуйте отправить комментарий, содержащий одно из запрещенных слов.
3. Убедитесь, что появляется сообщение об ошибке:

```
Ваш комментарий содержит недопустимые слова.
```

4. Попробуйте отправить комментарий без запрещенных слов.
5. Убедитесь, что отзыв успешно добавляется.

Полный код с учетом всех изменений

Файл `events/forms.py`

```
# events/forms.py

import re
from django import forms
from django.core.exceptions import ValidationError
from .models import Review

PROFANITY_WORDS = ['foo', 'bar', 'baz'] # Замените на свои запрещенные слова

def validate_no_profanity(value):
    for bad_word in PROFANITY_WORDS:
        if re.search(r'\b' + re.escape(bad_word) + r'\b', value,
re.IGNORECASE):
            raise ValidationError('Ваш комментарий содержит недопустимые слова.')
    return value

class ReviewForm(forms.ModelForm):
    honeypot = forms.CharField(required=False, widget=forms.HiddenInput,
label="Leave empty")

    comment = forms.CharField(
```

```

        widget=forms.Textarea,
        validators=[validate_no_profanity],
        max_length=500,
        label='Комментарий'
    )

class Meta:
    model = Review
    fields = ['name', 'email', 'rating', 'comment']
    widgets = {
        'rating': forms.Select(choices=[(i, i) for i in range(1, 6)]),
    }

def __init__(self, *args, **kwargs):
    self.event = kwargs.pop('event', None)
    super().__init__(*args, **kwargs)

def clean_email(self):
    email = self.cleaned_data.get('email')
    if Review.objects.filter(event=self.event, email=email).exists():
        raise forms.ValidationError('Вы уже оставляли отзыв для этого мероприятия.')
    return email

def clean(self):
    cleaned_data = super().clean()
    honeypot = cleaned_data.get('honeypot')
    if honeypot:
        raise forms.ValidationError('Пожалуйста, оставьте это поле пустым.')
    return cleaned_data

```

Файл `events/views.py`

Убедитесь, что представление передает ошибки формы в шаблон.

```

# events/views.py

from django.shortcuts import render, get_object_or_404, redirect
from django.contrib import messages
from django.core.exceptions import ValidationError
from .models import Event
from .forms import ReviewForm

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':

```

```

review_form = ReviewForm(request.POST, event=event)
try:
    if review_form.is_valid():
        review = review_form.save(commit=False)
        review.event = event
        review.save()
        messages.success(request, 'Ваш отзыв успешно добавлен.')
        return redirect('events:event_detail', event_id=event.id)
    else:
        messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
except ValidationError as e:
    review_form.add_error(None, e)
else:
    review_form = ReviewForm(event=event)

return render(request, 'events/event_detail.html', {
    'event': event,
    'reviews': reviews,
    'review_form': review_form,
})

```

Файл `events/templates/events/event_detail.html`

```

<!-- events/templates/events/event_detail.html -->

{% extends 'base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <!-- ... остальной контент мероприятия ... -->

    <h2>Отзывы</h2>
    {% if reviews %}
        {% for review in reviews %}
            <div>
                <p><strong>{{ review.name }}</strong> - Оценка: {{
review.rating }}/5</p>
                <p>{{ review.comment }}</p>
                <p><em>{{ review.created_at|date:"d.m.Y H:i" }}</em></p>
            </div>
        {% endfor %}
    {% else %}
        <p>Нет отзывов о данном мероприятии.</p>
    {% endif %}

```



```
<!-- Форма добавления отзыва -->
<h2>Оставить отзыв</h2>
<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    <div>
        {{ review_form.name.label_tag }}
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <div>
        {{ review_form.email.label_tag }}
        {{ review_form.email }}
        {{ review_form.email.errors }}
    </div>
    <div>
        {{ review_form.rating.label_tag }}
        {{ review_form.rating }}
        {{ review_form.rating.errors }}
    </div>
    <div>
        {{ review_form.comment.label_tag }}
        {{ review_form.comment }}
        {{ review_form.comment.errors }}
    </div>
    <button type="submit">Отправить</button>
</form>
{% endblock %}
```

Дополнительные рекомендации

- **Расширение списка запрещенных слов:**
 - Вы можете загрузить список запрещенных слов из файла или использовать сторонние библиотеки для фильтрации нецензурной лексики.
- **Использование сторонних пакетов:**
 - Есть готовые библиотеки для фильтрации нецензурных слов, например, `django-profanity-check` или `profanity-filter`.
 - Установка:

```
pip install profanity-filter
```

- Использование:

```
from profanity_filter import ProfanityFilter

pf = ProfanityFilter()

def validate_no_profanity(value):
    if pf.is_profane(value):
        raise ValidationError('Ваш комментарий содержит
недопустимые слова.')
    return value
```

- **Учтите варианты написания слов:**

- Пользователи могут обходить фильтр, заменяя буквы на похожие символы или пропуская буквы.
- Более сложные алгоритмы и библиотеки могут помочь в таких случаях.

Вы успешно реализовали проверку на нецензурные слова в комментариях, создав пользовательский валидатор для поля `comment`. Теперь при вводе запрещенных слов пользователи будут получать сообщение об ошибке, и отзыв не будет сохранен до исправления комментария.

Дополнительное задание

1. Реализация проверки reCAPTCHA

Цель: Добавить защиту формы с использованием Google reCAPTCHA.

Шаги:

1. **Зарегистрируйтесь на сайте [Google reCAPTCHA](#)** и получите ключи сайта и секретный ключ.
2. **Установите пакет `django-recaptcha`:**

```
pip install django-recaptcha
```

3. **Добавьте `captcha` в `INSTALLED_APPS` в `settings.py`:**

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
```

```
'captcha',  
]
```

4. Добавьте ключи reCAPTCHA в настройки:

```
# settings.py  
  
RECAPTCHA_PUBLIC_KEY = 'ваш_публичный_ключ'  
RECAPTCHA_PRIVATE_KEY = 'ваш_секретный_ключ'  
RECAPTCHA_USE_SSL = True
```

5. Обновите форму ReviewForm:

```
# events/forms.py  
  
from captcha.fields import ReCaptchaField  
from captcha.widgets import ReCaptchaV2Checkbox  
  
class ReviewForm(forms.ModelForm):  
    captcha = ReCaptchaField(widget=ReCaptchaV2Checkbox)  
  
    class Meta:  
        model = Review  
        fields = ['name', 'email', 'rating', 'comment', 'captcha']  
        widgets = {  
            'rating': forms.Select(choices=[(i, i) for i in range(1,  
6)]),  
        }  
  
# ... остальной код ...
```

6. Проверьте работу формы и убедитесь, что reCAPTCHA отображается и работает корректно.

2. Ограничение количества отзывов от одного пользователя в день

Цель: Предотвратить спам, ограничив количество отзывов, которые пользователь может оставить за день.

Шаги:

1. Добавьте проверку в методе `clean_email` формы ReviewForm:

```
# events/forms.py
```

```
from django.utils import timezone
from datetime import timedelta

def clean_email(self):
    email = self.cleaned_data.get('email')
    time_threshold = timezone.now() - timedelta(days=1)
    reviews_count = Review.objects.filter(email=email,
created_at__gte=time_threshold).count()
    if reviews_count >= 5:
        raise forms.ValidationError('Вы превысили лимит отзывов на
сегодня.')
```

Пояснение:

- Проверяем количество отзывов, оставленных с данного email за последние 24 часа.
- Устанавливаем лимит, например, 5 отзывов в день.

Заключение

Поздравляем! Вы успешно:

- Научились добавлять пользовательскую валидацию в формы Django.
- Реализовали защиту форм от спама и автоматических отправок.
- Научились обрабатывать и отображать ошибки валидации для пользователей.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- Почему важно проводить валидацию данных как на стороне клиента, так и на стороне сервера?
- Валидация данных на стороне клиента обеспечивает мгновенную обратную связь пользователю и уменьшает нагрузку на сервер, предотвращая отправку некорректных данных. Однако она может быть обойдена злоумышленниками, поэтому валидация на стороне сервера необходима для обеспечения

безопасности и целостности данных, гарантируя, что все входящие данные проверены и соответствуют требованиям приложения.

- **Какие еще методы защиты форм вы можете предложить?**
 - Помимо валидации данных и использования капчи, можно применять следующие методы защиты форм: ограничение частоты отправки форм (rate limiting), использование токенов CSRF для предотвращения межсайтовых запросов, проверка заголовков HTTP-запросов, применение скрытых полей (honeypot), шифрование чувствительных данных, а также использование безопасных протоколов передачи данных, таких как HTTPS.
 - **Как обеспечить баланс между удобством использования формы и ее безопасностью?**
 - Для обеспечения баланса между удобством и безопасностью формы следует внедрять интуитивно понятные интерфейсы с четкой структурой и обратной связью, избегать излишней сложности для пользователя, одновременно применяя необходимые меры безопасности, такие как валидация и защита от CSRF. Использование вспомогательных инструментов, таких как всплывающие подсказки и авто-заполнение, также может повысить удобство без ущерба для безопасности. Важно проводить тестирование UX и безопасности, чтобы найти оптимальный баланс, удовлетворяющий как пользователей, так и требования безопасности.
-

Полезные ресурсы

- [Документация Django: Валидация форм](#)
- [Документация Django: Валидаторы](#)
- [django-recaptcha](#)
- [django-ratelimit](#)

Практическое занятие 29: Реализация регистрации и входа пользователя

Цель занятия:

- Научиться реализовывать регистрацию новых пользователей в Django-приложении.
 - Настроить механизм аутентификации пользователей с помощью встроенной системы Django.
 - Понять, как создавать формы регистрации и входа, а также обрабатывать их данные.
 - Обеспечить возможность пользователям входить в систему и выходить из нее.
-

Задачи занятия

1. Введение в систему аутентификации Django
 2. Создание формы регистрации пользователя
 3. Реализация представления для регистрации
 4. Создание шаблона регистрации
 5. Настройка входа пользователя
 6. Создание шаблона входа
 7. Настройка выхода пользователя
 8. Добавление ссылок на регистрацию и вход в навигацию
 9. Тестирование функциональности
-

Задача 1: Введение в систему аутентификации Django

1.1. Что такое система аутентификации Django?

- **Django Authentication System** — встроенная система Django для управления пользователями, их регистрацией, входом и выходом.
- Предоставляет готовые модели, формы и представления для работы с пользователями.
- Позволяет быстро и безопасно внедрять аутентификацию в ваше приложение.

1.2. Почему мы используем встроенную систему аутентификации?

- **Безопасность:** Django тщательно протестировал и проверил свою систему аутентификации на безопасность.
- **Удобство:** Многие задачи уже реализованы, вам нужно только настроить их под свои нужды.
- **Расширяемость:** Систему можно расширять и настраивать под требования вашего приложения.

Задача 2: Создание формы регистрации пользователя

2.1. Использование встроенной формы `UserCreationForm`

Шаги:

1. Создайте файл `forms.py` в вашем приложении (если его нет).

```
touch events/forms.py
```

2. Импортируйте `UserCreationForm` и создайте свою форму на его основе:

```
# events/forms.py

from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User

class SignUpForm(UserCreationForm):
    email = forms.EmailField(max_length=254, required=True,
        help_text='Обязательное поле. Введите действительный адрес электронной почты.')

    class Meta:
        model = User
        fields = ('username', 'email', 'password1', 'password2')
```

Пояснение:

- **Наследуемся от `UserCreationForm`**, чтобы использовать встроенные методы и валидацию.

- Добавляем поле `email`, так как по умолчанию оно не включено.
 - Определяем класс `Meta`, чтобы указать, какие поля будут в форме.
-

Задача 3: Реализация представления для регистрации

3.1. Создание функции `signup` в `views.py`

Шаги:

1. Откройте `views.py` в вашем приложении `events`.
2. Импортируйте необходимые модули и формы:

```
# events/views.py

from django.shortcuts import render, redirect
from django.contrib.auth import login
from django.contrib import messages
from .forms import SignUpForm
```

3. Добавьте функцию `signup`:

```
# events/views.py

def signup(request):
    if request.method == 'POST':
        form = SignUpForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, 'Регистрация прошла успешно!')
            return redirect('events:home') # Замените на вашу главную
            страницу
        else:
            messages.error(request, 'Пожалуйста, исправьте ошибки
            ниже.')
    else:
        form = SignUpForm()
    return render(request, 'events/signup.html', {'form': form})
```

Пояснение:

- Обработываем метод `POST`: Если форма отправлена, проверяем ее валидность.

- Сохраняем нового пользователя и выполняем автоматический вход с помощью `login(request, user)`.
 - Перенаправляем пользователя на главную страницу или другую страницу по вашему выбору.
 - При методе `GET` отображаем пустую форму регистрации.
-

Задача 4: Создание шаблона регистрации

4.1. Создание файла `signup.html`

Шаги:

1. Создайте файл `signup.html` в директории `templates/events/`:

```
mkdir -p events/templates/events
touch events/templates/events/signup.html
```

2. Добавьте следующий код в `signup.html`:

```
<!-- events/templates/events/signup.html -->

{% extends 'events/base.html' %}

{% block title %}Регистрация{% endblock %}

{% block content %}
    <h2>Регистрация</h2>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Зарегистрироваться</button>
    </form>
{% endblock %}
```

Пояснение:

- Используем базовый шаблон `base.html`.
 - Отображаем форму регистрации, используя `{{ form.as_p }}` для простого отображения.
-

Задача 5: Настройка входа пользователя

5.1. Использование встроенных представлений Django

Django предоставляет готовые представления для входа и выхода пользователя.

Шаги:

1. Импортируйте необходимые модули в `urls.py`:

```
# events/urls.py

from django.urls import path
from django.contrib.auth import views as auth_views
from . import views

app_name = 'events'

urlpatterns = [
    # ... другие маршруты ...
    path('signup/', views.signup, name='signup'),
    path('login/',
    auth_views.LoginView.as_view(template_name='events/login.html'),
    name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
]
```

Пояснение:

- Используем `LoginView` и `LogoutView` из `django.contrib.auth.views`.
- Указываем `template_name` для `LoginView`, чтобы использовать наш шаблон.

5.2. Создание шаблона входа `login.html`

Шаги:

1. Создайте файл `login.html` в директории `templates/events/`:

```
touch events/templates/events/login.html
```

2. Добавьте следующий код в `login.html`:

```
<!-- events/templates/events/login.html -->

{% extends 'events/base.html' %}
```

```
{% block title %}Вход{% endblock %}

{% block content %}
    <h2>Вход</h2>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Войти</button>
    </form>
    <p>Нет аккаунта? <a href="{% url 'events:signup' %}">Зарегистрируйтесь</a></p>
{% endblock %}
```

Пояснение:

- Отображаем форму входа, предоставляем ссылку на регистрацию.

Задача 6: Настройка выхода пользователя

6.1. Добавление ссылки на выход в навигацию

Шаги:

1. Откройте `base.html` и добавьте ссылки на вход и выход:

```
<!-- templates/base.html -->
```

```
{% if user.is_authenticated %} Hello, {{ user.username }}! {% csrf_token %} Logout {% else %} Login Sign Up {% endif %}
```

Пояснение:

- Проверяем, аутентифицирован ли пользователь, и отображаем соответствующие ссылки.
- Используем `user.is_authenticated`, который всегда доступен в шаблонах благодаря контекстному процессору.

Задача 7: Настройка перенаправления после входа и выхода

7.1. Настройка URL перенаправления в `settings.py`

Шаги:

1. Откройте `settings.py` и добавьте следующие настройки:

```
# settings.py

LOGIN_REDIRECT_URL = 'events:home' # Замените на ваше имя маршрута для
главной страницы
LOGOUT_REDIRECT_URL = 'events:home' # Или другую страницу
```

Пояснение:

- `LOGIN_REDIRECT_URL` : URL, на который будет перенаправлен пользователь после успешного входа.
- `LOGOUT_REDIRECT_URL` : URL для перенаправления после выхода.

Задача 8: Добавление ссылок на регистрацию и вход в навигацию

8.1. Обновление навигации в `base.html`

Мы уже сделали это в предыдущих шагах, но убедитесь, что ваша навигация выглядит примерно так:

```
<!-- templates/base.html -->

<nav>
  <!-- ... другие ссылки ... -->
  {% if user.is_authenticated %}
    Привет, {{ user.username }}!
    <a href="{% url 'events:logout' %}">Выйти</a>
  {% else %}
    <a href="{% url 'events:login' %}">Войти</a>
    <a href="{% url 'events:signup' %}">Регистрация</a>
  {% endif %}
</nav>
```

Задача 9: Тестирование функциональности

9.1. Проверка регистрации

- Перейдите по адресу: <http://127.0.0.1:8000/signup/>
- Заполните форму регистрации, убедитесь, что происходит успешная регистрация и вход.

9.2. Проверка входа и выхода

- Выйдите из системы, перейдя по ссылке "Выйти".
- Перейдите по адресу: <http://127.0.0.1:8000/login/>
- Войдите с помощью созданных учетных данных.
- Убедитесь, что перенаправление происходит корректно.

Дополнительные задания

1. Добавление валидации на уникальность email

Цель: Убедиться, что каждый email адрес регистрируется только один раз.

Шаги:

1. Создайте пользовательский валидатор для поля email в SignUpForm:

```
# events/forms.py

from django.core.exceptions import ValidationError

class SignUpForm(UserCreationForm):
    email = forms.EmailField(max_length=254, required=True,
        help_text='Обязательное поле. Введите действительный адрес электронной почты.')

    class Meta:
        model = User
        fields = ('username', 'email', 'password1', 'password2')

    def clean_email(self):
        email = self.cleaned_data.get('email')
        if User.objects.filter(email=email).exists():
            raise ValidationError('Пользователь с таким email уже существует.')
        return email
```

Пояснение:

- Проверяем, существует ли пользователь с таким же email.
- Вызываем `ValidationError`, если email уже используется.

2. Добавление возможности восстановления пароля

Цель: Позволить пользователям сбрасывать пароль, если они его забыли.

Шаги:

1. Добавьте маршруты для восстановления пароля в `urls.py`:

```
# events/urls.py

from django.contrib.auth import views as auth_views

urlpatterns += [
    path('password_reset/',
         auth_views.PasswordResetView.as_view(template_name='events/password_reset.html'), name='password_reset'),
    path('password_reset/done/',
         auth_views.PasswordResetDoneView.as_view(template_name='events/password_reset_done.html'), name='password_reset_done'),
    path('reset/<uidb64>/<token>/',
         auth_views.PasswordResetConfirmView.as_view(template_name='events/password_reset_confirm.html'), name='password_reset_confirm'),
    path('reset/done/',
         auth_views.PasswordResetCompleteView.as_view(template_name='events/password_reset_complete.html'), name='password_reset_complete'),
]
```

2. Создайте необходимые шаблоны и настройте отправку почты в `settings.py`.

Пояснение:

- В рамках данного занятия настройка восстановления пароля может быть сложной, так как требует настройки отправки электронной почты.
- Рекомендуется выполнить это задание после изучения соответствующих материалов.

Заключение

Поздравляем! Вы успешно:

- Реализовали регистрацию новых пользователей в вашем приложении.

- Настроили систему аутентификации с использованием встроенных возможностей Django.
 - Создали формы и шаблоны для регистрации и входа пользователей.
 - Обеспечили возможность входа и выхода из системы.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!

Успехов в разработке!

Вопросы для обсуждения

- **Почему важно использовать встроенную систему аутентификации Django вместо создания собственной?**
 - Использование встроенной системы аутентификации Django обеспечивает высокую степень безопасности и надежности, так как она тщательно протестирована и поддерживается сообществом. Это экономит время разработки, позволяя избежать ошибок и уязвимостей, связанных с самостоятельной реализацией механизмов аутентификации. Кроме того, встроенная система легко интегрируется с другими компонентами Django, обеспечивая совместимость и расширяемость.
 - **Как обеспечить безопасность данных при регистрации и входе пользователей?**
 - Для обеспечения безопасности данных при регистрации и входе пользователей следует использовать HTTPS для шифрования передаваемых данных, применять защиту от CSRF-атак с помощью токенов, использовать сильные хеши для паролей (Django автоматически хеширует пароли пользователей), а также внедрять ограничения на количество попыток входа для предотвращения атак перебором. Дополнительно рекомендуется регулярно обновлять зависимости и следить за уязвимостями в используемых пакетах.
 - **Какие преимущества предоставляет использование классов-представлений (Class-Based Views) для реализации регистрации и входа пользователей?**
 - Классы-представления в Django предлагают более структурированный и повторно используемый подход к разработке, позволяя легко наследоваться и расширять функциональность. Они облегчают организацию кода, делают его более читаемым и поддерживаемым. Использование классов-представлений также позволяет эффективно использовать миксины и встроенные методы, что ускоряет процесс разработки и снижает вероятность ошибок.
-

Полезные ресурсы

- [Документация Django: Аутентификация](#)
- [Документация Django: Форма регистрации пользователей](#)
- [Видео-тutorial по регистрации и входу в Django](#)
- [Django Girls Tutorial: Регистрация пользователей](#)

Практическое занятие 30: Настройка прав доступа и ролей пользователей

Цель занятия:

- Настроить права доступа в приложении Django.
 - Ограничить возможность добавления и редактирования мероприятий только администраторам.
 - Разрешить оставлять отзывы только аутентифицированным пользователям.
 - Понять, как использовать декораторы и миксины для контроля доступа.
 - Обновить шаблоны для отображения элементов в зависимости от прав доступа пользователя.
-

Задачи занятия

1. Введение в права доступа и роли в Django
 2. Ограничение доступа к добавлению и редактированию мероприятий
 3. Настройка доступа к добавлению отзывов
 4. Обновление шаблонов в соответствии с правами доступа
 5. Выполнение практического упражнения. Проверка прав доступа
 6. Выполнение дополнительных заданий. Настройка групп и разрешений
-

Задача 1: Введение в права доступа и роли в Django

1.1. Что такое права доступа и роли?

- **Роли пользователей:** Определенные группы пользователей с особыми правами (например, администраторы, редакторы, обычные пользователи).
- **Права доступа:** Разрешения, которые определяют, какие действия может выполнять пользователь в приложении.

1.2. Как Django управляет правами доступа?

- **Декораторы:** Функции, которые можно применять к представлениям для ограничения доступа (например, `@login_required`).

- **Миксины:** Классы, которые можно использовать в классах-представлениях для ограничения доступа (например, `LoginRequiredMixin`).
 - **Система разрешений:** Django предоставляет встроенную систему разрешений на уровне моделей.
-

Задача 2: Ограничение доступа к добавлению и редактированию мероприятий

2.1. Ограничение доступа к представлениям добавления и редактирования мероприятий

Цель: Сделать так, чтобы только администраторы могли добавлять и редактировать мероприятия.

Шаги:

1. **Импортируйте декоратор `user_passes_test` в `views.py`:**

```
# events/views.py

from django.contrib.auth.decorators import user_passes_test
```

2. **Создайте функцию-проверку для определения, является ли пользователь администратором:**

```
# events/views.py

def is_admin(user):
    return user.is_authenticated and user.is_staff
```

3. **Примените декоратор `user_passes_test` к представлениям `add_event` и `edit_event`:**

```
# events/views.py

@user_passes_test(is_admin)
def add_event(request):
    # Ваш существующий код
```

```
# events/views.py

@user_passes_test(is_admin)
```

```
def edit_event(request, event_id):  
    # Ваш существующий код
```

4. Обновите представление удаления мероприятия `EventDeleteView`:

Если вы используете класс-представление, можно использовать `UserPassesTestMixin`:

```
# events/views.py  
  
from django.contrib.auth.mixins import UserPassesTestMixin  
  
class EventDeleteView(UserPassesTestMixin, DeleteView):  
    model = Event  
    template_name = 'events/delete_event.html'  
    success_url = reverse_lazy('events:event_list')  
  
    def test_func(self):  
        return self.request.user.is_authenticated and  
self.request.user.is_staff  
  
    def handle_no_permission(self):  
        return redirect('events:login')
```

Пояснение:

- `test_func`: Метод, который проверяет, имеет ли пользователь необходимые права.
- `handle_no_permission`: Метод, который вызывается, если пользователь не имеет доступа.

2.2. Перенаправление неавторизованных пользователей

- Если неавторизованный пользователь попытается получить доступ к представлению, он будет перенаправлен на страницу входа.

Задача 3: Настройка доступа к добавлению отзывов

3.1. Ограничение доступа к добавлению отзывов только для аутентифицированных пользователей

Цель: Разрешить оставлять отзывы только зарегистрированным и вошедшим в систему пользователям.

Шаги:

1. Импортируйте декоратор `login_required` в `views.py`:

```
# events/views.py

from django.contrib.auth.decorators import login_required
```

2. Обновите представление `event_detail`, чтобы использовать `login_required` для обработки POST запросов:

```
# events/views.py

from django.utils.decorators import method_decorator

@login_required(login_url='events:login')
def post_review(request, event):
    review_form = ReviewForm(request.POST, event=event)
    try:
        if review_form.is_valid():
            review = review_form.save(commit=False)
            review.event = event
            review.user = request.user # Если вы хотите связать отзыв
с пользователем
            review.save()
            messages.success(request, 'Ваш отзыв успешно добавлен.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
    except ValidationError as e:
        review_form.add_error(None, e)
    return review_form # Возвращаем форму с ошибками

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':
        # Если пользователь не аутентифицирован, перенаправляем на
страницу входа
        if not request.user.is_authenticated:
            return redirect('events:login')
        # Обрабатываем отзыв
        review_form = post_review(request, event)
    else:
        review_form = ReviewForm(event=event)

    return render(request, 'events/event_detail.html', {
        'event': event,
```

```
        'reviews': reviews,
        'review_form': review_form,
    })
```

Пояснение:

- Разделяем логику обработки отзыва в отдельную функцию `post_review`, которую декорируем `@login_required`.
- Внутри `event_detail` проверяем, если метод `POST` и пользователь не аутентифицирован, перенаправляем на страницу входа.
- Если пользователь аутентифицирован, вызываем `post_review`.

3. Добавьте поле `user` в модель `Review` (опционально):

Если вы хотите связать отзывы с пользователями, можно добавить поле `user` в модель `Review`.

```
# events/models.py

from django.contrib.auth.models import User

class Review(models.Model):
    # ... существующие поля ...
    user = models.ForeignKey(User, on_delete=models.CASCADE, null=True,
                             blank=True)
    # ... остальной код ...
```

Не забудьте выполнить миграции:

```
python manage.py makemigrations
python manage.py migrate
```

4. Обновите форму `ReviewForm` (если необходимо):

Если вы добавили поле `user` в модель, убедитесь, что оно не отображается в форме:

```
# events/forms.py

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment'] # Не включаем
        поле 'user'
```

Задача 4: Обновление шаблонов в соответствии с правами доступа

4.1. Скрытие кнопок добавления и редактирования мероприятий для неадминистраторов

Шаги:

1. В шаблоне, где находится ссылка на добавление мероприятия (`add_event`), добавьте проверку:

```
<!-- Например, в templates/events/event_list.html -->

{% if user.is_authenticated and user.is_staff %}
    <a href="{% url 'events:add_event' %}">Добавить мероприятие</a>
{% endif %}
```

2. В шаблоне `event_detail.html` скрывайте ссылки на редактирование и удаление для неадминистраторов:

```
<!-- templates/events/event_detail.html -->

{% if user.is_authenticated and user.is_staff %}
    <a href="{% url 'events:edit_event' event.id %}">Редактировать</a>
    <a href="{% url 'events:delete_event' event.id %}">Удалить</a>
{% endif %}
```

4.2. Скрытие формы добавления отзыва для неаутентифицированных пользователей

Шаги:

1. В шаблоне `event_detail.html` оберните форму отзыва проверкой:

```
<!-- templates/events/event_detail.html -->

{% if user.is_authenticated %}
    <!-- Форма добавления отзыва -->
    <h2>Оставить отзыв</h2>
    <form method="post">
        {% csrf_token %}
        {{ review_form.non_field_errors }}
        <div>
            {{ review_form.name.label_tag }}
        </div>
    </form>
{% endif %}
```

```
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <!-- ... остальные поля формы ... -->
    <button type="submit">Отправить</button>
</form>
{% else %}
    <p><a href="{% url 'events:login' %}?next={{ request.path
}}">Войдите</a>, чтобы оставить отзыв.</p>
{% endif %}
```

Пояснение:

- `{% if user.is_authenticated %}` проверяет, вошел ли пользователь в систему.
- **Предоставляем ссылку на вход**, передавая параметр `next` для перенаправления обратно после входа.

Задача 5: Практическое упражнение: Проверка прав доступа

Задание для студентов:

- **Создайте обычного пользователя** и попробуйте добавить или отредактировать мероприятие.
 - Убедитесь, что у вас нет доступа и вы перенаправлены на страницу входа или получаете сообщение об ошибке.
- **Создайте администратора** (встроенный суперпользователь Django) и убедитесь, что у вас есть доступ к добавлению и редактированию мероприятий.
- **Попробуйте оставить отзыв, не войдя в систему.**
 - Убедитесь, что вы не можете этого сделать и вас перенаправляют на страницу входа.

Цель упражнения:

- Убедиться, что права доступа настроены правильно.
- Понять, как поведение приложения меняется в зависимости от ролей пользователей.

Дополнительные задания

1. Настройка групп и разрешений

Цель: Использовать встроенную систему групп и разрешений Django для более гибкой настройки прав доступа.

Шаги:

1. Создайте группы пользователей в админке Django:

- **Администраторы мероприятий:** Пользователи, которые могут добавлять и редактировать мероприятия.
- **Обычные пользователи:** Пользователи, которые могут оставлять отзывы.

2. Назначьте соответствующие разрешения группам:

- **Администраторы мероприятий:**
 - `events.add_event`
 - `events.change_event`
 - `events.delete_event`
- **Обычные пользователи:**
 - `events.add_review`

3. Обновите функции-проверки в `views.py`, чтобы использовать разрешения:

```
# events/views.py

from django.contrib.auth.decorators import permission_required

@permission_required('events.add_event', login_url='events:login')
def add_event(request):
    # Ваш существующий код
```

Пояснение:

- `@permission_required` проверяет, имеет ли пользователь указанное разрешение.
- Убедитесь, что пользователи имеют соответствующие разрешения через группы.

2. Ограничение редактирования и удаления мероприятий только их создателем

Цель: Разрешить редактирование и удаление мероприятия только пользователю, который его создал (например, администратору).

Шаги:

1. Добавьте поле `created_by` в модель `Event`:


```
# events/models.py

from django.contrib.auth.models import User

class Event(models.Model):
    # ... существующие поля ...
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)
    # ... остальной код ...
```

Не забудьте выполнить миграции.

2. При создании мероприятия сохраняйте информацию о создателе:

```
# events/views.py

@user_passes_test(is_admin)
def add_event(request):
    if request.method == 'POST':
        form = EventForm(request.POST)
        if form.is_valid():
            event = form.save(commit=False)
            event.created_by = request.user
            event.save()
            messages.success(request, 'Мероприятие успешно добавлено.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            form = EventForm()
    return render(request, 'events/add_event.html', {'form': form})
```

3. Обновите представления `edit_event` и `EventDeleteView`, чтобы проверять, является ли пользователь создателем мероприятия:

```
# events/views.py

@user_passes_test(is_admin)
def edit_event(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    if event.created_by != request.user:
        return redirect('events:event_detail', event_id=event.id)
    # Остальной код представления
```

```
# events/views.py

class EventDeleteView(UserPassesTestMixin, DeleteView):
    # ... ваш существующий код ...

    def test_func(self):
```

```
event = self.get_object()
return self.request.user.is_authenticated and self.request.user
== event.created_by
```

Заключение

Поздравляем! Вы успешно:

- Настроили права доступа и роли пользователей в вашем приложении.
- Ограничили доступ к определенным действиям только для администраторов.
- Разрешили оставлять отзывы только аутентифицированным пользователям.
- Научились использовать декораторы и миксины для контроля доступа.
- Обновили шаблоны для отображения элементов в зависимости от прав доступа пользователя.

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!

Успехов в разработке!

Вопросы для обсуждения

- Почему важно ограничивать доступ к определенным действиям только для администраторов?
- Ограничение доступа повышает безопасность приложения, предотвращает несанкционированные изменения данных и функций, гарантирует, что только уполномоченные пользователи могут выполнять критические действия, такие как добавление или редактирование мероприятий.
- Как декораторы и миксины помогают в управлении правами доступа в Django?
- Декораторы позволяют легко применять проверку прав доступа к функциональным представлениям, добавляя условия для выполнения действий. Миксины, используемые с классами-представлениями, предоставляют встроенные методы для проверки прав доступа, обеспечивая повторное использование кода и упрощая управление доступом на уровне классов.
- Какие преимущества дает использование встроенной системы разрешений Django при настройке ролей пользователей?

- Встроенная система разрешений Django обеспечивает гибкость и масштабируемость управления доступом, позволяя создавать кастомные разрешения, группировать пользователей и назначать им соответствующие права. Это упрощает администрирование и обеспечивает централизованный контроль над доступом к различным частям приложения.
-

Полезные ресурсы

- [Документация Django: Управление доступом пользователей](#)
- [Документация Django: Декораторы доступа](#)
- [Документация Django: Миксины доступа](#)
- [Видео-туториал по правам доступа в Django](#)

Практическое занятие 31: Введение в Django REST Framework, создание API

Цель занятия:

- Понять концепцию RESTful API и ее использование.
 - Научиться устанавливать и настраивать Django REST Framework (DRF) в проекте Django.
 - Создать API для вашего приложения с использованием DRF.
 - Научиться создавать сериализаторы и представления для обработки данных.
 - Научиться тестировать и использовать созданные API-эндпоинты.
-

Задачи занятия

1. Введение в RESTful API и Django REST Framework
 2. Установка и настройка Django REST Framework
 3. Создание сериализаторов для моделей
 4. Создание API-представлений
 5. Настройка URL-маршрутизации для API
 6. Тестирование API-эндпоинтов
-

Задача 1: Введение в RESTful API и Django REST Framework

1.1. Что такое RESTful API?

- **REST (Representational State Transfer)** — архитектурный стиль взаимодействия клиент-сервер, использующий стандартные HTTP-методы.
- **RESTful API** позволяет обмениваться данными между клиентом и сервером в формате JSON или XML.
- **Преимущества RESTful API:**
 - Легкость и простота использования.
 - Кроссплатформенность.
 - Стандартные методы HTTP (GET, POST, PUT, DELETE).

1.2. Что такое Django REST Framework?

- **Django REST Framework (DRF)** — мощный и гибкий инструмент для построения веб-API на основе Django.
 - **Особенности DRF:**
 - Простые и мощные сериализаторы данных.
 - Гибкие представления (Views) для обработки запросов.
 - Поддержка аутентификации и разрешений.
 - Возможность генерации интерактивной документации API.
-

Задача 2: Установка и настройка Django REST Framework

2.1. Установка DRF

Шаги:

1. Установите Django REST Framework с помощью pip:

```
pip install djangorestframework
```

2. Убедитесь, что установка прошла успешно:

```
pip show djangorestframework
```

2.2. Настройка проекта для использования DRF

Шаги:

1. Добавьте 'rest_framework' в список `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'rest_framework',
    'events', # Ваше приложение
]
```

2. (Опционально) Настройте глобальные настройки DRF:

```
# settings.py
```

```
REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.AllowAny',
    ],
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
        'rest_framework.authentication.BasicAuthentication',
    ],
}
```

Пояснение:

- Здесь мы устанавливаем разрешения по умолчанию (`AllowAny`), чтобы любой пользователь мог получить доступ к API.
- Вы можете настроить аутентификацию и разрешения позже, в зависимости от требований вашего проекта.

Задача 3: Создание сериализаторов для моделей

3.1. Что такое сериализаторы?

- **Сериализаторы** преобразуют сложные типы данных, такие как объекты моделей Django, в форматы данных, которые можно легко преобразовать в JSON или XML.
- **Сериализаторы** также обеспечивают валидацию входящих данных при создании или обновлении объектов.

3.2. Создание сериализатора для модели `Event`

Шаги:

1. Создайте файл `serializers.py` в приложении `events`:

```
touch events/serializers.py
```

2. Откройте `serializers.py` и добавьте следующий код:

```
# events/serializers.py

from rest_framework import serializers
from .models import Event

class EventSerializer(serializers.ModelSerializer):
    class Meta:
```

```
model = Event
fields = '__all__'
```

Пояснение:

- Мы создаем класс `EventSerializer`, который наследуется от `serializers.ModelSerializer`.
- Поле `fields = '__all__'` означает, что мы хотим включить все поля модели `Event`.

3. Создайте сериализатор для модели `Review` (если необходимо):

```
# events/serializers.py

from .models import Review

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = '__all__'
```

Задача 4: Создание API-представлений

4.1. Использование функциональных представлений API

Шаги:

1. Откройте `views.py` в приложении `events`.
2. Импортируйте необходимые модули:

```
# events/views.py

from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .serializers import EventSerializer
from .models import Event
```

3. Создайте представление для получения списка мероприятий:

```
# events/views.py

@api_view(['GET', 'POST'])
def api_event_list(request):
```

```

if request.method == 'GET':
    events = Event.objects.all()
    serializer = EventSerializer(events, many=True)
    return Response(serializer.data)
elif request.method == 'POST':
    serializer = EventSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data,
status=status.HTTP_201_CREATED)
    return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)

```

Пояснение:

- Декоратор `@api_view` определяет, какие методы HTTP поддерживаются (GET, POST).
- При GET запросе возвращаем список всех мероприятий.
- При POST запросе создаем новое мероприятие после валидации данных.

4. Создайте представление для получения, обновления и удаления конкретного мероприятия:

```

# events/views.py

@api_view(['GET', 'PUT', 'DELETE'])
def api_event_detail(request, pk):
    try:
        event = Event.objects.get(pk=pk)
    except Event.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = EventSerializer(event)
        return Response(serializer.data)
    elif request.method == 'PUT':
        serializer = EventSerializer(event, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
    elif request.method == 'DELETE':
        event.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

```

Пояснение:

- Обработываем `GET`, `PUT` и `DELETE` запросы для работы с конкретным объектом `Event`.

4.2. Использование классов-представлений API (опционально)

Шаги:

1. Импортируйте классы-представления:

```
# events/views.py

from rest_framework import generics
```

2. Создайте класс-представление для списка мероприятий:

```
# events/views.py

class EventListAPIView(generics.ListCreateAPIView):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

3. Создайте класс-представление для деталей мероприятия:

```
# events/views.py

class EventDetailAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

Пояснение:

- Классы-представления позволяют сократить код и предоставить более гибкую функциональность.

Задача 5: Настройка URL-маршрутизации для API

5.1. Добавление маршрутов API в `urls.py`

Шаги:

1. Откройте `urls.py` в приложении `events`.
2. Импортируйте представления API:

```
# events/urls.py

from . import views
```

3. Добавьте URL-маршруты для API:

Если вы используете функциональные представления:

```
# events/urls.py

urlpatterns += [
    path('api/events/', views.api_event_list, name='api_event_list'),
    path('api/events/<int:pk>', views.api_event_detail,
name='api_event_detail'),
]
```

Если вы используете классы-представления:

```
# events/urls.py

from django.urls import path
from .views import EventListAPIView, EventDetailAPIView

urlpatterns += [
    path('api/events/', EventListAPIView.as_view(),
name='api_event_list'),
    path('api/events/<int:pk>', EventDetailAPIView.as_view(),
name='api_event_detail'),
]
```

5.2. (Опционально) Использование маршрутизаторов DRF

Шаги:

1. Импортируйте `routers` из DRF:

```
# events/urls.py

from rest_framework import routers
```

2. Создайте маршрутизатор и зарегистрируйте представления:

```
# events/urls.py
```

```
router = routers.DefaultRouter()
router.register(r'api/events', views.EventViewSet)
```

3. Включите маршруты маршрутизатора:

```
# events/urls.py

urlpatterns += router.urls
```

4. Создайте EventViewSet в views.py:

```
# events/views.py

from rest_framework import viewsets

class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

Пояснение:

- Маршрутизаторы и ViewSets позволяют автоматизировать создание URL-маршрутов и упростить код представлений.

Задача 6: Тестирование API-эндпоинтов

6.1. Использование браузера для тестирования

- Перейдите по URL: <http://127.0.0.1:8000/api/events/>
- Вы должны увидеть список мероприятий в формате JSON.
- DRF предоставляет веб-интерфейс для взаимодействия с API через браузер.

6.2. Использование инструмента `curl`

Пример получения списка мероприятий:

```
curl -X GET http://127.0.0.1:8000/api/events/
```

Пример создания нового мероприятия:

```
curl -X POST -H "Content-Type: application/json" -d '{"title": "Новое мероприятие", "description": "Описание", "date": "2023-10-31"}'
http://127.0.0.1:8000/api/events/
```

6.3. Использование Postman или Insomnia

- **Postman** и **Insomnia** — популярные инструменты для тестирования API.
 - **Шаги:**
 - Установите Postman или Insomnia.
 - Создайте новый запрос к вашему API.
 - Тестируйте различные методы (GET, POST, PUT, DELETE).
-

Дополнительные задания

1. Добавление аутентификации и разрешений

Цель: Ограничить доступ к API только для аутентифицированных пользователей.

Шаги:

1. Настройте разрешения в `settings.py`:

```
# settings.py

REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

2. Добавьте маршруты для аутентификации в `urls.py`:

```
# events/urls.py

from rest_framework.auth_token import views as drf_views

urlpatterns += [
    path('api-token-auth/', drf_views.obtain_auth_token,
        name='api_token_auth'),
]
```

3. Установите `rest_framework.auth_token`:

```
pip install django-rest-framework-auth-token
```

4. Добавьте `'rest_framework.auth_token'` в `INSTALLED_APPS`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'rest_framework.authtoken',
]
```

5. Выполните миграции:

```
python manage.py migrate
```

6. Используйте токены для аутентификации при запросах к API.

2. Создание сериализаторов и представлений для модели

Review

- Создайте `ReviewSerializer` и соответствующие представления для работы с отзывами через API.
- Настройте вложенные сериализаторы, чтобы при получении мероприятия также получать связанные отзывы.

Заключение

Поздравляем! Вы успешно:

- Установили и настроили Django REST Framework в своем проекте.
- Создали сериализаторы для моделей.
- Реализовали API-представления для обработки запросов.
- Настроили маршрутизацию URL для API.
- Протестировали API-эндпоинты с помощью браузера и инструментов тестирования.

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!

Успехов в разработке!

Вопросы для обсуждения

- **Каковы преимущества использования Django REST Framework по сравнению с обычными представлениями Django?**
 - DRF предоставляет готовые инструменты для создания API, такие как сериализаторы, ViewSets и маршрутизаторы, что упрощает и ускоряет разработку. Он также поддерживает гибкие настройки аутентификации, разрешений и пагинации, что делает его более мощным и удобным для работы с RESTful API по сравнению с традиционными представлениями Django.
 - **Как DRF помогает в реализации RESTful API?**
 - DRF предоставляет структуру и компоненты, специально разработанные для создания RESTful API, включая сериализаторы для преобразования данных, ViewSets для обработки CRUD операций и маршрутизаторы для автоматической генерации URL. Это позволяет разработчикам легко создавать, управлять и масштабировать API, следуя принципам REST.
 - **Какие методы аутентификации поддерживает DRF?**
 - DRF поддерживает различные методы аутентификации, включая базовую аутентификацию, токен-аутентификацию, аутентификацию на основе сессий, JWT (JSON Web Tokens) и OAuth. Это позволяет выбрать наиболее подходящий метод для конкретных требований безопасности и удобства использования вашего API.
-

Полезные ресурсы

- [Документация Django REST Framework](#)
- [Tutorial: Django REST Framework](#)
- [Видео-туториал по Django REST Framework](#)
- [Демонстрация использования DRF](#)

Практическое занятие 32: Работа с сериализаторами и представлениями API

Цель занятия:

- Углубить знания по работе с Django REST Framework (DRF).
 - Научиться создавать сложные сериализаторы для вложенных данных и связанных моделей.
 - Освоить использование различных видов представлений (Views) в DRF, включая ViewSets и Generic Views.
 - Реализовать функциональность API для создания, чтения, обновления и удаления (CRUD) данных.
 - Понять, как добавить фильтрацию, поиск и пагинацию в API.
-

Задачи занятия

1. Повторение основ DRF и сериализаторов
 2. Создание вложенных сериализаторов для моделей `Event` и `Review`
 3. Использование ViewSets и маршрутизаторов
 4. Реализация фильтрации, поиска и пагинации
 5. Настройка разрешений и аутентификации
 6. Тестирование API с использованием Postman или другого инструмента
-

Задача 1: Повторение основ DRF и сериализаторов

1.1. Что такое сериализаторы в DRF?

- **Сериализаторы** в DRF позволяют преобразовывать сложные типы данных, такие как объекты моделей Django, в простые типы данных Python, которые могут быть легко преобразованы в формат JSON или XML.
- **Десериализация**: Преобразование входящих данных обратно в сложные типы, после проверки данных на корректность.

1.2. Типы представлений в DRF

- **Function-Based Views (FBV)**: Представления, основанные на функциях, с использованием декораторов `@api_view`.

- **Class-Based Views (CBV):** Представления, основанные на классах, используя `APIView`, `GenericAPIView` и их производные.
- **ViewSet:** Позволяют объединить логически связанные представления в один класс, облегчая маршрутизацию.

Задача 2: Создание вложенных сериализаторов для моделей `Event` и `Review`

2.1. Создание сериализатора для модели `Review`

Шаги:

1. Откройте `serializers.py` в приложении `events` и добавьте `ReviewSerializer`:

```
# events/serializers.py

from rest_framework import serializers
from .models import Event, Review

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ['id', 'name', 'email', 'rating', 'comment',
                  'created_at']
```

2. Обновите `EventSerializer`, чтобы включить связанные отзывы:

```
# events/serializers.py

class EventSerializer(serializers.ModelSerializer):
    reviews = ReviewSerializer(many=True, read_only=True)

    class Meta:
        model = Event
        fields = ['id', 'title', 'description', 'date', 'location',
                  'reviews']
```

Пояснение:

- Добавляем поле `reviews` в `EventSerializer`, которое использует `ReviewSerializer` для отображения связанных отзывов.
- Устанавливаем `many=True`, так как мероприятие может иметь несколько отзывов.

- Устанавливаем `read_only=True`, чтобы предотвратить создание отзывов через сериализатор мероприятия.

2.2. Создание сериализатора для создания и обновления отзывов

Шаги:

1. Создайте `ReviewCreateSerializer` для обработки создания отзывов:

```
# events/serializers.py

class ReviewCreateSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ['id', 'name', 'email', 'rating', 'comment', 'event']

    def create(self, validated_data):
        return Review.objects.create(**validated_data)
```

Пояснение:

- Этот сериализатор будет использоваться для создания новых отзывов.
- Включаем поле `event`, чтобы связать отзыв с мероприятием.

2.3. Обновление моделей для корректной работы с сериализаторами

Убедитесь, что в модели `Event` есть обратная связь с отзывами:

```
# events/models.py

class Event(models.Model):
    # ... существующие поля ...
    # Отзывы связаны через related_name='reviews' в модели Review
```

Пояснение:

- Если в модели `Review` вы не указали `related_name`, Django по умолчанию создаст связь `review_set`.
- Для удобства лучше явно указать `related_name` в поле `ForeignKey` модели `Review`:

```
# events/models.py

class Review(models.Model):
```

```
# ... существующие поля ...
event = models.ForeignKey(Event, related_name='reviews',
on_delete=models.CASCADE)
```

Не забудьте выполнить миграции после изменений в моделях:

```
python manage.py makemigrations
python manage.py migrate
```

Задача 3: Использование ViewSets и маршрутизаторов

3.1. Создание ViewSet для модели Event

Шаги:

1. Откройте `views.py` и создайте `EventViewSet`:

```
# events/views.py

from rest_framework import viewsets
from .models import Event
from .serializers import EventSerializer

class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

2. Создайте `ReviewViewSet` для модели `Review`:

```
# events/views.py

from .models import Review
from .serializers import ReviewSerializer, ReviewCreateSerializer

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.all()
    serializer_class = ReviewSerializer

    def get_serializer_class(self):
        if self.action in ['create', 'update', 'partial_update']:
```

```
        return ReviewCreateSerializer
    return ReviewSerializer
```

Пояснение:

- Используем `get_serializer_class`, чтобы использовать разные сериализаторы в зависимости от действия.

3.2. Настройка маршрутизатора для ViewSets

Шаги:

1. Откройте `urls.py` в приложении `events`:

```
# events/urls.py

from rest_framework import routers
from .views import EventViewSet, ReviewViewSet

router = routers.DefaultRouter()
router.register(r'api/events', EventViewSet)
router.register(r'api/reviews', ReviewViewSet)
```

2. Добавьте маршруты маршрутизатора в `urlpatterns`:

```
# events/urls.py

urlpatterns = [
    # ... ваши другие URL-маршруты ...
] + router.urls
```

Пояснение:

- Маршрутизатор автоматически создаст необходимые URL для ViewSets.

Задача 4: Реализация фильтрации, поиска и пагинации

4.1. Добавление пагинации

Шаги:

1. Настройте пагинацию в `settings.py`:

```
# settings.py

REST_FRAMEWORK = {
    # ... ваши другие настройки ...
    'DEFAULT_PAGINATION_CLASS':
    'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,
}
```

Пояснение:

- Устанавливаем размер страницы по умолчанию на 10 элементов.

4.2. Добавление фильтрации и поиска

Шаги:

1. Установите пакет `django-filter`:

```
pip install django-filter
```

2. Добавьте `django_filters` в `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'django_filters',
]
```

3. Настройте фильтрацию в `settings.py`:

```
# settings.py

REST_FRAMEWORK = {
    # ... ваши другие настройки ...
    'DEFAULT_FILTER_BACKENDS':
    ['django_filters.rest_framework.DjangoFilterBackend',
    'rest_framework.filters.SearchFilter'],
}
```

4. Добавьте фильтрацию и поиск в `EventViewSet`:

```
# events/views.py

class EventViewSet(viewsets.ModelViewSet):
```

```
queryset = Event.objects.all()
serializer_class = EventSerializer
filterset_fields = ['date', 'location']
search_fields = ['title', 'description']
```

Пояснение:

- `filterset_fields` позволяет фильтровать по указанным полям.
- `search_fields` позволяет выполнять поиск по указанным полям.

5. Добавьте фильтрацию и поиск в `ReviewViewSet` (если необходимо):

```
# events/views.py

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.all()
    # ... остальной код ...
    filterset_fields = ['rating', 'event']
    search_fields = ['name', 'comment']
```

4.3. Тестирование фильтрации и поиска

- Примеры запросов:
 - Фильтрация по дате:

```
GET /api/events/?date=2023-10-31
```

- Поиск по заголовку:

```
GET /api/events/?search=концерт
```

- Комбинированная фильтрация и пагинация:

```
GET /api/events/?date=2023-10-31&search=концерт&page=2
```

Задача 5: Настройка разрешений и аутентификации

5.1. Использование разрешений

Шаги:

1. Импортируйте класс разрешений в `views.py` :

```
# events/views.py

from rest_framework.permissions import IsAuthenticatedOrReadOnly
```

2. Примените разрешения к ViewSets:

```
# events/views.py

class EventViewSet(viewsets.ModelViewSet):
    # ... ваш существующий код ...
    permission_classes = [IsAuthenticatedOrReadOnly]
```

Пояснение:

- `IsAuthenticatedOrReadOnly` позволяет аутентифицированным пользователям создавать и изменять данные, а неаутентифицированным — только читать.

3. Примените аналогичные разрешения к `ReviewViewSet` :

```
# events/views.py

class ReviewViewSet(viewsets.ModelViewSet):
    # ... ваш существующий код ...
    permission_classes = [IsAuthenticatedOrReadOnly]
```

5.2. Настройка аутентификации через токены

Шаги:

1. Установите `django-rest-framework-simplejwt` :

```
pip install django-rest-framework-simplejwt
```

2. Настройте аутентификацию в `settings.py` :

```
# settings.py

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    # ... остальные настройки ...
}
```

3. Добавьте маршруты для получения токена в `urls.py`:

```
# events/urls.py

from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns += [
    path('api/token/', TokenObtainPairView.as_view(),
         name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(),
         name='token_refresh'),
]
```

Пояснение:

- `TokenObtainPairView` используется для получения токена доступа и обновления.
- `TokenRefreshView` используется для обновления токена доступа.

4. Тестирование аутентификации:

- **Получение токена:**

```
POST /api/token/
{
  "username": "your_username",
  "password": "your_password"
}
```

- **Использование токена при запросах:**

Добавьте заголовок `Authorization: Bearer your_access_token` к вашим запросам.

Задача 6: Тестирование API с использованием Postman или другого инструмента

6.1. Использование Postman для тестирования

- **Шаги:**
 - Откройте Postman и создайте новый запрос.
 - Установите метод запроса (GET, POST, PUT, DELETE).

- Введите URL эндпоинта API.
- Если требуется аутентификация, добавьте заголовок `Authorization` с токеном.
- Для методов `POST` и `PUT` добавьте тело запроса в формате JSON.
- Отправьте запрос и проанализируйте ответ.

6.2. Проверка функциональности

- Проверьте следующие сценарии:
 - Получение списка мероприятий с пагинацией, фильтрацией и поиском.
 - Получение деталей конкретного мероприятия.
 - Создание нового мероприятия (для аутентифицированных пользователей).
 - Обновление и удаление мероприятия (для аутентифицированных пользователей).
 - Работа с отзывами: получение списка, создание, обновление, удаление.
-

Дополнительные задания

1. Реализация вложенных маршрутов для отзывов мероприятия

Цель: Создать эндпоинты для работы с отзывами, привязанными к конкретному мероприятию.

Шаги:

1. Установите пакет `drf-nested-routers` :

```
pip install drf-nested-routers
```

2. Настройте вложенные маршруты в `urls.py` :

```
# events/urls.py

from rest_framework_nested import routers
from .views import EventViewSet, ReviewViewSet

router = routers.DefaultRouter()
router.register(r'api/events', EventViewSet)

events_router = routers.NestedDefaultRouter(router, r'api/events',
lookup='event')
```



```
events_router.register(r'reviews', ReviewViewSet, basename='event-
reviews')

urlpatterns = [
    # ... другие URL ...
] + router.urls + events_router.urls
```

3. Обновите ReviewViewSet для работы с вложенными маршрутами:

```
# events/views.py

class ReviewViewSet(viewsets.ModelViewSet):
    serializer_class = ReviewSerializer

    def get_queryset(self):
        return Review.objects.filter(event_id=self.kwargs['event_pk'])

    def perform_create(self, serializer):
        event = get_object_or_404(Event, pk=self.kwargs['event_pk'])
        serializer.save(event=event)
```

Пояснение:

- Теперь отзывы будут доступны по URL: `/api/events/{event_id}/reviews/`.

2. Добавление документации API с помощью Swagger или Redoc

Цель: Автоматически сгенерировать документацию для вашего API.

Шаги:

1. Установите пакет drf-yasg :

```
pip install drf-yasg
```

2. Добавьте его в INSTALLED_APPS в settings.py :

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'drf_yasg',
]
```

3. Настройте маршруты для документации в urls.py :

```
# events/urls.py

from django.urls import re_path
from rest_framework import permissions
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
    openapi.Info(
        title="Event API",
        default_version='v1',
        description="API для управления мероприятиями и отзывами",
    ),
    public=True,
    permission_classes=(permissions.AllowAny,),
)

urlpatterns += [
    re_path(r'^swagger(?P<format>\.json|\.yaml)$',
    schema_view.without_ui(cache_timeout=0), name='schema-json'),
    re_path(r'^swagger/$', schema_view.with_ui('swagger',
    cache_timeout=0), name='schema-swagger-ui'),
    re_path(r'^redoc/$', schema_view.with_ui('redoc', cache_timeout=0),
    name='schema-redoc'),
]
```

4. Проверьте доступность документации:

- Swagger UI: <http://127.0.0.1:8000/swagger/>
- Redoc: <http://127.0.0.1:8000/redoc/>

Заключение

Поздравляем! Вы успешно:

- Создали сложные сериализаторы для вложенных и связанных моделей.
- Использовали ViewSets и маршрутизаторы для упрощения кода и маршрутизации.
- Реализовали фильтрацию, поиск и пагинацию в вашем API.
- Настроили разрешения и аутентификацию для защиты API.
- Тестировали ваш API с помощью инструментов, таких как Postman.
- Изучили дополнительные возможности DRF для создания мощного и гибкого API.

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!

Успехов в разработке!

Вопросы для обсуждения

- **Почему сериализаторы важны в Django REST Framework?**
 - Сериализаторы позволяют преобразовывать сложные типы данных, такие как модели Django, в форматы данных, пригодные для передачи по сети (например, JSON), и обратно. Это обеспечивает эффективную и безопасную передачу данных между клиентом и сервером, а также валидацию входящих данных. Без сериализаторов было бы сложно управлять преобразованием и проверкой данных, что могло бы привести к ошибкам и уязвимостям в приложении.
 - **В чем разница между ViewSets и Generic Views в DRF, и когда использовать каждый из них?**
 - ViewSets объединяют несколько представлений (например, list, create, retrieve, update, destroy) в одном классе, упрощая маршрутизацию и уменьшая количество кода. Это особенно полезно для стандартных CRUD операций, позволяя быстро создавать API без необходимости писать отдельные представления для каждого действия. Generic Views предоставляют более гибкий подход для создания отдельных представлений с определенной функциональностью, что полезно, когда требуется более детальная настройка или нестандартное поведение. Использовать ViewSets рекомендуется для стандартных API, а Generic Views — когда необходима более тонкая настройка или специфические действия.
 - **Как реализовать фильтрацию и пагинацию в Django REST Framework?**
 - Для реализации фильтрации данных можно использовать пакет django-filter вместе с DRF. Необходимо установить django-filter, добавить его в INSTALLED_APPS и настроить DEFAULT_FILTER_BACKENDS в settings.py. Затем в ViewSet определить поля, по которым будет производиться фильтрация, используя filterset_fields. Для пагинации нужно настроить параметры пагинатора в settings.py, указав DEFAULT_PAGINATION_CLASS и PAGE_SIZE. DRF автоматически применит эти настройки к API-эндпоинтам, обеспечивая удобную навигацию по большим наборам данных.
-

Полезные ресурсы

- [Документация Django REST Framework](#)
- [Документация по сериализаторам DRF](#)

- [Документация по ViewSets и маршрутизаторам](#)
- [drf-nested-routers](#)
- [drf-yasg: Генерация документации Swagger](#)